
DecentSampler

Release 1.23.5

David Hilowitz

Jul 10, 2026

CONTENTS

1	Contents	3
1.1	File Format Overview	3
1.2	The <ui> element	4
1.3	The <groups> element (required)	18
1.4	The <effects> element	36
1.5	The <midi> element	48
1.6	The <noteSequences> element	50
1.7	The <modulators> element	51
1.8	The <tags> element	61
1.9	The <buses> element	62
1.10	Appendix A: The Color Format	64
1.11	Appendix B: The <binding> element	64
1.12	Appendix C: Boilerplate .dspreset File	77
1.13	Useful Tutorials and Resources	79

DecentSampler is a free sampling plug-in that allows music composers to use multi-samples in the DecentSampler format. This document is a guide to creating samples in that format.

If you have no familiarity with the DecentSamples format, [this video](#) is a great place to start. After that, you'll want to start at the [File Format Overview](#) to learn a bit more about how to create sample libraries.

Note: This project is under active development.

CONTENTS

1.1 File Format Overview

At its core each DecentSampler sample library consists of two things: a folder containing a bunch of assets like audio files and pictures, and a single text file (called a **dspreset** file) which describes how the engine should use all of those files. This reference document is a guide to creating **dspreset** files.

dspreset files are just XML files. As such, each one begins with an XML declaration:

```
<?xml version="1.0" encoding="UTF-8"?>
```

1.1.1 The top-level <DecentSampler> element (required)

At the top level of every **dspreset** file is a <DecentSampler> element. Every file **must** have one. Here is a list of attributes:

- **minVersion** (optional): This is the minimum version on which this preset is known to run. If a user is running an old version of DS, and a developer has specified a minVersion for their instrument, a dialog box will show up telling users that their version is outdated and that they should upgrade in order to get the full effect. They *can* then choose to ignore this warning or hit download. The dialog box does not show up for iOS users as most of them have auto-updates turned on.

Example:

```
<?xml version="1.0" encoding="UTF-8"?>
<DecentSampler minVersion="1.0.0">
  <!-- More tags go here. :) -->
</DecentSampler>
```

Underneath the top-level <DecentSampler> element you can put any number of other elements, all of which are described in the sections that follow.

1.2 The <ui> element

The <ui> element is how you specify a user interface for your instrument. Each **dspreset** should have at most one <ui> element. There are several important attributes:

- **coverArt** (optional): A relative or absolutely path to a cover art image to use. After the first time this library is opened, this will get displayed on the “My Libraries” tab.
- **bgImage** (optional): A relative or absolutely path to a background image to use.
- **bgColor** (required): An eight digit hex value indicating the background color to be used for the background of the UI. This color will be drawn underneath any background image specified by bgimage.
- **width** (required): The width of your user interface. Recommended value: 812.
- **height** (required): The height of your user interface. Recommended value: 375.

Example:

```
<DecentSampler>
  <ui width="812" height="375">
    <tab name="main">
      <labeled-knob x="560" y="0" label="Tone" type="float" minValue="60" maxValue="22000
↪ "
          textColor="FF000000" value="22000.0" uid="y8AA4uuURh3">
        <binding type="effect" level="instrument" position="0" parameter="FX_FILTER_
↪ FREQUENCY"/>
      </labeled-knob>
      <label x="360" y="0" width="50" height="30" text="Reverb"/>
      <control x="360" y="30" parameterName="Reverb" type="float" minValue="0" maxValue=
↪ "1" textColor="FF000000" value="0.5">
        <!-- Your <binding /> elements should go here -->
      </control>
    </tab>
  </ui>
</DecentSampler>
```

1.2.1 The <tab> element

The <tab> element lives underneath the <ui> element. Every <ui> must have at most one <tab> element.

Attributes:

- **name** (optional): An optional name to be associated with this tab. This is currently not displayed anywhere.

1.2.2 The <button> element

The <button> element allows you to create a button within your UI. It lives underneath the <tab> element. There are two styles of buttons: text buttons and image buttons.

Attributes:

Attribute	Required	Description	Default
x	(required)	The x position of the menu	None
y	(required)	The y position of the menu	None
width	(required)	The width of the menu	None
height	(required)	The height of the menu	None
value	(optional)	This is the 0-based index of the button state that is currently selected. A value of 0 means that the first state is active.	0
style	(optional)	The type of button we want. There are two valid values: <code>text</code> , <code>image</code> .	<code>text</code>
mainImage	(required for image buttons)	For image buttons only. The path of the main image to display for this button. This can also be set at the state level so that it only applies to a specific state.	None
hoverImage	(optional)	For image buttons only. The path of the main image to display when the user hovers their mouse over this button. This can also be set at the state level so that it only applies to a specific state.	None
clickImage	(optional)	For image buttons only. The path of the main image to display when the user clicks down on this button. This can also be set at the state level so that it only applies to a specific state.	None
disabled	(optional)	The opacity of the button when it is disabled. This is a floating point value between 0 and 1. Default: 0.5	0.5
visible	(optional)	This controls whether or not this button is visible. There are two valid values: <code>true</code> , <code>false</code> .	<code>true</code>
enabled	(optional)	This controls whether or not this button is enabled. There are two valid values: <code>true</code> , <code>false</code> .	<code>true</code>
tags	(optional)	A comma-separated list of tags to be associated with this button. These can be used in conjunction with any binding that takes a <code>controlIndex</code> (instead of the <code>controlIndex</code>)	None
tooltip	(optional)	A tool tip to display when the user hovers over this control.	

Example:

```
<button x="10" y="40" width="120" height="30" style="image" value="0" mainImage=
  ↳ "samples/ButtonMainImage.png" hoverImage="samples/ButtonHoverImage.png" clickImage=
  ↳ "samples/ButtonSelectedImage.png">
  <!-- Your button states go here -->
</button>
```

The <state> element

In order for your button to work, it must contain at least one <state> elements.

Attributes:

At-tribute	Required	Description
name	(required for text buttons)	The text to display on a text button when this state is active
mainIma	(required for image buttons)	For image buttons only. The path of the main image to display for this button when the button is in the current state.
hoverIm	(optional)	For image buttons only. The path of the image to display when the user hovers their mouse over this button when the button is in the current state.
clickIm	(optional)	For image buttons only. The path of the image to display when the user clicks down on this button when the button is in the current state.

In order to have your <button> elements actually do something useful, you need to attach bindings to them. Here's an example:

```
<button x="10" y="30" width="70" height="50" style="image" value="0" >
  <state name="English" mainImage="samples/EFlag_MainImage.png" hoverImage="samples/
↳ EFlag_HoverImage.png" clickImage="samples/EFlag_SelectedImage.png">
    <!-- Turn on this group -->
    <binding type="general" level="group" position="0" parameter="ENABLED"
↳ translation="fixed_value" translationValue="true" />
    <!-- Turn off this group -->
    <binding type="general" level="group" position="1" parameter="ENABLED"
↳ translation="fixed_value" translationValue="false" />
  </state>
  <state name="French" mainImage="Samples/FFlag_MainImage.png" hoverImage="Samples/
↳ FFlag_HoverImage.png" clickImage="Samples/FFlag_SelectedImage.png">
    <!-- Turn off this group -->
    <binding type="general" level="group" position="0" parameter="ENABLED"
↳ translation="fixed_value" translationValue="false" />
    <!-- Turn on this group -->
    <binding type="general" level="group" position="1" parameter="ENABLED"
↳ translation="fixed_value" translationValue="true" />
  </state>
</button>
```

As you can see, this example uses a button to switch between two groups. You'll note the liberal use of the `fixed_value` translation mode above. This means that when any of these options are selected, a fixed predetermined value is used for the value of that binding.

1.2.3 The <image> element

The <image> element allows you to place a static image into your user interface. It lives underneath the <tab> element. Attributes:

- **x** (required): The x position of your image where (0,0) is the top-left corner
- **y** (required): The y position of your image where (0,0) is the top-left corner
- **width** (required): The width in pixels of the image component
- **height** (required): The height in pixels of the image component
- **path** (required): The relative path of the image file to show in this component
- **aspectRatioMode** (required): Whether or not the engine should preserve the aspect ratio of the image. Note: regardless of these settings, you still need to specify a width and height for your image element. Valid values: `preserve`, `stretch`. Default value is `preserve`.
- **opacity** (optional): The opacity of the image. This is a floating point value between 0 and 1. Default: 1
- **visible** (optional): This controls whether or not this image is visible. There are two valid values: `true` (default), `false`.
- **tooltip** (optional): A tool tip to display when the user hovers over this image.

1.2.4 The <multiFrameImage> element

The <multiFrameImage> element allows you to play a sequence of images as an animation. The expectation is that all the frames of the animation will be loaded in a single image, arranged in a strip – either horizontal or vertical. This is the same format as is used by the custom knobs above. It lives underneath the <tab> element. Attributes:

- **x** (required): The x position of your image where (0,0) is the top-left corner
- **y** (required): The y position of your image where (0,0) is the top-left corner
- **width** (required): The width in pixels of the image component
- **height** (required): The height in pixels of the image component
- **path** (required): The relative path of the image file to show in this component
- **numFrames** (required): The number of frames in the animation
- **frameRate** (required): The frame rate of the animation in frames per second. The maximum supported frame rate is 24 frames per second.
- **opacity** (optional): The opacity of the animation. This is a floating point value between 0 and 1. Default: 1
- **sourceFormat** (required): The orientation of the frames within the image strip. Valid values: `horizontal_image_strip`, `vertical_image_strip`.
- **playbackMode** (optional): The direction in which the animation should play. Valid values: `forward_loop`, `forward_once`, `reverse_loop`, `reverse_once`, `ping_pong_loop` (forth and back), and `stopped`. Default value is `forward_loop`.
- **visible** (optional): This controls whether or not this image is visible. There are two valid values: `true` (default), `false`.
- **tags** (optional): A comma-separated list of tags to be associated with this image. These can be used in conjunction with any binding that takes a `controlIndex` (instead of the `controlIndex`)
- **tooltip** (optional): A tool tip to display when the user hovers over this image.

Example:

```
<multiFrameImage x="10" y="10" width="64" height="64" path="Images/Animation.png"
↳numFrames="31" imageStripOrientaton="vertical" frameRate="24" playbackMode="forward"/>
```

1.2.5 The <label> element

The <label> element allows you to place a static block of text into your user interface. It lives underneath the <tab> element. Attributes:

- **x** (required): The x position of your control where (0,0) is the top-left corner
- **y** (required): The y position of your control where (0,0) is the top-left corner
- **text** (required): The actual text that should be displayed as part of the label.
- **textColor** (optional): An 8 digit hex value indicating the text color to be used for the label. See [Appendix A](#) for an explanation on these hex values.
- **textSize** (optional): A font size for the text label. Default: 12
- **width** (required): The width in pixels of the label.
- **height** (required): The height in pixels of the label.
- **vAlign** (optional): The vertical alignment of the text within the box described by the width and height attributes. Valid values: top,bottom, center. Default is center.
- **hAlign** (optional): The horizontal alignment of the text within the box described by the width and height attributes. Valid values: left,right, center. Default is center.
- **orientation** (optional): The orientation of the text within the box described by the width and height attributes. Valid values: horizontal, vertical_up, vertical_down. Default is horizontal.
- **tags** (optional): A comma-separated list of tags to be associated with this label. These can be used in conjunction with any binding that takes a `controlIndex` (instead of the `controlIndex`)
- **visible** (optional): This controls whether or not this text label is visible. There are two valid values: `true` (default), `false`.
- **tooltip** (optional): A tool tip to display when the user hovers over this label.

A label's text can also be set dynamically using bindings using the `TEXT` binding parameter name.

1.2.6 The <rectangle> element

The <rectangle> element allows you to draw filled rectangles with optional borders in your UI. It lives underneath the <tab> element. This is useful for creating backgrounds, panels, dividers, or decorative elements.

Attributes:

Attribute	Re-quired	Description	Default
x	(re-quired)	The x position of the rectangle's top-left corner	None
y	(re-quired)	The y position of the rectangle's top-left corner	None
width	(re-quired)	The width of the rectangle in pixels	None
height	(re-quired)	The height of the rectangle in pixels	None
fillColor	(op-tional)	An 8-character hex value for the fill color. See Appendix A for format details. Use #00000000 for transparent fill.	#FF808080
borderColor	(op-tional)	An 8-character hex value for the border color. See Appendix A for format details.	#00000000
borderThick	(op-tional)	The thickness of the border in pixels. Set to 0 for no border.	0
visible	(op-tional)	Controls whether the rectangle is visible. Valid values: true, false.	true

Example:

```
<tab name="main">
  <!-- Background panel -->
  <rectangle x="0" y="0" width="812" height="375" fillColor="#FF2a2a2a"/>

  <!-- Panel with border -->
  <rectangle x="30" y="70" width="240" height="135" fillColor="#FF2d2d2d" borderColor="
  ↪ #FFe74c3c" borderThickness="1"/>

  <!-- Decorative stripe -->
  <rectangle x="0" y="0" width="8" height="255" fillColor="#FF4a90e2"/>
</tab>
```

1.2.7 The <line> element

The <line> element allows you to draw lines in your UI. It lives underneath the <tab> element. This is useful for creating dividers, borders, or decorative elements.

Attributes:

Attribute	Re-quired	Description	Default
x1	(re-quired)	The x coordinate of the line's start point	None
y1	(re-quired)	The y coordinate of the line's start point	None
x2	(re-quired)	The x coordinate of the line's end point	None
y2	(re-quired)	The y coordinate of the line's end point	None
lineColor	(op-tional)	An 8-character hex value for the line color. See Appendix A for format details.	#FFFFFF
lineThickness	(op-tional)	The thickness of the line in pixels	1.0
visible	(op-tional)	Controls whether the line is visible. Valid values: <code>true</code> , <code>false</code> .	<code>true</code>

Example:

```
<tab name="main">
  <!-- Horizontal divider line -->
  <line x1="30" y1="95" x2="270" y2="95" lineColor="#FFe74c3c" lineThickness="2"/>

  <!-- Vertical divider line -->
  <line x1="280" y1="70" x2="280" y2="205" lineColor="#FF555555" lineThickness="1"/>

  <!-- Diagonal decorative line -->
  <line x1="790" y1="155" x2="804" y2="215" lineColor="#804a90e2" lineThickness="2"/>
</tab>
```

1.2.8 The <oscilloscope> element

The <oscilloscope> element displays a live waveform of the plugin's audio output inside your UI. It lives underneath the <tab> element. The waveform is a mono mix of the left and right output channels and refreshes at approximately 20 fps.

Attributes:

Attribute	Re- quired	Description	Default
x	(re- quired)	The x position of the element's top-left corner	None
y	(re- quired)	The y position of the element's top-left corner	None
width	(re- quired)	The width of the element in pixels	None
height	(re- quired)	The height of the element in pixels	None
backgroundCc	(op- tional)	An 8-character hex value for the background fill color. See Appendix A for format details.	#FF000000
waveColor	(op- tional)	An 8-character hex value for the waveform line color. See Appendix A for format details.	#FF00FF00
lineThicknes	(op- tional)	The stroke width of the waveform line in pixels	1.5
showCenterLi	(op- tional)	When set to <code>true</code> , draws a faint horizontal center line at the zero-crossing level. Valid values: <code>true</code> , <code>false</code> .	<code>false</code>
visible	(op- tional)	Controls whether the oscilloscope is visible. Valid values: <code>true</code> , <code>false</code> .	<code>true</code>

Example:

```
<tab name="main">
  <oscilloscope x="10" y="10" width="300" height="80"
    backgroundColor="#FF101010"
    waveColor="#FF00FF88"
    lineThickness="1.5"
    showCenterLine="true"/>
</tab>
```

1.2.9 The <labeled-knob> and <control> elements

The <labeled-knob> and <control> elements live underneath the <tab> element. These tags correspond to user controls (usually round radial dials) that can be used as part of a UI. These two element types are the same except that <labeled-knob> elements contain built-in labels, where as <control> elements do not. Every tab can have many <control> or <labeled-knob> elements underneath it.

For precise UI creation, it may be advisable to use a combination of <control> & <label> elements rather than <labeled-knob>.

Attributes:

- **x** (required): The x position of your control where (0,0) is the top-left corner
- **y** (required): The y position of your control where (0,0) is the top-left corner
- **width** (required): The width in pixels of the knob + label.
- **height** (required): The height in pixels of the knob + label.
- **parameterName** (required): In a situation where the sampler does not have enough room to display the full UI, a shrunken down version of the UI will be used. In such situations, this control will be labeled using the `parameterName`. It is good practice to always include a `parameterName`. If not `parameterName` is specified and a value `label` is specified, then that will be used instead.

- **style** (optional): The specific kind of control that is created. The following values are supported: `linear_bar`, `linear_bar_vertical`, `linear_horizontal`, `linear_vertical`, `rotary`, `rotary_horizontal_drag`, `rotary_horizontal_vertical_drag`, `rotary_vertical_drag`, `custom_skin_vertical_drag`, `custom_skin_horizontal_drag`. Default: `rotary_vertical_drag`.
- **showLabel** (optional): A true/false value dictating whether or not a built-in label should be displayed. Default: true for `<labeled-knob>` and false for `<control>` elements
- **label** (optional): If `showLabel` is true, the actual text that should be displayed above the control.
- **parameterName** (required): In a situation where the sampler does not have enough room to display the full UI, a shrunk down version of the UI will be used. In such situations, this control will be labeled using the `parameterName`. It is good practice to always include a `parameterName`.
- **minValue** (optional): The minimum value of your control. Default: 0
- **maxValue** (optional): The maximum value of your control. Default: 1
- **value** (optional): The initial value of your control. Default: 0
- **defaultValue** (optional): If a user double-clicks on the control, the control's value will be set to this default value. If no default value is specified, then nothing will happen on double-click.
- **valueType** (optional): There are several possible values for this: `float` which yields numbers with two decimal points, `integer` which yields whole numbers, `multi_state`, which allows the user to choose between multiple states, and `musical_time` which yields musical time increments in beats and measures. In order to use the `multi_state` option, you must also specify several `<state name="something">` type elements. Default: `float`
- **textColor** (optional): An 8 digit hex value indicating the text color to be used for the label. See [Appendix A](#) for an explanation on these hex values.
- **textSize** (optional): A font size for the text label. Default: 12
- **trackForegroundColor** (optional): An 8 digit hex value indicating the foreground color to use for the knob track. See [Appendix A](#) for an explanation on these hex values.
- **trackBackgroundColor** (optional): An 8 digit hex value indicating the background color to use for the knob track. See [Appendix A](#) for an explanation on these hex values.
- **tags** (optional): A comma-separated list of tags to be associated with this control. These can be used in conjunction with any binding that takes a `controlIndex` (instead of the `controlIndex`)
- **visible** (optional): This controls whether or not this control is visible. There are two valid values: `true` (default), `false`.
- **enabled** (optional): This controls whether or not this control is enabled. There are two valid values: `true` (default), `false`.
- **tooltip** (optional): A tool tip to display when the user hovers over this control.
- **snapMode** (optional): This attribute controls how the control should snap to values. Valid values: `none`, `whole_numbers`, `tenths`, `hundredths`, `thousandths`, and `stop_points`. Default: `none`.
- **snapStopPoints** (optional): A comma-separated list of values that the control should snap to when `snapMode` is set to `stop_points`. Default: `none`.
- **defeatSnapWithShift** (optional): A true/false value indicating whether or not the user can defeat the snap-to-value behavior by holding down the shift key. Default: `false`.

It is also possible to use custom control graphics using the following attributes:

- **customSkinImage** (optional): This is path to an image to use for the control. This is expected to be a JPEG or PNG in KnobMan format. A huge gallery of compatible knobs can be found [here](#).

- **customSkinNumFrames** (optional): The number of animation frames contained in the KnobMan image pointed to by customSkinImage.
- **customSkinImageOrientation** (optional): The orientation of the frames within the KnobMan image pointed to by customSkinImage. Valid values: `horizontal`, `vertical`. Default: `vertical`.
- **mouseDragSensitivity** (optional): An integer number describing how sensitive the control should be to mouse drags. The higher the number, the less sensitive the control will be to mouse movements.

If you are using custom knobs, it's important that you specify a `style=` value of either `custom_skin_vertical_drag` or `custom_skin_horizontal_drag`.

Example:

```
<DecentSampler>
  <ui>
    <tab>
      <labeled-knob x="560" y="0" label="Tone" type="float" minValue="60" maxValue="22000
↪ text="FF000000" value="22000.0">
        <!-- Your <binding /> elements should go here -->
      </labeled-knob>
      <label x="360" y="0" width="50" height="30" text="Reverb"/>
      <control x="360" y="30" parameterName="Reverb" type="float" minValue="0" maxValue=
↪ "1" text="FF000000" value="0.5" style="custom_skin_vertical_drag" customSkinImage=
↪ "Samples/ENIGMA-nolight.png" customSkinNumFrames="31" customSkinImageOrientation=
↪ "horizontal" mouseDragSensitivity="100">
        <!-- Your <binding /> elements should go here -->
      </control>
    </tab>
  </ui>
</DecentSampler>
```

To learn how to make knobs actually control parameters of your instrument, see “Appendix B: Bindings” section below.

Multi-State Controls

As of 1.13.6, it is now possible to use the `multi_state` value type for controls. This allows you to create controls that can switch between multiple states, rather than just a continuous range of values. Here is an example of how you might use one of these:

```
<ui>
  <tab>
    <control x="80" y="10" style="rotary" width="84" height="84" trackForegroundColor=
↪ "FFF06C55" trackBackgroundColor="66999999" parameterName="Language" valueType="multi_
↪ state" value="10.0">
      <state name="English" mainImage="samples/EFlag_MainImage.png" hoverImage=
↪ "samples/EFlag_HoverImage.png" clickImage="samples/EFlag_SelectedImage.png">
        <!-- Various <binding />s that get triggered when the first state is selected -
↪ -->
      </state>
      <state name="French" mainImage="Samples/FFlag_MainImage.png" hoverImage="Samples/
↪ FFlag_HoverImage.png" clickImage="Samples/FFlag_SelectedImage.png">
        <!-- Various <binding />s that get triggered when the second state is selected
↪ -->
      </state>
```

(continues on next page)

(continued from previous page)

```

    </control>
  </tab>
</ui>

```

1.2.10 The <menu> element

The <menu> element allows you to create a drop-down menu within your UI.

Attributes:

Attribute	Re-quired	Description
x	(re-quired)	The x position of the menu
y	(re-quired)	The y position of the menu
width	(re-quired)	The width of the menu
height	(re-quired)	The height of the menu
value	(op-tional)	This is the 1-based index of the menu option that is currently selected. NOTE: Index numbers for menu items start at 1. A value of 0 means that no item is selected.
tags	(op-tional)	A comma-separated list of tags to be associated with this menu. These can be used in conjunction with any binding that takes a <code>controlIndex</code> (instead of the <code>controlIndex</code>)
visible	(op-tional)	This controls whether or not this menu is visible. There are two valid values: <code>true</code> , <code>false</code> . <code>true</code>
enabled	(op-tional)	This controls whether or not this menu is enabled. There are two valid values: <code>true</code> , <code>false</code> . <code>true</code>
textColor	(op-tional)	A hex ARGB color value for the menu text (e.g., “FFFFFFF” for white text).
background-Color	(op-tional)	A hex ARGB color value for the menu background (e.g., “FF333333” for dark gray).
highlighted-TextColor	(op-tional)	A hex ARGB color value for the highlighted (selected) menu text (e.g., “FF000000” for black text).
highlighted-Background-Color	(op-tional)	A hex ARGB color value for the highlighted (selected) menu background (e.g., “FFCCCCC” for light gray).
vAlign	(op-tional)	The vertical alignment of the menu text. Valid values are “top”, “center”, “bottom”. “center”
hAlign	(op-tional)	The horizontal alignment of the menu text. Valid values are “left”, “center”, “right”. “left”
tooltip	(op-tional)	A tool tip to display when the user hovers over this control.

Example:

```

<menu x="10" y="40" width="120" height="30" value="2">
  <!-- Your menu options go here -->

```

(continues on next page)

(continued from previous page)

</menu>

The <option> element

In order for your drop-down menu to have options, it must contain <option> elements.

Attributes:

Attribute	Required	Description
name	(required)	The name of this element

That's right. The <option> element has only one attribute. In order to have your <option> elements actually do something useful, you need to attach bindings to them. Here's an example:

```

<menu x="10" y="40" width="120" height="30" requireSelection="true" placeholderText=
↳ "Choose..." value="2">
  <option name="Menu Option 1">
    <!-- Set the text of a label element -->
    <binding type="control" level="ui" position="2" parameter="TEXT" translation=
↳ "fixed_value" translationValue="You chose the first option." />
    <!-- Turn on this group -->
    <binding type="general" level="group" position="0" parameter="ENABLED"
↳ translation="fixed_value" translationValue="true" />
    <!-- Turn off this group -->
    <binding type="general" level="group" position="1" parameter="ENABLED"
↳ translation="fixed_value" translationValue="false" />
  </option>
  <option name="Menu Option 2">
    <!-- Set the text of a label element -->
    <binding type="control" level="ui" position="2" parameter="TEXT" translation=
↳ "fixed_value" translationValue="You chose the second option." />
    <!-- Turn off this group -->
    <binding type="general" level="group" position="0" parameter="ENABLED"
↳ translation="fixed_value" translationValue="false" />
    <!-- Turn on this group -->
    <binding type="general" level="group" position="1" parameter="ENABLED"
↳ translation="fixed_value" translationValue="true" />
  </option>
</menu>

```

As you can see, this example uses a menu to switch between two groups. It also sets the text of a text label. You'll note the liberal use of the new `fixed_value` translation mode above. This means that when any of these options are selected, a fixed predetermined value is used for the value of that binding.

1.2.11 The <xyPad> element

The <xyPad> element allows you to create a two-dimensional pad control within your UI. This control functions almost like two sliders, one for the x-axis and one for the y-axis. Each axis has a range from 0.0 to 1.0. Each axis can be bound to a different parameter. The <xyPad> element lives underneath the <tab> element.

Attributes:

- **x** (required): The x position of your control where (0,0) is the top-left corner
- **y** (required): The y position of your control where (0,0) is the top-left corner
- **width** (required): The width in pixels of the control.
- **height** (required): The height in pixels of the control.
- **markerDiameter** (optional): The diameter of the marker that moves around the control. Default: 10
- **markerOutlineColor** (optional): An 8 digit hex value indicating the color of the marker's outline. See [Appendix A](#) for an explanation on these hex values.
- **markerFillColor** (optional): An 8 digit hex value indicating the color of the marker's fill. See [Appendix A](#) for an explanation on these hex values.
- **outlineColor** (optional): An 8 digit hex value indicating the color of the control's outline. See [Appendix A](#) for an explanation on these hex values.
- **bgColor** (optional): An 8 digit hex value indicating the color of the control's background. See [Appendix A](#) for an explanation on these hex values.
- **tooltip** (optional): A tool tip to display when the user hovers over this control.
- **xValue** (optional): The initial value of the x-axis. Default: 0
- **yValue** (optional): The initial value of the y-axis. Default: 0
- **tags** (optional): A comma-separated list of tags to be associated with this control. These can be used in conjunction with any binding that takes a `controlIndex` (instead of the `controlIndex`)
- **visible** (optional): This controls whether or not this control is visible. There are two valid values: `true` (default), `false`.
- **enabled** (optional): This controls whether or not this control is enabled. There are two valid values: `true` (default), `false`.

The <x> and <y> elements

Below the <xyPad> element, you can specify <x> and <y> elements. These elements are used to specify the bindings for the x and y axes of the control.

Example:

```
<xyPad x="10" y="10" width="300" height="100" parameterName="Pad" xValue="0.5" yValue="0.5"
  bgColor="77FFCC00" markerFillColor="FFFFFFF" outlineColor="77FFFFFFF">
  <x>
    <binding type="amp" level="group" groupIndex="0" parameter="AMP_VOLUME" translation=
  ↪ "linear" translationOutputMin="0" translationOutputMax="1" />
    <binding type="amp" level="group" groupIndex="1" parameter="AMP_VOLUME" translation=
  ↪ "linear" translationOutputMin="1" translationOutputMax="0" />
  </x>
  <y>
```

(continues on next page)

(continued from previous page)

```

<binding type="effect" level="instrument" effectIndex="0" parameter="FX_FILTER_
↪FREQUENCY"
  translation="table"
  translationTable="0,33;0.3,150;0.4,450;0.5,1100;0.7,4100;0.9,11000;1.0001,22000"/
↪>
</y>
</xyPad>

```

1.2.12 The <keyboard> element

The <keyboard> element lives underneath the <ui> element. This is where you specify settings relating to the on-screen keyboard. There should be only one <keyboard> element in your preset file. At this point, the only settings are color ranges which are specified using <color> sub-elements.

The <color> element

You can use <color> elements to change the color of portions of the on-screen keyboard. You can have as many <color> elements as you like. Only white keys are affected. It's worth noting that colors specified in the <color> elements are overlayed on top of the white keys using a 75% transparency, so choose your colors accordingly. This is done to preserve the readability of the key labels.

```

<DecentSampler>
  <ui>
    <!-- Other stuff here -->
    <keyboard>
      <color loNote="36" hiNote="50" color="FF2C365E" />
      <color loNote="51" hiNote="57" color="FF6D9DC5" />
      <color loNote="58" hiNote="67" color="FFCCF3F5" />
      <color loNote="68" hiNote="73" color="FFE8DA9B" />
      <color loNote="74" hiNote="84" color="FFD19D61" />
    </keyboard>
  </ui>
  <!-- Other stuff here -->
</DecentSampler>

```

At-tribute	Description
loNote (re-quired)	The bottom of the range for which this color should be displayed. Format: MIDI Note number.
hiNote (re-quired)	The top of the range for which this color should be displayed. Format: MIDI Note number.
color (re-quired)	A text representation of the color to be used for this key range. See Appendix A for an explanation on these hex values.

1.3 The <groups> element (required)

In this section, we'll find elements that pertain to samples and sample-mapping.

Every **dspreset** file should have one and only one <groups> element. This is where you specify the samples that make up your sample library. This element lives right underneath the top-level <DecentSampler> element. The basic structure is this:

```
<DecentSampler>
  <groups>
    <group>
      <sample /> <!-- This is where -->
      <sample /> <!-- the samples -->
      <sample /> <!-- get defined -->
    </group>
  </groups>
</DecentSampler>
```

Attribute	Description
volume (optional)	The volume of the instrument as a whole. This will be reflected in the UI in the top-right corner. Value can be in linear 0.0-1.0 or in decibels. If it's in decibels you must append dB after the value (example: "3dB"). Default: 1.0 (no volume change)
globalpitch (optional)	Global pitch adjustment for changing note pitch. In semitones. For example 1.0 would be a half-step up. Default: 0
glide (optional)	The glide/portamento time in seconds. A value of 0.0 would mean no portamento. This value can also be set at the <group> and <sample> levels, although most people will want to set it globally at the <groups> level. Default: 0.0
glide (optional)	Controls the glide/portamento behavior. Possible values are: always (glide is always performed), legato (glide is performed only when transitioning from one note to another), and off . This value can also be set at the <group> and <sample> levels, although most people will want to set it globally at the <groups> level. Default: legato

1.3.1 The <group> element

Samples and oscillators live in groups. There can be many group elements under the <groups> element. It can be useful to sort your samples into groups in order to apply similar settings to them or to control them with a knob. Each group can contain <sample> elements, an <oscillator> element, or both together. The order of groups in a file matters insofar as bindings will often reference groups by using an index. The first group in a file is group 0, the second is group 1, etc.

Attribute	Description	
enabled	Whether or not this group is enabled. Possible values: true, false. Default: true	(optional)
volume	The volume of the group. Value can be in linear 0.0-1.0 or in decibels. If it's in decibels you must append dB after the value (example: "3dB"). Default: 1.0	(optional)
ampVel	The degree to which the velocity of the incoming notes affects the volume of the samples in this group. 0 = not at all. 1 = volume is completely determined by incoming velocity. When the value is 1, a velocity of 127 (max velocity) yields a gain 1.0 (full volume), a velocity of 63 (half velocity) yields a gain of 0.5 (half volume), etc.	(optional)
groupPitch	Group-level pitch adjustment for changing note pitch. In semitones. For example 1.0 would be a half-step up and -1 would a half-step down. Default: 0	(optional)
pitch	A number from 0.0 to 1.0. 0 means that the pitch will stay the same regardless of what note is played. 1 means that the pitch will increase by one semitone when the note increases by one semitone (normal key pitch tracking). Applies to all samples and oscillators in the group. Default: 1	(optional)
glide	The glide/portamento time in seconds for samples and oscillators in this group. A value of 0.0 means no portamento. Can be overridden at the `` level. Inherits from `` level if not specified. Default: 0.0	(optional)
glide	Controls the glide/portamento behavior for samples and oscillators in this group. Possible values: `always` (glide is always performed), `legato` (glide only when transitioning from one note to another), `off` (no glide). Inherits from `` level if not specified. Default: `legato`	(optional)

The <sample> element

Underneath the <group> elements are <sample> elements. Each sample corresponds to a playable “zone” of your instrument. Attributes:

Attribute	Description
path (require)	The relative path of the sample file to play for this zone. Supported audio formats are WAV (.wav), AIFF (.aif, .aiff), and FLAC (.flac).
root (require)	The MIDI note number (from 1 to 127) of the note.
loNo (optional)	The MIDI note number (from 1 to 127) of the lowest note for which the zone should be triggered. Default: 0.
hiNo (optional)	The MIDI note number (from 1 to 127) of the highest note for which the zone should be triggered. Default: 127.
loVe (optional)	The lowest velocity for which this zone should be triggered. Default: 0
hiVe (optional)	The highest velocity for which this zone should be triggered. Default: 127
loCC (optional)	Using these parameter, you can use MIDI continuous controllers to filter whether or not a note should be played. This lets you, for example, have one set of samples that get played when the piano sustain pedal is down and another set that get played when it is up. Each time a MIDI CC value comes for a specific CC#, the engine stores that value. When a “note on” signal is received, the engine makes a decision (based on the last received value and the range defined by these attributes) about whether or not this sample should be played. If you use loCCN, you must also use a corresponding hiCCN for the same MIDI CC number so that you are defining a range of values. Example: loCC64=“90” and hiCC64=“127” would mean that a “note on” message will only trigger this sample if the last received value for CC64 (Sustain Pedal) is between 90 and 127. This can also be set at the <group> level. Default:-1 (off)
hiCC (optional)	
star (optional)	The frame/sample position of the start of the sample audio. This is useful if the sample starts midway through the audio file. Default: 0
end (optional)	The frame/sample position of the end of the sample audio. The is useful is the zone ends before the end of the audio file. Default: the file’s length in samples minus 1.
tuni (optional)	A fine-tuning number (in semitones) for changing the note pitch. e.g 1.0 would be a half-step up. Default: 0
volu (optional)	The volume of the sample. Value can be in linear 0.0-1.0 or in decibels. If it’s in decibels you must append dB after the value (example: “3dB”). Default: 1.0
pan (optional)	A number of -100 to 100. -100 is panned all the way to the left, 100 is panned all the way to the right. This can also be set at the <group> or <groups> levels. Default: 0
pitc (optional)	A number from 0.0 to 1.0. 0 means that the pitch will stay the same regardless of what note is played. 1 means that the pitch will increase by one semitone when the note increases by one semitone (i.e. normal key pitch tracking). This can also be set at the <group> level. Default: 1
glid (optional)	The glide/portamento time in seconds for this sample. A value of 0.0 means no portamento. Inherits from `` or `` level if not specified. Default: 0.0
glid (optional)	Controls the glide/portamento behavior for this sample. Possible values: `always`, `legato`, or `off`. Inherits from `` or `` level if not specified. Default: `legato`
trig (optional)	Valid values: attack means a sample is played when the <i>note on</i> message is received. release means the sample is played when the <i>note off</i> message is received (aka a release trigger). first means that the sample will only be played if no other notes are playing. legato means that the sample will only be played if <i>some</i> other notes are already playing. continuous means that the sample will always play. This can also be set at the <group> level. Default: attack .
rele (optional)	Controls the volume decay rate for release-triggered samples based on how long the note was held. When a release trigger sample plays, its volume decreases based on the time elapsed since the note was initially pressed. The value can be specified in two formats: Decibel format (recommended) : Append “dB” to specify decay in decibels per second. For example, “3dB” means the volume decreases by 3dB for each second the note was held. If you specify a positive dB value, it will be automatically converted to negative (decay direction). Linear format : A decimal value between 0.0 and 1.0 specifying the linear gain decay factor per second. For example, 0.3 means the volume decreases by a factor of 0.3 each second. This is useful for piano pedal-up samples and other release triggers where shorter notes should have louder release samples. This can also be set at the <group> or <groups> levels. Default: 0.0 (no decay).
20	
tags (optional)	A command-separated list of tags. Example: tags=“rt,mic1”. These are useful when controlling volumes using tags. See Appendix D.
onLo (optional)	If you want a sample to be triggered when a MIDI CC controller message comes in, for example for

Looping

Attribute	Description
loopS (optional)	The frame/sample position of the start of the sample's loop. If this is not specified, but the sample is a wave file with embedded loop markers, those will be used instead. Default: 0
loopE (optional)	The frame/sample position of the end of the sample's loop. If this is not specified, but the sample is a wave file with embedded loop markers, those will be used instead. Default: the file's length in samples minus 1.
loopC (optional)	When loop crossfades are used, instead of simply looping at a specific end point, a portion of the audio from before the loop point is faded in just as the audio from the end of the loop is faded out. In this way, smooth audio loops can be achieved on samples that weren't specifically prepared as looping. This parameter is used for specifying the length of the crossfade region in frames/samples. This can also be set at the <group> level. Default: 0 (crossfades off).
loopC (optional)	This parameter is used to specify the curve used for crossfading when loop crossfades are turned on. This can also be set at the <group> level. Value values: <code>linear</code> , <code>equal_power</code> . Default: <code>equal_power</code> .
loopE (optional)	A boolean value indicating whether or not the loop should be used. Valid values: <code>true</code> , <code>false</code>

Amplitude Envelope

Each sample has its own ADSR amplitude envelope.

Attribute	Description
ampEnvEnab (optional)	This turns the amplitude envelope on and off. Valid values are: <code>false</code> and <code>true</code> (default).
attack (optional)	The attack time in seconds of the amplitude envelope of this zone. This can also be set at the <group> or <groups> levels.
decay (optional)	The decay time in seconds of the amplitude envelope of this zone. This can also be set at the <group> or <groups> levels.
sustain (optional)	The sustain level (0.0 - 1.0) of the amplitude envelope of this zone. This can also be set at the <group> or <groups> levels.
release (optional)	The release time in seconds of the amplitude envelope of this zone. This can also be set at the <group> or <groups> levels.

The curve shapes of the attack, decay, and release zones can be changed as well. All three of the of the following parameters use the same system: a value from -100 to 100 that determines the shape of the curve. **-100 is a logarithmic curve, 0 is a linear curve, and 100 is an exponential curve.**

Attribute	Description	Default Value
attackCur (optional)	A value from -100 to 100 that determines the shape of the attack curve. This can also be set at the <group> or <groups> levels.	-100 (logarithmic)
decayCur (optional)	A value from -100 to 100 that determines the shape of the decay curve. This can also be set at the <group> or <groups> levels.	100 (exponential)
releaseCur (optional)	A value from -100 to 100 that determines the shape of the release curves. This can also be set at the <group> or <groups> levels.	100 (exponential)

Round Robins

Round robins allow different samples to be played each time a zone is triggered. This is especially useful with sounds that have short attacks (such as drums), and is a great way to keep your sample libraries from sounding fake. In order for round robins to work, you must specify both a `seqMode` and a `seqPosition` for all samples. If you have several different sets of round robins with different lengths, you'll want to set the `seqLength` value as well. There are several round-robin modes:

- **round_robin**: This causes samples to be triggered sequentially according to their `seqPosition` values.
- **random**: This causes random samples to be chosen from within the group of samples. If there are more than two round robins, then the algorithm makes sure not to hit the same one twice in a row.
- **true_random**: This causes random samples to be chosen from within the group of samples.
- **always**: This just turns round robins off.

At-tribute	Description
seqMod (optional)	Valid values are <code>random</code> , <code>true_random</code> , <code>round_robin</code> , and <code>always</code> . A value indicating the desired round robin behavior for this sample or group of samples. This can also be set at the <code><group></code> and <code><groups></code> levels. Default: <code>always</code>
seqLen (optional)	The length of the round robin queue. This can also be set at the <code><group></code> or <code><groups></code> levels. If this is left out, then the engine will try to auto-detect the length of the round robin sequence. Default: 0
seqPos (optional)	A number indicating this zone's position in the round robin queue. This can also be set at the <code><group></code> level. Default: 1

Voice-Muting / Legato

Looping

At-tribut	Description
sile (optional)	A command-separated list of tags. Example: <code>tags="rt,mic1"</code> . If a sample containing one of these tags gets triggered, then this sample will be stopped. This is useful when setting up drums as it will allow you to mute one hi-hat when another hi-hat plays. See Appendix D.
sile (optional)	Controls how quickly voices get silenced. <code>fast</code> = immediately; <code>normal</code> = triggers the sample's release phase. This second option, when used in conjunction with the <code>release</code> attribute, allows you to specify a longer release time. Values: <code>fast</code> , <code>normal</code> . Default: <code>fast</code>
sile (optional)	Specifies a custom fade-out time in seconds when a voice is silenced by the <code>silencedByTags</code> mechanism or tag polyphony limits. When set to a value greater than 0, this overrides the <code>silencingMode</code> behavior and uses the specified decay time for the fade-out. This provides precise control over silencing transitions. For example, <code>silencingDecay="0.05"</code> creates a 50ms fade-out. Default: 0 (uses <code>silencingMode</code> instead)
prev (optional)	Only play this sample if the previously triggered note equals one of these notes. Format: a comma-separated list of MIDI note numbers (from 1 to 127) of the note.
lega (optional)	This is similar to the <code>previousNote</code> attribute. This causes the engine to only play the sample if the previously triggered note is exactly this semitone distance from the previous note. For example, if the note for which this sample is being triggered is a C3 and the <code>legatoInterval</code> is set to -2, then the sample will only play if the previous note was a D3 because D3 minus 2 semitones equals C3. Format: This can be a positive or negative whole number.

Routing Audio

My default all audio is routed to the main output. However, using the attributes below you can route audio to any of the 16 buses or directly to any one of the 16 auxiliary outputs. You can also specify the volume of the audio being sent

to each output.

Attribute	Description
output1Target (optional)	The first audio output for this sample. This is a string that specifies the audio output that the sample should be routed to. The available options are <code>MAIN_OUTPUT</code> (the main audio output), <code>NO_OUTPUT</code> (audio is not routed anywhere), <code>BUS_1</code> (the first bus defined in the <code><buses></code> element), <code>BUS_2</code> (the second bus), ..., <code>BUS_16</code> (bus 16), <code>AUX_STEREO_OUTPUT_1</code> (auxiliary output 1), <code>AUX_STEREO_OUTPUT_2</code> (auxiliary output 2), ..., <code>AUX_STEREO_OUTPUT_16</code> (auxiliary output 16). Default: <code>MAIN_OUTPUT</code>
output2Target (optional)	The second audio output for this sample. This is a string that specifies the audio output that the sample should be routed to. The available options are the same as for <code>output1Target</code> . Default: <code>NO_OUTPUT</code>
output3Target (optional)	The third audio output for this sample. This is a string that specifies the audio output that the sample should be routed to. The available options are the same as for <code>output1Target</code> . Default: <code>NO_OUTPUT</code>
output4Target (optional)	The fourth audio output for this sample. This is a string that specifies the audio output that the sample should be routed to. The available options are the same as for <code>output1Target</code> . Default: <code>NO_OUTPUT</code>
output5Target (optional)	The fifth audio output for this sample. This is a string that specifies the audio output that the sample should be routed to. The available options are the same as for <code>output1Target</code> . Default: <code>NO_OUTPUT</code>
output6Target (optional)	The sixth audio output for this sample. This is a string that specifies the audio output that the sample should be routed to. The available options are the same as for <code>output1Target</code> . Default: <code>NO_OUTPUT</code>
output7Target (optional)	The seventh audio output for this sample. This is a string that specifies the audio output that the sample should be routed to. The available options are the same as for <code>output1Target</code> . Default: <code>NO_OUTPUT</code>
output8Target (optional)	The eighth audio output for this sample. This is a string that specifies the audio output that the sample should be routed to. The available options are the same as for <code>output1Target</code> . Default: <code>NO_OUTPUT</code>
output1Volume (optional)	The volume of the audio being sent to the sample's first output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume. Default: 1.0
output2Volume (optional)	The volume of the audio being sent to the sample's second output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume. Default: 1.0
output3Volume (optional)	The volume of the audio being sent to the sample's third output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume. Default: 1.0
output4Volume (optional)	The volume of the audio being sent to the sample's fourth output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume. Default: 1.0
output5Volume (optional)	The volume of the audio being sent to the sample's fifth output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume. Default: 1.0
output6Volume (optional)	The volume of the audio being sent to the sample's sixth output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume. Default: 1.0
output7Volume (optional)	The volume of the audio being sent to the sample's seventh output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume. Default: 1.0
output8Volume (optional)	The volume of the audio being sent to the sample's eighth output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume. Default: 1.0

The `<oscillator>` element

The `<oscillator>` element allows you to add synthesized waveforms to your instrument, which can be used alongside or instead of traditional audio samples. Oscillators are useful for creating sub-bass layers, adding harmonic content, or building entirely synthesized instruments within DecentSampler.

Oscillators live inside `<group>` elements, just like `<sample>` elements. Each group can contain samples, an oscillator, or both, allowing you to layer oscillators with samples within the same group or create multi-oscillator patches by using multiple groups.

Basic Usage:

The simplest oscillator configuration requires only a waveform type:

```
<group name="Sub Bass" volume="0.6" groupTuning="-12">
  <oscillator waveform="sine" />
</group>
```

This creates a sine wave oscillator tuned one octave below the played note.

Attributes:

The `<oscillator>` element itself has only one attribute:

Attribute	Requirement	Description	Default
waveform	(optional)	The waveform shape. Valid values: <code>sine</code> , <code>saw</code> , <code>square</code> , <code>triangle</code> , <code>noise</code> (or <code>white_noise</code>), <code>pluck1</code> , <code>wavetable</code> , <code>harmonic</code> , <code>fm6op</code> .	<code>sine</code>
damp1	(optional)	Only for pluck1 waveform. Controls the decay time of the plucked string. Range: 0.0 to 1.0. Lower values (closer to 0.0) create heavily damped, shorter sounds. Higher values (closer to 1.0) create minimal damping with longer, more resonant decay. This simulates the natural damping characteristics of string materials and playing techniques.	0.5
pluck	(optional)	Only for pluck1 waveform. Blends between different excitation signals to control the timbral character. Range: 0.0 to 1.0. At 0.0, the oscillator uses a smooth triangle wave excitation producing a softer, mellower tone. At 1.0, it uses a noise burst excitation producing a brighter, more aggressive attack with richer harmonics. Intermediate values blend between the two extremes.	0.5
wavet	(optional)	Only for wavetable waveform. Path to the multi-frame wavetable .wav file, relative to the .dspreset file. The file should contain all wavetable frames concatenated in a single audio file. If the file contains a <code>clm</code> RIFF chunk (Serum-compatible format), the frame size is detected automatically.	(none)
wavet	(optional)	Only for wavetable waveform. Number of audio samples per wavetable frame. When the wavetable file contains a <code>clm</code> RIFF chunk, this value is read automatically and any explicit setting is overridden.	2048
wavet	(optional)	Only for wavetable waveform. The initial playback position within the wavetable, expressed as a normalized value from 0.0 (first frame) to 1.0 (last frame). Intermediate values produce a linear crossfade between adjacent frames. Can be animated using the <code>OSCILLATOR_WAVETABLE_POSITION</code> binding parameter.	0.0
random	(optional)	Only for wavetable waveform. When true, each note-on randomizes the oscillator's start phase instead of always resetting to zero. Strongly recommended when layering multiple wavetable groups (e.g. detuned unison voices) to prevent phase cancellation between voices. Has no effect on other waveform types.	false
wavet	(optional)	Only for wavetable waveform. Controls whether adjacent wavetable frames are linearly crossfaded as the position moves. When true (default), the position knob smoothly interpolates between adjacent frames — ideal for morphing wavetables where intermediate timbres are musically meaningful. When false, the oscillator snaps to the nearest integer frame with no crossfading — useful for wavetables that contain discrete, unrelated shapes (e.g. sine, triangle, saw, square) where blended intermediates are unwanted. Can be changed at runtime via the <code>OSCILLATOR_WAVETABLE_FRAME_INTERPOLATION</code> binding parameter.	true

When using `waveform="harmonic"`, you can define additive harmonic content using these optional attributes (typically set on the parent `<group>` element):

Attribute	Description	De- fault
numPartials	Number of active harmonic partials. Integer range: 1–64.	8
harmonicTilt	Spectral tilt control for harmonic partials. Range: -1.0 to 1.0. Positive values reduce higher partials (darker tone), negative values emphasize higher partials (brighter tone).	0.
harmonicOddEvenBalance	Crossfades between odd and even partial emphasis. Range: 0.0 to 1.0. 0.0 emphasizes odd partials, 1.0 emphasizes even partials, 0.5 is balanced.	0.
harmonicNormalization	Loudness compensation amount for summed harmonic energy. Range: 0.0 to 1.0. 0.0 disables compensation, 1.0 applies full compensation.	0.
harmonicPartial1Level	Per-partial amplitude controls for partials 1–64. Each level is a floating-point value from 0.0 to 1.0.	0.
...		0
harmonicPartial64Level		

`harmonicPartial1Level` controls the fundamental, `harmonicPartial2Level` controls the first overtone, and so on.

When using `waveform="fm6op"`, set the FM parameters on the parent `<group>` element (they are inherited by the oscillator). All FM attributes are optional:

Attribute	Description	Default
fmAlgorithm	DX7-compatible algorithm number (1–32). Selects the operator routing topology—which operators are carriers (produce audio) and which are modulators.	1
fmOp1Ratio	Frequency ratio for operator 1 (primary carrier in most algorithms). Multiplies the played note frequency. 2.0 = one octave up, 0.5 = one octave down.	1.0
fmOp1Detune	DX7-compatible pitch detune for operator 1. Range -7 to +7. Uses the classic DX7 detune algorithm: higher pitch offsets at lower notes, smaller offsets at higher notes. 0 = no detune, positive values sharpen, negative values flatten.	0
fmOp1Mode	Frequency mode for operator 1. ratio (default) = frequency is a ratio of the played note. fixed = absolute frequency in Hz specified by fmOp1FixedFreq .	ratio
fmOp1FixedFreq	Fixed frequency in Hz for operator 1 when fmOp1Mode ="fixed". Ignored when mode is ratio . Useful for metallic/bell-like timbres.	440.0
fmOp1Level	Output/modulation level for operator 1. Range 0.0–1.0.	1.0
fmOp1VelocitySensitivity	Velocity sensitivity for operator 1. Range 0–7 (DX7 convention). 0 = no velocity response (full level regardless of velocity). 7 = maximum velocity scaling (level varies from 0 at vel=1 to full at vel=127).	0
fmOp1Feedback	Self-feedback amount for operator 1. Range 0.0–1.0. Note: only the physically correct feedback operator for each algorithm produces sound (usually Op6).	0.0
fmOp1Attack	ADSR attack time (seconds) for operator 1's internal envelope. A value of -1 means the operator has no independent envelope and is gated only by the outer group ADSR.	0.0
fmOp1Decay	ADSR decay time (seconds) for operator 1.	0.0
fmOp1Sustain	ADSR sustain level (0.0–1.0) for operator 1.	1.0
fmOp1Release	ADSR release time (seconds) for operator 1. A value of -1 (sentinel) lets the outer group ADSR control the release.	-1.0
fmOp2Ratio fmOp6Ratio	... Frequency ratios for operators 2–6. Same semantics as fmOp1Ratio .	1.0
fmOp2Detune fmOp6Detune	... Pitch detune for operators 2–6. Range -7 to +7. Same semantics as fmOp1Detune .	0
fmOp2Mode fmOp6Mode	... Frequency modes for operators 2–6. ratio or fixed . Same semantics as fmOp1Mode .	ratio
fmOp2FixedFreq fmOp6FixedFreq	... Fixed frequencies in Hz for operators 2–6 when respective mode is fixed .	440.0
fmOp2Level fmOp6Level	... Output/modulation levels for operators 2–6. Range 0.0–1.0.	1.0
fmOp2VelocitySensitivity ...	... Velocity sensitivity for operators 2–6. Range 0–7. Same semantics as fmOp1VelocitySensitivity .	0
fmOp6VelocitySensitivity		
fmOp2Feedback fmOp6Feedback	... Self-feedback amounts for operators 2–6. fmOp6Feedback is the primary feedback and affects all algorithms.	0.0
fmOp2Attack fmOp6Attack	... ADSR attack times for operators 2–6.	0.0
fmOp2Decay fmOp6Decay	... ADSR decay times for operators 2–6.	0.0
fmOp2Sustain fmOp6Sustain	... ADSR sustain levels for operators 2–6. Range 0.0–1.0.	1.0
fmOp2Release fmOp6Release	... ADSR release times for operators 2–6. -1 = sentinel (outer ADSR controls).	-1.0

DX7-compatible operator envelope (optional). Each operator can alternatively use a 4-stage DX7-style envelope instead of the ADSR above. Set `fmOpNEgType="dx7"` on the group to activate it, then provide the eight rate/level parameters:

Attribute	Description	De- fault
fmOp1EgType	Envelope type for operator 1. <code>adsr</code> = standard ADSR (default). <code>dx7</code> = 4-stage DX7-compatible envelope using integer rate/level parameters below.	<code>adsr</code>
fmOp1EgRate1	Envelope rates R1–R4 for operator 1 (DX7 mode only). Range 0–99. Higher values are faster. R1 = attack rate, R2 = decay rate, R3 = sustain rate (holds at L3 while key	99,
...	is held), R4 = release rate.	99,
fmOp1EgRate4		0, 99
fmOp1EgLevel1	Envelope target levels L1–L4 for operator 1 (DX7 mode only). Range 0–99. L1 =	99,
...	peak after attack, L2 = level after first decay, L3 = sustain level (held while key is	99,
fmOp1EgLevel4	down), L4 = level after release.	99, 0
fmOp2EgType	Envelope types for operators 2–6. Same as <code>fmOp1EgType</code> .	<code>adsr</code>
...		
fmOp6EgType		
fmOp2EgRate1	Envelope rates R1–R4 for operators 2–6 (DX7 mode only). Same semantics as	99,
...	<code>fmOp1EgRate1</code> – <code>fmOp1EgRate4</code> .	99,
fmOp6EgRate4		0, 99
fmOp2EgLevel1	Envelope levels L1–L4 for operators 2–6 (DX7 mode only). Same semantics as	99,
...	<code>fmOp1EgLevel1</code> – <code>fmOp1EgLevel4</code> .	99,
fmOp6EgLevel4		99, 0

The DX7 rate/level envelope uses the same integer parameter conventions as classic 6-operator DX7 synthesizers: rates and levels are both in the range 0–99, and the values are directly compatible with patch data from vintage hardware and standard DX7 patch banks. See [How to Use FM Synthesis](#) for worked examples.

DX7-compatible operator envelope (optional). Each operator can alternatively use a 4-stage DX7-style envelope instead of the ADSR above. Set `fmOpNEgType="dx7"` on the group to activate it, then provide the eight rate/level parameters:

Attribute	Description	De- fault
fmOp1EgType	Envelope type for operator 1. <code>adsr</code> = standard ADSR (default). <code>dx7</code> = 4-stage DX7-compatible envelope using integer rate/level parameters below.	<code>adsr</code>
fmOp1EgRate1	Envelope rates R1–R4 for operator 1 (DX7 mode only). Range 0–99. Higher values are faster. R1 = attack rate, R2 = decay rate, R3 = sustain rate (holds at L3 while key	99,
...	is held), R4 = release rate.	99,
fmOp1EgRate4		0, 99
fmOp1EgLevel1	Envelope target levels L1–L4 for operator 1 (DX7 mode only). Range 0–99. L1 =	99,
...	peak after attack, L2 = level after first decay, L3 = sustain level (held while key is	99,
fmOp1EgLevel4	down), L4 = level after release.	99, 0
fmOp2EgType	Envelope types for operators 2–6. Same as <code>fmOp1EgType</code> .	<code>adsr</code>
...		
fmOp6EgType		
fmOp2EgRate1	Envelope rates R1–R4 for operators 2–6 (DX7 mode only). Same semantics as	99,
...	<code>fmOp1EgRate1</code> – <code>fmOp1EgRate4</code> .	99,
fmOp6EgRate4		0, 99
fmOp2EgLevel1	Envelope levels L1–L4 for operators 2–6 (DX7 mode only). Same semantics as	99,
...	<code>fmOp1EgLevel1</code> – <code>fmOp1EgLevel4</code> .	99,
fmOp6EgLevel4		99, 0

The DX7 rate/level envelope uses the same integer parameter conventions as classic 6-operator DX7 synthesizers: rates and levels are both in the range 0–99, and the values are directly compatible with patch data from vintage hardware and standard DX7 patch banks. See [How to Use FM Synthesis](#) for worked examples.

All FM operator parameters can be modulated in real time using bindings with the parameter names `OSCILLATOR_FM_ALGORITHM`, `OSCILLATOR_FM_OP1_LEVEL` ... `OSCILLATOR_FM_OP6_LEVEL`, `OSCILLATOR_FM_OP1_RATIO` ... `OSCILLATOR_FM_OP6_RATIO`, and `OSCILLATOR_FM_OP1_FEEDBACK` ... `OSCILLATOR_FM_OP6_FEEDBACK`. See [Appendix B](#) for independent per-operator ADSR envelope, and an optional DX7-compatible 4-stage rate/level

All other oscillator parameters are inherited from the parent `<group>` element.

Waveform Types:

- ``sine``: A pure sine wave with no harmonics. Ideal for sub-bass and clean tones.
- ``saw``: A sawtooth wave with rich harmonics. Great for bright, cutting sounds.
- ``square``: A square wave with odd harmonics. Useful for hollow, reed-like tones.
- ``triangle``: A triangle wave with fewer harmonics than saw. Produces a mellower tone.
- ``noise`` (or ``white_noise``): White noise generator. Perfect for percussion textures, hi-hats, snares, wind sounds, and adding grit to other sounds.
- **pluck1**: A physical modeling oscillator based on digital waveguide synthesis that simulates plucked string behavior. Unlike traditional waveforms that cycle continuously, `pluck1` generates a single excitation (like plucking a string) that decays naturally over time. Ideal for synthesizing plucked string instruments like guitar, bass, harp, koto, gayageum, and other stringed instruments. This oscillator includes two parameters for controlling the sound character: `damping` and `pluckType`.
- **wavetable**: A user-defined wavetable oscillator that reads frames from a multi-frame `.wav` file. The playback position (which frame is played) is controlled by the `wavetablePosition` attribute and can be scanned in real time using the `OSCILLATOR_WAVETABLE_POSITION` binding parameter—allowing LFOs, envelopes, MIDI CCs, or UI knobs to sweep through timbres. Supports Serum-compatible wavetable files with automatic frame size detection via the `c1m` RIFF chunk.
- **harmonic**: An additive oscillator that sums harmonic partials. Use `numPartials` to control how many partials are active (1–64), and `harmonicPartial1Level` ... `harmonicPartial64Level` (0.0–1.0) to set each partial's level. Use `harmonicTilt` for brightness shaping, `harmonicOddEvenBalance` to bias odd/even harmonics, and `harmonicNormalization` for loudness compensation. These parameters are bindable and modulatable via oscillator harmonic binding parameters in [Appendix B](#).
- **fm6op**: A 6-operator FM synthesizer implementing the 32 classic Yamaha DX7 algorithm topologies. Each algorithm defines which operators are carriers (they produce the final audio output) and which are modulators (they phase-modulate another operator to add harmonics). Every operator has its own frequency ratio, output level, feedback amount, an independent per-operator ADSR envelope, and an optional DX7-compatible 4-stage rate/level envelope. Set `fmAlgorithm` plus the `fmOp1`–`fmOp6` family of attributes on the parent `<group>` element. See [How to Use FM Synthesis](#) for a full guide. **Group-Level Parameters:**

Oscillators inherit and respect most of the same parameters as samples when set at the `<group>` level, including:

- ``groupTuning`` - Pitch adjustment in semitones
- ``loNote`` / ``hiNote`` - Note range triggering
- ``attack``, ``decay``, ``sustain``, ``release`` - ADSR envelope parameters
- ``attackCurve``, ``decayCurve``, ``releaseCurve`` - Envelope curve shapes
- ``volume`` - Oscillator group volume
- ``pan`` - Stereo positioning

- ``ampVelTrack`` - Velocity sensitivity
- ``pitchKeyTrack`` - Keyboard pitch tracking (0.0 to 1.0). A value of 0 means the oscillator stays at a fixed pitch regardless of which key is pressed. A value of 1 means normal keyboard tracking (default)
- ``glideTime`` - Portamento/glide time in seconds (inherited from `` level if not specified)
- ``glideMode`` - Portamento behavior: ``always``, ``legato``, or ``off`` (inherited from `` level if not specified)

Modulation Support:

Oscillators fully support modulation through bindings and modulators, just like samples:

- **GROUP_TUNING** - Modulate oscillator pitch (e.g., for vibrato LFOs)
- **GLOBAL_TUNING** - Global pitch modulation affects all oscillators
- **GROUP_VOLUME** - Modulate oscillator amplitude
- **AMP_VOLUME** - Control oscillator volume with UI controls or MIDI

Example with LFO vibrato:

```
<modulators>
  <lfo shape="sine" frequency="5.0" modAmount="0.15" scope="global">
    <binding type="amp" level="group" position="0" parameter="GROUP_TUNING"
      modBehavior="add" translation="linear"
      translationOutputMin="-0.5" translationOutputMax="0.5"/>
  </lfo>
</modulators>
```

Multi-Oscillator Example:

Here's a practical example showing how to layer multiple oscillators to create a rich synthesized sound:

```
<DecentSampler minVersion="1.15.0">
  <groups>
    <!-- Fundamental saw wave -->
    <group name="Fundamental" volume="0.5" attack="0.01" decay="0.3" sustain="0.7"
    ↪release="0.5">
      <oscillator waveform="saw" />
    </group>

    <!-- Sine wave octave up for brightness -->
    <group name="Octave Up" volume="0.3" groupTuning="12" attack="0.02" decay="0.3"
    ↪sustain="0.7" release="0.5">
      <oscillator waveform="sine" />
    </group>

    <!-- Sub bass (limited to lower notes) -->
    <group name="Sub Bass" volume="0.6" groupTuning="-12" loNote="0" hiNote="60"
      attack="0.01" decay="0.3" sustain="0.7" release="0.8">
      <oscillator waveform="sine" />
    </group>

    <!-- Detuned oscillators for stereo width -->
    <group name="Detuned Left" volume="0.25" pan="-50" groupTuning="-0.1"
      attack="0.01" decay="0.3" sustain="0.7" release="0.5">
      <oscillator waveform="saw" />
```

(continues on next page)

(continued from previous page)

```

    </group>

    <group name="Detuned Right" volume="0.25" pan="50" groupTuning="0.1"
        attack="0.01" decay="0.3" sustain="0.7" release="0.5">
        <oscillator waveform="saw" />
    </group>
</groups>

<effects>
    <effect type="lowpass" frequency="8000.0" resonance="1.0"/>
    <effect type="reverb" wetLevel="0.15" roomSize="0.5"/>
</effects>
</DecentSampler>

```

Monophonic Lead Synth with Portamento:

Here's a complete example of a Moog-style monophonic lead synthesizer with portamento:

```

<DecentSampler minVersion="1.15.0">
  <ui width="812" height="375">
    <tab name="main">
      <labeled-knob x="10" y="20" width="90" label="Attack" type="float"
        minValue="0.0" maxValue="2.0" value="0.01">
        <binding type="amp" level="instrument" position="0" parameter="ENV_ATTACK"/>
      </labeled-knob>
      <labeled-knob x="110" y="20" width="90" label="Glide Time" type="float"
        minValue="0.0" maxValue="2.0" value="0.3">
        <binding type="general" level="instrument" position="0" parameter="GLIDE_TIME"/>
      </labeled-knob>
      <labeled-knob x="210" y="20" width="90" label="Cutoff" type="float"
        minValue="0" maxValue="22000" value="20000.0">
        <binding type="effect" level="instrument" position="0" parameter="FX_FILTER_
↪FREQUENCY"/>
      </labeled-knob>
      <labeled-knob x="10" y="130" width="90" label="Detune" type="float"
        minValue="-1" maxValue="1" value="0.1">
        <binding type="amp" level="group" position="1" parameter="GROUP_TUNING"/>
      </labeled-knob>
    </tab>
  </ui>

  <!-- glideTime and glideMode set at groups level apply to all oscillators -->
  <groups glideTime="0.3" glideMode="legato" attack="0.01" decay="0.3"
    sustain="0.7" release="0.5" volume="0.8">
    <!-- Two detuned saw oscillators for thickness -->
    <group name="Osc1" tags="osc1">
      <oscillator waveform="saw" />
    </group>
    <group name="Osc2" tags="osc2">
      <oscillator waveform="saw" />
    </group>
  </groups>

```

(continues on next page)

(continued from previous page)

```

<effects>
  <effect type="lowpass" frequency="2000.0" resonance="0.3"
    envelope_amount="3000.0" envelope_attack="0.01"
    envelope_decay="0.3" envelope_sustain="0.0" envelope_release="0.5"/>
</effects>

<modulators>
  <!-- LFO vibrato -->
  <lfo shape="sine" frequency="5.0" modAmount="0.15" scope="global">
    <binding type="amp" level="group" tags="osc1,osc2" parameter="GROUP_TUNING"
      modBehavior="add" translation="linear"
      translationOutputMin="-0.5" translationOutputMax="0.5"/>
  </lfo>
</modulators>

<!-- Use tag polyphony to enforce monophonic behavior -->
<tags>
  <tag name="osc1" volume="1" pan="0" polyphony="1" />
  <tag name="osc2" volume="1" pan="0" polyphony="1" />
</tags>
</DecentSampler>

```

This preset demonstrates:

- **Portamento/Glide:** Set via `glideTime`` (0.3 seconds) and `glideMode`` (legato - only glides between connected notes)
- **Monophonic behavior:** Achieved using tag polyphony limits (`polyphony="1"`)
- **Detuned oscillators:** Two saw waves with slightly different tuning for a fatter sound
- **Modulation:** LFO vibrato applied to both oscillators via `GROUP_TUNING` binding
- **Filter envelope:** Adds movement to the timbre

Combining Oscillators with Samples:

You can combine oscillators and samples within the same group or across separate groups. Here's an example mixing them in a single group to add a sub-bass layer to piano samples:

```

<groups>
  <!-- Piano samples with built-in sine wave sub-bass -->
  <group name="Piano with Sub" volume="1.0">
    <sample path="Samples/Piano/C3.wav" rootNote="60" loNote="59" hiNote="61" />
    <sample path="Samples/Piano/D3.wav" rootNote="62" loNote="61" hiNote="63" />
    <!-- more samples... -->
    <!-- Oscillator mixed in the same group -->
    <oscillator waveform="sine" />
  </group>
</groups>

```

Or use separate groups for independent control:

```

<groups>
  <!-- Sample-based piano sound -->
  <group name="Piano Samples" volume="1.0">

```

(continues on next page)

(continued from previous page)

```

    <sample path="Samples/Piano/C3.wav" rootNote="60" loNote="59" hiNote="61" />
    <!-- more samples... -->
  </group>

  <!-- Sine wave sub-bass layer -->
  <group name="Sub Layer" volume="0.3" groupTuning="-12" hiNote="60">
    <oscillator waveform="sine" />
  </group>
</groups>

```

Physical Modeling with pluck1:

The `pluck1` waveform uses digital waveguide synthesis to simulate plucked string instruments. Unlike traditional oscillator waveforms that cycle continuously, `pluck1` generates a transient excitation that decays naturally, making it ideal for realistic plucked string sounds:

```

<DecentSampler minVersion="1.15.0">
  <ui width="812" height="375">
    <tab name="main">
      <labeled-knob x="10" y="20" width="90" label="Damping" type="float"
        minValue="0.0" maxValue="1.0" value="0.5">
        <binding type="general" level="group" position="0"
          parameter="OSCILLATOR_DAMPING"/>
      </labeled-knob>
      <labeled-knob x="110" y="20" width="90" label="Pluck Type" type="float"
        minValue="0.0" maxValue="1.0" value="0.5">
        <binding type="general" level="group" position="0"
          parameter="OSCILLATOR_PLUCK_TYPE"/>
      </labeled-knob>
    </tab>
  </ui>

  <groups attack="0.001" decay="0.1" sustain="0.8" release="0.2">
    <group name="Plucked String" volume="0.8">
      <oscillator waveform="pluck1" damping="0.5" pluckType="0.5" />
    </group>
  </groups>
</DecentSampler>

```

In this example:

- `damping="0.5"`: Moderate decay time - the string sustains for a reasonable duration
- `pluckType="0.5"`: Balanced between smooth (triangle wave) and bright (noise burst) excitation
- `OSCILLATOR_DAMPING` and `OSCILLATOR_PLUCK_TYPE` binding parameters allow real-time control from the UI

The `pluck1` oscillator is particularly effective for:

- **Bass guitars**: Use lower `damping` values (0.3-0.5) with higher `pluckType` (0.6-0.8) for percussive attack
- **Acoustic guitars**: Medium `damping` (0.5-0.7) with balanced `pluckType` (0.4-0.6)
- **Harp and lyres**: Higher `damping` (0.7-0.9) with lower `pluckType` (0.2-0.4) for softer, sustained tones

- **Ethnic stringed instruments:** Experiment with `pluckType` to match the excitation character of traditional instruments like koto, sitar, gayageum, etc. **User-Defined Wavetable Oscillator:**

The wavetable waveform lets you load any multi-frame .wav file as a wavetable. Each frame is a fixed-size window of samples that the oscillator cycles through at the correct pitch. Moving the position between frames transitions between different timbres, enabling classic wavetable scanning effects.

Wavetable File Format:

- The file must be a standard .wav containing all frames concatenated in sequence (e.g., 128 frames × 2048 samples = 262144 samples total).
- If the file contains a **clm RIFF chunk** (Serum-compatible format), the frame size is detected automatically. Files exported from Serum or other compatible tools include this chunk.
- If no clm chunk is present, the frame size defaults to 2048 samples. You can override this by setting wavetableFrameSize on the <oscillator> element.

Wavetable Scanning with a Knob:

```
<DecentSampler minVersion="1.15.0">
  <ui width="812" height="375">
    <tab name="main">
      <labeled-knob x="10" y="20" width="90" label="Position" type="float"
        minValue="0.0" maxValue="1.0" value="0.0">
        <binding type="general" level="group" position="0"
          parameter="OSCILLATOR_WAVETABLE_POSITION"
          translation="linear"
          translationOutputMin="0.0" translationOutputMax="1.0"/>
      </labeled-knob>
    </tab>
  </ui>

  <groups attack="0.01" decay="0.5" sustain="1.0" release="0.5">
    <group name="Wavetable Oscillator">
      <oscillator waveform="wavetable"
        wavetableFile="Wavetables/MyWavetable.wav"
        wavetablePosition="0.0"/>
    </group>
  </groups>
</DecentSampler>
```

Wavetable Scanning with an LFO:

```
<DecentSampler minVersion="1.15.0">
  <ui width="812" height="375">
    <tab name="main">
      <labeled-knob x="10" y="20" width="90" label="LFO Rate" type="float"
        minValue="0.1" maxValue="10.0" value="1.0">
        <binding type="modulator" level="instrument" modulatorIndex="0"
          parameter="FREQUENCY"/>
      </labeled-knob>
      <labeled-knob x="110" y="20" width="90" label="LFO Depth" type="float"
        minValue="0.0" maxValue="1.0" value="0.5">
        <binding type="modulator" level="instrument" modulatorIndex="0"
          parameter="MOD_AMOUNT"/>
      </labeled-knob>
    </tab>
  </ui>
</DecentSampler>
```

(continues on next page)

(continued from previous page)

```

    </tab>
  </ui>

  <groups attack="0.01" decay="0.5" sustain="1.0" release="0.5">
    <group name="Wavetable Oscillator" wavetablePosition="0.5">
      <oscillator waveform="wavetable"
        wavetableFile="Wavetables/MyWavetable.wav"/>
    </group>
  </groups>

  <modulators>
    <lfo shape="sine" frequency="1.0" modAmount="0.5" scope="voice">
      <binding type="general" level="group" position="0"
        parameter="OSCILLATOR_WAVETABLE_POSITION"
        modBehavior="add"
        translationOutputMin="0.0" translationOutputMax="0.5"/>
    </lfo>
  </modulators>
</DecentSampler>

```

In this example, the LFO continuously sweeps the wavetable position around a center value of 0.5, scanning through timbres at the LFO rate. The `modBehavior="add"` means the LFO value is added on top of the static `wavetablePosition` set on the group.

Selecting the active wavetable from a menu:

Because `wavetableFile` cannot be changed at runtime via a binding, the way to offer a wavetable selector is to define one `<group>` per wavetable (each with a unique tag) and use a `<menu>` with `TAG_ENABLED` bindings to switch between them:

```

<ui width="812" height="375">
  <tab name="main">
    <menu x="20" y="68" width="220" height="26" value="1">
      <option name="Growl 01">
        <binding type="amp" level="tag" identifier="growl" parameter="TAG_ENABLED"
          translation="fixed_value" translationValue="true"/>
        <binding type="amp" level="tag" identifier="fm" parameter="TAG_ENABLED"
          translation="fixed_value" translationValue="false"/>
      </option>
      <option name="FM 01">
        <binding type="amp" level="tag" identifier="growl" parameter="TAG_ENABLED"
          translation="fixed_value" translationValue="false"/>
        <binding type="amp" level="tag" identifier="fm" parameter="TAG_ENABLED"
          translation="fixed_value" translationValue="true"/>
      </option>
    </menu>
  </tab>
</ui>

```

(continues on next page)

(continued from previous page)

```

<groups>
  <group tags="growl">
    <oscillator waveform="wavetable" wavetableFile="Wavetables/Growl/Growl 01.wav"
    ↪randomPhase="true"/>
  </group>
  <group tags="fm" enabled="false">
    <oscillator waveform="wavetable" wavetableFile="Wavetables/FM/FM 01.wav"
    ↪randomPhase="true"/>
  </group>
</groups>

```

Each menu option enables exactly one tag and disables all others. The initial `enabled="false"` on non-default groups matches the menu's default selection (`value="1"` = first option).

Tips for Wavetable Sounds:

- Use **Serum-format** wavetables (exported from Serum or compatible tools) for automatic frame size detection.
- A **knob** scanning the position gives manual timbral control, while an **LFO** creates evolving, animated textures.
- Combine with envelope modulation on `OSCILLATOR_WAVETABLE_POSITION` for timbral sweeps triggered by note velocity or the amplitude envelope.
- Pair with `groupTuning` detuning and panning for unison-style / supersaw-like layering.
- Always set **`randomPhase="true"`** when layering multiple wavetable groups. Without it, all voices start at phase zero and cancel each other. **UI Control for Oscillators:**

You can create UI controls to switch waveforms dynamically:

```

<ui width="812" height="375">
  <tab name="main">
    <menu x="10" y="70" width="150" height="30" value="1">
      <option name="Sine">
        <binding type="general" level="group" position="0"
        parameter="OSCILLATOR_WAVEFORM"
        translation="fixed_value" translationValue="sine"/>
      </option>
      <option name="Saw">
        <binding type="general" level="group" position="0"
        parameter="OSCILLATOR_WAVEFORM"
        translation="fixed_value" translationValue="saw"/>
      </option>
      <option name="Square">
        <binding type="general" level="group" position="0"
        parameter="OSCILLATOR_WAVEFORM"
        translation="fixed_value" translationValue="square"/>
      </option>
    </menu>
  </tab>
</ui>

```

Tips and Best Practices:

1. **Layer multiple waveforms:** Combine different waveform types at different octaves for complexity.
2. **Use detuning:** Create stereo width by layering slightly detuned oscillators panned left and right.

3. **Limit note ranges:** Use ``loNote`` and ``hiNote`` to restrict certain oscillators to specific registers (e.g., sub-bass only on low notes).
4. **Add envelope variation:** Use different ADSR settings on each oscillator group for evolving textures.
5. **Monophonic leads:** Use tag polyphony (``polyphony="1"``) combined with ``glideMode="legato"`` for classic synthesizer lead sounds.
6. **Modulation:** Apply LFOs and envelopes to ``GROUP_TUNING`` for vibrato, tremolo, and evolving textures.

Feature Parity with Samples:

As of DecentSampler 1.15.0, oscillators support most sample features including:

- **Pitch key tracking** (`pitchKeyTrack`) - Control how oscillators respond to keyboard pitch
- **Portamento/Glide** (`glideTime`, `glideMode`) - Smooth pitch transitions between notes
- **Modulation bindings** - Full support for `GROUP_TUNING`, `GLOBAL_TUNING`, and `GROUP_VOLUME` modulation
- **Tag-based polyphony** - Use tags with polyphony limits for monophonic/polyphonic control
- **ADSR envelopes** - Complete amplitude envelope control with curve shaping
- **Effects chains** - Group-level and global effects apply to oscillators
- **Velocity tracking** - `ampVelTrack` for velocity-sensitive volume
- **Note range filtering** - `loNote`/`hiNote` for keyboard zones

Limitations:

- Oscillators do not support looping parameters (``loopStart``, ``loopEnd``, etc.) since they generate continuous waveforms.
- Round-robin sequencing (``seqMode``, ``seqPosition``) is not applicable to oscillators.
- Sample-specific features like ``start``/``end`` positions and embedded loop markers don't apply.

Version Requirements:

Oscillators were introduced in **DecentSampler version 1.15.0**. All oscillator features including portamento and modulation support require **version 1.15.0 or higher**. Set the appropriate ``minVersion`` attribute:

```
<DecentSampler minVersion="1.15.0">
```

1.4 The `<effects>` element

Adding global, instrument-wide effects is easy: just add an `<effects>` element right below your top-level `<DecentSampler>` element.

It is also possible to have effects that only get added to a specific group. To adding effects that only apply to a specific group, all you need to do is create an `<effects>` group that lives underneath the `<group>` element for the group you want to affect.

Group level effects are initialized every time a note is started and destroyed every time a note is stopped. If you play two notes simultaneously, two instances of this effect will be created and these will be independent of each other. As a result, they use more CPU than global effects.

NOTE: Only certain effects will work as group-level effects: lowpass filter, hipass filter, bandpass filter, gain, and chorus. Delay and reverb cannot work properly as they will be deleted before their tail peters out.

1.4.1 The <effect> element

Within the <effects> element, you can have any number of <effect> sub-elements. These specify parameters for each individual effect that you would like to have in your global effects chain. There are currently only a handful effects available although more could definitely be added on request:

All <effect> elements support a **tags** attribute — a comma-separated list of tag names. Tags allow bindings to target effects by name instead of by numeric index. For example:

```
<effects>
  <effect type="lowpass" frequency="22000" tags="main-filter" />
</effects>
```

A binding can then reference this effect using `tags="main-filter"` instead of `effectIndex="0"`:

```
<binding type="effect" level="instrument" tags="main-filter" parameter="FX_FILTER_
↪FREQUENCY"
  translation="linear" translationOutputMin="100" translationOutputMax="8000" />
```

Low-pass, Band pass, and Hi-pass filter

A 2-pole resonant filter that can be either a lowpass, bandpass, or highpass filter

Example:

```
<DecentSampler>
  <effects>
    <effect type="lowpass" resonance="0.7" frequency="22000" />
  </effects>
</DecentSampler>
```

There is also a single pole version of the lowpass filter that can be accessed using the `lowpass_1pl` effect type. This version does not have a resonance parameter.

```
<DecentSampler>
  <effects>
    <effect type="lowpass_1pl" frequency="22000" />
  </effects>
</DecentSampler>
```

Attributes:

At-tribute	Type	Valid Range	De-fault
type	Re-quired	The type of filter	
resonanc	Op-tional	The filter reso-nance (Q)	0.7
frequenc	Op-tional	The filter fre-quency	22000

Notch EQ Filter

A simple notch filter.

Example:

```
<DecentSampler>
  <effects>
    <effect type="notch" q="0.7" frequency="22000" />
  </effects>
</DecentSampler>
```

Attributes:

Attribute	Type		Valid Range	Default
type	Required	The type of filter	Must be notch	
frequency	Required	The filter frequency	60 - 22000.0	10000
Q	Optional	Q is the ratio of center frequency to bandwidth	0.01 - 18.0	0.7

Peak EQ Filter

A peak filter centred around a given frequency, with a variable Q and gain.

Example:

```
<DecentSampler>
  <effects>
    <effect type="peak" q="0.7" frequency="22000" gain="2.0" />
  </effects>
</DecentSampler>
```

Attributes:

At-tribute	Type		Valid Range	De-fault
type	Re-quired	The type of filter	Must be peak	
frequenc	Re-quired	The filter frequency	60 - 22000.0	10000
Q	Op-tional	Q is the ratio of center frequency to bandwidth	0.01 - 18.0	0.7
gain	Re-quired	Values greater than 1.0 will boost the high frequencies, values less than 1.0 will attenuate them.	0 - 1.0	1.0

Gain effect

Applies a volume boost or cut to the output signal.

Example:

```
<DecentSampler>
  <effects>
    <effect type="gain" level="-6" />
  </effects>
</DecentSampler>
```

Attributes:

Attribute	Type	Default	Valid Range
type	Require		gain
level	Require	0	dB: -99 – 24; linear: 0.0 – 8.0
levelUnit	Optional	decibels	decibels, linear

Reverb effect

Example:

```
<DecentSampler>
  <effects>
    <effect type="reverb" roomSize="" damping="" wetLevel="" />
  </effects>
</DecentSampler>
```

Attributes:

Attribute	Type	Valid Range	Default
type	Required	Must be reverb	reverb
roomSize	Optional	The reverb “room size”	0.7
damping	Optional	The reverb damping level	0.3
wetLevel	Optional	The volume of reverb signal	0

Delay effect

A simple delay effect that can be controlled either in seconds or using musical time increments based on the host tempo. For a complete explanation of how to use tempo-syncing, see [here](#).

Attributes:

Attribute	Type	Valid Range	Default
type	Required	Must be delay	delay
delayType	Optional	Determines whether the delay will be synced to DAW tempo or not, as well as what format you will be using for the delayTime parameter. There are two possible values: 1) seconds, which is the default, means that delayTime will be specified in seconds and will not change when the DAW tempo changes; 2) musical_time means that delay time will be specified using an integer value generated by a control which is setup to use the musical_time valueType parameter. In order for this to work, you will need to be using the plug-in within a DAW that provides a tempo to the plug-in.	seconds musical
delayTime	Optional	The delay time in seconds	0 - 20.0 0.7
feedback	Optional	The feedback level.	0 - 1.0, where 0 is no feedback, 1.0 is max feedback.
stereoOffset	Optional	The parameter allows you to introduce delay variations between the left and right channels. Half of this amount is subtracted from the left channel's delay time and half of this amount is added to the right channel's delay time. For example, if the delayTime is 0.5 seconds and the stereoOffset is 0.02 s, then the actual left channel delay time will be 0.49s and the actual right channel delay time will be 0.51s so that the two channels are offset by 0.02 seconds.	-10 - 10 0
wetLevel	Optional	The volume of the delay signal	0 - 1.0 0.5

Example of using the delay effect when specifying time in seconds:

```
<DecentSampler>
  <effects>
    <effect type="delay" delayTime="0.5" stereoOffset="0.01" feedback="0.2" wetLevel="0.5" />
  </effects>
</DecentSampler>
```

Example of using the delay effect when specifying time in musical time:

```
<DecentSampler>
  <ui>
    <tab>
      <labeled-knob x="180" y="40" label="Delay Time" valueType="musical_time" />
    </tab>
  </ui>
</DecentSampler>
```

(continues on next page)

(continued from previous page)

```

        minValue="0" maxValue="20" value="10" defaultValue="10">
    <binding type="effect" level="instrument" position="0" parameter="FX_DELAY_TIME"
↪ />
    </labeled-knob>
  </tab>
</ui>
<effects>
  <effect type="delay" delayTimeFormat="musical_time" delayTime="10" stereoOffset="0.01
↪ "
    feedback="0.3" wetLevel="0.5" />
</effects>
</DecentSampler>
```

Chorus effect

Example:

```

<DecentSampler>
  <effects>
    <effect type="chorus" mix="0.5" modDepth="0.2" modRate="0.2"/>
  </effects>
</DecentSampler>
```

Attributes:

At-tribute	Type		Valid Range	De-fault
type	Re-quired	Must be chorus	chorus	
mix	Op-tional	The wet/dry mix which controls how much of the chorus signal we hear	0 - 1.0, where 1.0 is just chorus, 0 is just dry signal.	0.5
modDep	Op-tional	The modulation depth of the effect	0 - 1.0, where 0 is no modulation, 1.0 is max modulation.	0.2
modRat	Op-tional	The modulation speed in Hz.	0 - 10.0	0.2

Phaser effect

Example:

```

<DecentSampler>
  <effects>
    <effect type="phaser" mix="0.5" modDepth="0.2" modRate="0.2" centerFrequency="400"
↪ feedback="0.7" />
  </effects>
</DecentSampler>
```

Attributes:

Attribute	Type		Valid Range	De- fault
type	Re- quired	Must be phaser	phaser	
mix	Op- tional	The wet/dry mix which controls how much of the phaser signal we hear	0 - 1.0, where 1.0 is just phaser, 0 is just dry signal.	0.5
modDepth	Op- tional	The modulation depth of the effect	0 - 1.0, where 0 is no modulation, 1.0 is max modulation.	0.2
modRate	Op- tional	The modulation speed in Hz.	0 - 10.0	0.2
centerFreq	Op- tional	The center frequency (in Hz) of the phaser all-pass filters modulation	0 - 22000	400
feedback	Op- tional	Sets the feedback volume of the phaser.	-1 - 1.0	0.7

Convolution effect

This effect allows you to use a convolution reverb or amp simulation to your sample library. Depending on the length of the impulse response, the convolution effect can use substantial CPU, so you'll definitely want to do some testing both with and without the convolution effect turned on.

Example:

```
<DecentSampler>
  <effects>
    <effect type="convolution" mix="0.5" irFile="Samples/Hall 3.wav" />
  </effects>
</DecentSampler>
```

Attributes:

At- tribute	Type		Valid Range	De- fault
type	Re- quired	Must be convolution	convolution	
mix	Op- tional	The wet/dry mix controls how much of the convolution signal we hear	0 - 1.0, where 1.0 is just convolution, 0 is just dry signal.	0.5
irFile	Re- quired	The path of the WAV or AIFF to use as an Impulse Response (IR) file	String	No de- fault

Pitch Shifter effect

As of version 1.13.3, Decent Sampler contains an old-school pitch shifter effect, which allows you to shift the pitch of the audio signal up or down by a specified number of semitones. This can be useful for creating harmonies or for adding subtle chorus effects.

Attributes:

At-tribute	Type	Valid Range	De-fault
type	Re-quired	Must be <code>pitch_shift</code>	<code>pitch_shift</code>
pitchSl	Op-tional	The number of semitones to shift the pitch of the audio signal	-24 - 24, where -24 means two octaves down, 0 means no pitch shift, and 24 means two octaves up
mix	Op-tional	The wet/dry mix controls how much of the pitch-shifted signal we hear	0 - 1.0, where 1.0 is just pitch-shifted signal, 0 is just dry signal. 0.5

Example:

```
<DecentSampler>
  <effects>
    <effect type="pitch_shift" pitchShift="2" mix="0.5" />
  </effects>
</DecentSampler>
```

Wave Folder effect

Introduced in version 1.7.2. This effect allows you to fold a waveform back on itself. This is very useful for generating additional harmonic content.

Attributes:

At-tribute	Type	Valid Range	De-fault
type	Re-quired	Must be <code>wave_folder</code>	<code>wave_folder</code>
drive	Op-tional	The volume of the input signal	1 - 100, where 100 means the signal is amplified by a factor of 100 and 1 means no amplification is applied 1
thresh	Op-tional	The amplitude above which wave folding should take place	0 - 10.0 0.25

Because wave folding tends to sound better when applied on a per-voice basis, it usually makes sense to set up the wave folder at the group level (separate group effects get created for each keypress). Example:

```
<DecentSampler pluginVersion="1">
  <ui>
    <tab>
      <labeled-knob x="180" y="40" label="Drive" type="float" minValue="1" maxValue="100
↪ " textColor="FF000000" value="1">
        <binding type="effect" level="group" groupIndex="0" effectIndex="0" parameter=
↪ "FX_DRIVE" translation="linear" />
      </labeled-knob>
      <labeled-knob x="280" y="40" label="Threshold" type="float" minValue="0" maxValue=
↪ "1" value="1" textColor="FF000000">
        <binding type="effect" level="group" groupIndex="0" effectIndex="0" parameter=
↪ "FX_THRESHOLD" translation="linear" />
      </labeled-knob>
    </tab>
  </ui>
```

(continues on next page)

(continued from previous page)

```

</ui>
<groups>
  <group>
    <!-- Samples go here. -->
    <effects>
      <effect type="wave_folder" drive="1" threshold="1" />
    </effects>
  </group>
</groups>

```

Wave Shaper effect

Introduced in version 1.7.2. This effect allows you to distort an audio signal. This is very useful for generating additional harmonic content.

Attributes:

At-tribute	Type		Valid Range	De-fault
type	Re-quire	Must be <code>wave_shaper</code>	<code>wave_shaper</code>	
drive	Op-tiona	The amount of distortion. This really just controls the volume of the input signal. The volume of the input signal	1 - 1000, where 1000 means the signal is amplified by a factor of 1000 and 1 means no amplification is applied	1
drive	Op-tiona	Introduces an extra gain boost to the drive	0 - 1.0	1
outpu	Op-tiona	The linear output level of the signal	0 - 1.0	0.1
highQ	Op-tiona	Whether or not oversampling is performed. Oversampling sounds better, but it's CPU intensive. If you want to save CPU, set this to false.	true, false	false

Because wave shaping tends to sound better when applied on a per-voice basis, it usually makes sense to set up the wave shaper at the group level (separate group effects get created for each keypress). Example:

```

<DecentSampler pluginVersion="1">
  <ui>
    <tab>
      <labeled-knob x="180" y="40" label="Drive" type="float" minValue="1" maxValue="40"
        ↪textColor="FF000000" value="0.5473124980926514">
        <binding type="effect" level="group" groupIndex="0" effectIndex="0" parameter=
        ↪"FX_DRIVE" translation="linear"/>
      </labeled-knob>
      <labeled-knob x="280" y="40" label="Drive Boost" type="float" minValue="0"
        ↪maxValue="1" value="0.328312486410141" textColor="FF000000">
        <binding type="effect" level="group" groupIndex="0" effectIndex="0" parameter=
        ↪"FX_DRIVE_BOOST" translation="linear"/>
      </labeled-knob>
      <labeled-knob x="380" y="40" label="Output Lvl" type="float" minValue="0" maxValue=
        ↪"1" value="0.328312486410141" textColor="FF000000">

```

(continues on next page)

(continued from previous page)

```

    <binding type="effect" level="group" groupIndex="0" effectIndex="0" parameter=
    ↪ "FX_OUTPUT_LEVEL" translation="linear"/>
  </labeled-knob>
</tab>
</ui>
<groups>
  <group>
    <!-- Samples go here. -->
    <effects>
      <effect type="wave_shaper" drive="0.5473124980926514" shape="0.328312486410141"
    ↪ outputLevel="0.1"/>
    </effects>
  </group>
</groups>

```

Stereo Simulator effect

Introduced in version 1.17.0. This effect converts a mono input signal into a pseudo-stereo signal using one of three classic algorithms.

```

<effect type="stereo_simulator"
  algorithm="adt"
  width="0.5"
  delayTime="0.005"
  modRate="0.5"
  modDepth="0.3" />

```

Attributes:

Attribute	Type		Valid Range	De- fault
type	Re- quired	Must be stereo_simulator	stereo_simulator	
algorithm	Op- tional	Pseudo-stereo algorithm to use	lauridsen, schroeder, adt	adt
width	Op- tional	Stereo spread and dry/wet amount; 0 = mono/dry, 1 = full stereo effect	0–1	0.5
delayTime	Op- tional	Delay time in seconds	0.001–0.030	0.005
modRate	Op- tional	LFO modulation rate in Hz (ADT algorithm only)	0.1–10.0	0.5
modDepth	Op- tional	LFO modulation depth (ADT algorithm only)	0–1	0.3

The three algorithms are:

- **lauridsen**: Complementary comb filters — simple and CPU-efficient.
- **schroeder**: Double-delay comb filters — slightly richer stereo image.
- **adt** (default): Artificial Double Tracking with LFO-modulated delay — the most convincing stereo widening effect. The `modRate` and `modDepth` parameters only affect this algorithm.

The width parameter serves as both the stereo spread amount and the dry/wet control. Setting width to 0 produces a mono dry signal; setting it to 1 applies the full stereo effect.

All parameters except `algorithm` are bindable. Example with a width knob:

```
<DecentSampler pluginVersion="1">
  <ui>
    <tab>
      <labeled-knob x="100" y="40" label="Width" type="float" minValue="0" maxValue="1"
↪value="0.5" textColor="FF000000">
        <binding type="effect" level="instrument" effectIndex="0" parameter="FX_WIDTH"
↪translation="linear" translationOutputMin="0" translationOutputMax="1"/>
      </labeled-knob>
    </tab>
  </ui>
  <groups>
    <group>
      <!-- Samples go here. -->
    </group>
  </groups>
  <effects>
    <effect type="stereo_simulator" algorithm="adt" width="0.5" delayTime="0.005"
↪modRate="0.5" modDepth="0.3" />
  </effects>
</DecentSampler>
```

Bit Crusher effect

The bit crusher reduces the bit depth and sample rate of the audio signal, producing a characteristic lo-fi, digital, crunchy sound. Bit depth reduction introduces quantisation noise, while sample rate reduction creates aliasing artifacts.

```
<effect type="bit_crusher" bitDepth="8" sampleRateReduction="4" mix="1.0"/>
```

Attributes:

Attribute	Type		Valid Range	De- fault
<code>type</code>	Re- quired	Must be <code>bit_crusher</code>	<code>bit_crusher</code>	
<code>bitDepth</code>	Op- tional	The number of bits used to quantise the audio signal. Lower values = more aggressive crushing.	1–24, where 24 is clean and 1 is maximum crushing.	24
<code>sampleRate</code>	Op- tional	Sample-rate reduction factor. A value of 4 means the effective sample rate is reduced to one quarter of the original.	1–32, where 1 is no reduction and 32 is maximum downsampling.	1
<code>mix</code>	Op- tional	The wet/dry mix which controls how much of the bit-crushed signal we hear.	0–1.0, where 1.0 is fully crushed and 0.0 is the original dry signal.	1.0

Both `bitDepth` and `sampleRateReduction` are bindable, so you can attach knobs to control them in real time.

Binding parameters for the bit crusher effect:

Binding parameter value	Description
FX_BIT_DEPTH	Controls the bit depth (1–24)
FX_SAMPLE_RATE_REDUCTION	Controls the sample-rate reduction factor (1–32)
FX_MIX	Controls the wet/dry mix (0–1)

Example with knobs for all three parameters:

```
<DecentSampler pluginVersion="1">
  <ui>
    <tab>
      <labeled-knob x="80" y="40" label="Bit Depth" type="float"
        minValue="1" maxValue="24" value="24" textColor="FF000000">
        <binding type="effect" level="instrument" effectIndex="0"
          parameter="FX_BIT_DEPTH" translation="linear"
          translationOutputMin="1" translationOutputMax="24"/>
      </labeled-knob>
      <labeled-knob x="185" y="40" label="Rate Reduction" type="float"
        minValue="1" maxValue="32" value="1" textColor="FF000000">
        <binding type="effect" level="instrument" effectIndex="0"
          parameter="FX_SAMPLE_RATE_REDUCTION" translation="linear"
          translationOutputMin="1" translationOutputMax="32"/>
      </labeled-knob>
      <labeled-knob x="290" y="40" label="Mix" type="float"
        minValue="0" maxValue="1" value="1" textColor="FF000000">
        <binding type="effect" level="instrument" effectIndex="0"
          parameter="FX_MIX" translation="linear"
          translationOutputMin="0" translationOutputMax="1"/>
      </labeled-knob>
    </tab>
  </ui>
  <groups>
    <group>
      <!-- Samples go here. -->
    </group>
  </groups>
  <effects>
    <effect type="bit_crusher" bitDepth="24" sampleRateReduction="1" mix="1.0"/>
  </effects>
</DecentSampler>
```

1.5 The <midi> element

MIDI mappings can be added to your instrument by adding a <midi> element right below your top-level <DecentSampler> element.

1.5.1 The <cc> element

Within the <midi> element, you can have any number of <cc> elements. These allow you to map changes in incoming continuous controller messages to specific parameters of your instrument. To use this functionality, you'll want to add a separate <cc> element for each CC number you would like to respond to. The <cc> element has a single required attribute `number=""` which specifies the number (from 0 to 127) of the continuous controller you would like to listen on. Beneath the <cc> element, you can have any number of bindings.

```
<midi>
  <cc number="11">
    <binding level="ui" type="control" position="0" parameter="VALUE" translation="linear
    ↪ "
        translationOutputMin="0" translationOutputMax="1"/>
  </cc>
  <cc number="1">
    <binding level="ui" type="control" position="1" parameter="VALUE" translation="linear
    ↪ "
        translationOutputMin="0" translationOutputMax="1"/>
  </cc>
</midi>
```

1.5.2 The <note> element

Within the <midi> element, you can have any number of <note> elements. These allow you to map specific notes to specific parameters of your instrument. To use this functionality, you'll want to add a separate <note> element for each MIDI note or range of notes you would like to respond to.

Here are the attributes of the <note /> element:

- **note** (required): This attribute specifies the MIDI note number (from 0 to 127) you would like to listen on. You can also specify ranges of notes by using a dash. For example `note="24-35"` would be used to specify bindings for the range of notes 24 through 35.
- **eventType** (optional): This attribute specifies the type of event to listen for. The default is `note_on`, but you can also specify `note_off` or `any`. The default is `any` if this attribute is not specified.
- **enabled** (optional): A true/false value that specifies whether this note listener is turned on.
- **swallowNotes** (optional): The bindings that live below this note listener are called before any notes are played. By default, `swallowNotes` is false, which means that the keypress will then be received by the sampler. If `swallowNotes` is true, the sampler will not receive the note. This is useful if you wish to prevent certain keys from triggering notes.

It is possible to enable and disable a note listener by targeting the `enabled` attribute.

Beneath the <note> element, you can have any number of bindings. Here is an example of how keyswitches might be set up:

```

<midi>
  <note note="11" enabled="true" eventType="note_on">
    <binding enabled="true" type="general" level="group" groupIndex="0" parameter=
    ↪ "ENABLED" translation="fixed_value" translationValue="true" />
    <binding enabled="true" type="general" level="group" groupIndex="1" parameter=
    ↪ "ENABLED" translation="fixed_value" translationValue="false" />
  </note>
  <note note="12" enabled="true" eventType="note_on">
    <binding enabled="true" type="general" level="group" groupIndex="0" parameter=
    ↪ "ENABLED" translation="fixed_value" translationValue="false" />
    <binding enabled="true" type="general" level="group" groupIndex="1" parameter=
    ↪ "ENABLED" translation="fixed_value" translationValue="true" />
  </note>
</midi>

```

In the above keyswitch example, MIDI note 11 turns on group 0 and turns off group 1, whereas MIDI note 12 does the opposite. Note the use of the `fixed_value` translation type.

1.5.3 The <velocity> element

Within the `<midi>` element, you can also have a `<velocity>` element. This element allows you to control an instrument in response to MIDI velocity messages. This is useful for creating dynamic responses based on how hard a note is played.

Example usage:

```

<midi>
  <velocity>
    <binding modAmount="0.3" level="group" parameter="FX_FILTER_FREQUENCY"
    ↪ groupIndex="0" effectIndex="0" type="effect"/>
  </velocity>
</midi>

```

In this example, the `<velocity>` element contains a single `<binding>` that modifies the `FX_FILTER_FREQUENCY` parameter of the first effect (effectIndex: 0) in the first group (groupIndex: 0) based on the velocity of incoming MIDI notes. The `modAmount` attribute specifies how much the velocity will affect the parameter.

Bindings within the <midi> section

The bindings that the `<cc>`, `<note>`, and `<velocity>` element listens on are the same as those used by the UI controls. See [Appendix B](#) for a complete description of these.

If you have a UI control mapped to the same internal parameter as a MIDI mapping, you'll want to have your MIDI mapping control the UI control instead of the parameter directly. The benefit of doing this is that, as the MIDI CC input is received, the UI control will be updated as well as the desired internal parameter.

The way to accomplish this is to make use of the `labeled_knob` or `control` binding types (`control` was introduced in version 1.1.7) as follows:

```

<binding level="ui" type="control" position="0" parameter="VALUE" translation="linear"
↪ translationOutputMin="0" translationOutputMax="1"/>

```

You'll notice that the `control` type has a `level` value of `ui` and a `parameter` value of `VALUE`. Another thing to notice is the `position=""` parameter. This contains the 0-based index of the control to be modified. **NOTE: The indexes of**

the parameter list includes all UI controls, including `<label>` and menu controls, so you'll want to account for that when calculating your positions.

An example of changing a menu option based on a MIDI note (keyswitch) would look like this:

```
<midi>
  <note note="11" eventType="note_on">
    <binding type="control" level="ui" position="1" parameter="VALUE" translation="fixed_
↪value" translationValue="1" />
  </note>
  <note note="12" eventType="note_on">
    <binding type="control" level="ui" position="1" parameter="VALUE" translation="fixed_
↪value" translationValue="2" />
  </note>
</midi>
```

1.6 The `<noteSequences>` element

As of 1.11.1, DecentSampler has a built-in note sequencer. It can be mapped to keys so that clusters of notes are played every time certain keys are pressed. It can also be mapped to UI controls such as buttons.

The `<noteSequences>` element is how you specify note sequences that can be used by this playback engine. There should be exactly one `<noteSequences>` element in each `<DecentSampler>` file.

The `<noteSequences>` element can contain one or more `<sequence>` elements:

1.6.1 The `<sequence>` element

The `<sequence>` element has the following attributes:

- **name** (optional): An optional descriptive name for the sequence. This is only used in the sample editor UI to help you identify the sequence.
- **length** (required): The length of the sequence in beats. This is a floating point number.
- **rate** (optional): The rate at which the sequence is played. This is a floating point number. The default is 1.0.

The `<sequence>` element can contain one or more `<note>` elements:

The `<note>` element

The `<note>` element has the following attributes:

- **position** (required): The position of the note in the sequence, in beats. This is a whole number.
- **velocity** (required): The velocity of the note. This is a floating point number between 0 and 1.
- **note** (required): The MIDI note number of the note.
- **length** (required): The length of the note in beats. This is a whole number.

This is an example showing the strum from an Omnichord:

```

<noteSequences>
  <sequence name="Maj1Slow" length="768.0" rate="96">
    <note position="0" velocity="1" note="48" length="768"/>
    <note position="11" velocity="1" note="52" length="757"/>
    <note position="29" velocity="1" note="55" length="739"/>
    <note position="46" velocity="1" note="60" length="722"/>
    <note position="63" velocity="1" note="64" length="705"/>
    <note position="81" velocity="1" note="67" length="687"/>
    <note position="99" velocity="1" note="72" length="669"/>
    <note position="117" velocity="1" note="76" length="651"/>
    <note position="134" velocity="1" note="79" length="634"/>
    <note position="153" velocity="1" note="84" length="615"/>
    <note position="171" velocity="1" note="88" length="597"/>
    <note position="192" velocity="1" note="91" length="576"/>
  </sequence>
</noteSequences>

```

For a full discussion of how to use note sequences, see the tutorial on *how to use note sequences*.

1.7 The <modulators> element

Version 1.6.0 of Decent Sampler officially introduces the new <modulators> section into the .dspreset format. This section lives below the top-level <DecentSampler> element and it is where all modulators for the entire sample library live.

```

<DecentSampler>
  <modulators>
    <!-- Your modulators go here. -->
  </modulators>
</DecentSampler>

```

1.7.1 The <lfo> element

Underneath the <modulators> section, you can have any number of different LFOs, which are defined using an <lfo> element, for example:

```

<modulators>
  <lfo shape="sine" frequency="2" modAmount="1.0"></lfo>
</modulators>

```

For an LFO with a 500ms delay before starting:

```

<modulators>
  <lfo shape="sine" frequency="2" modAmount="1.0" delayTime="0.500"></lfo>
</modulators>

```

For a tempo-synced LFO using musical subdivisions:

```

<modulators>
  <lfo shape="sine" frequencyFormat="musical_time" frequency="10" modAmount="1.0"></lfo>
</modulators>

```

In this mode, frequency is not Hz. It is an integer index into the same musical subdivision list used by `valueType="musical_time"` controls (for example, 10 = 1/8, 13 = 1/4, 19 = 1, 24 = 5/1).

This element has the following attributes:

- **shape**: controls the oscillator shape. Possible values are `sine`, `square`, `saw`.
- **frequency**: The speed of the LFO. If `frequencyFormat` is `hz`, this is in cycles per second. If `frequencyFormat` is `musical_time`, this is a musical subdivision index.
- **frequencyFormat**: How frequency is interpreted. Valid values are `hz` (default) and `musical_time`.
- **modAmount**: This value between 0 and 1 controls how much the modulation affects the things it is targeting. In conventional terms, this is like the modulation depth. Default value: 1.0.
- **delayTime**: The time in seconds to wait before the LFO starts outputting signal. During this delay period, the LFO outputs zero. Default value: 0.0 (no delay).
- **scope**: Whether or not this LFO exists for all notes or whether each keypress gets its own LFO. Possible values are `global` (default for LFOs) and `voice`. If `voice` is chosen, a new LFO is started each time a new note is pressed.
- **modBehavior**: This attribute controls how the LFO affects the parameter it is targeting. Possible values are `add`, `modulate`, `multiply`, and `set`. If `add` is chosen, the LFO will add its translated value directly to the target parameter (legacy behavior). If `modulate` is chosen, the LFO adds a zero-centered modulation delta around the target parameter's current/base value, so a neutral LFO value contributes no offset. If `multiply` is chosen, the LFO will multiply its value by the parameter it is targeting. If `set` is chosen, the LFO will set the parameter it is targeting to its value. Default value: `set`.

1.7.2 The <envelope> element

In addition to LFOs, you can also have additional ADSR envelopes. These can be useful for controlling group-level effects, such as low-pass filters. If this is what you wish to achieve, make sure you check out the section on group-level effects below.

To create an envelope, use an <envelope> element:

This element has the following attributes:

- **attack**: The length in seconds of the attack portion of the ADSR envelope
- **decay**: The length in seconds of the decay portion of the ADSR envelope
- **sustain**: The height of the sustain portion of the ADSR envelope. This is expressed as a value between 0 and 1.
- **release**: The length in seconds of the release portion of the ADSR envelope
- **modAmount**: This value between 0 and 1 controls how much the modulation affects the things it is targeting. In conventional terms, this is like the modulation depth. Default value: 1.0.
- **scope**: Whether or not this LFO exists for all notes or whether each keypress gets its own LFO. Possible values are `global` and `voice` (default for envelopes). If `voice` is chosen, a new LFO is started each time a new note is pressed.
- **modBehavior**: This attribute controls how the envelope affects the parameter it is targeting. Possible values are `add`, `modulate`, `multiply`, and `set`. If `add` is chosen, the envelope will add its translated value directly to the target parameter (legacy behavior). If `modulate` is chosen, the envelope adds a zero-centered modulation delta around the target parameter's current/base value, so a neutral envelope value contributes no offset. If `multiply` is chosen, the envelope will multiply its value by the parameter it is targeting. If `set` is chosen, the envelope will set the parameter it is targeting to its value. Default value: `set`.

- **attackCurve**: A numeric value from -100 to 100 that determines the shape of the attack portion of the ADSR envelope. Common values are -100 (logarithmic), 0 (linear), and 100 (exponential). Default value: -100 (logarithmic).
- **decayCurve**: A numeric value from -100 to 100 that determines the shape of the decay portion of the ADSR envelope. Common values are -100 (logarithmic), 0 (linear), and 100 (exponential). Default value: 100 (exponential).
- **releaseCurve**: A numeric value from -100 to 100 that determines the shape of the release portion of the ADSR envelope. Common values are -100 (logarithmic), 0 (linear), and 100 (exponential). Default value: 100 (exponential).

1.7.3 The <midiCC> element

The <midiCC> element allows you to use MIDI Continuous Controller (CC) messages as modulation sources. This makes it possible to control various parameters of your instrument in response to incoming MIDI CC data, such as the mod wheel, expression pedal, or any other MIDI controller.

```
<modulators>
  <midiCC number="1" modAmount="1.0" channel="voice">
    <!-- Bindings go here -->
  </midiCC>
</modulators>
```

This element has the following attributes:

- **number**: The MIDI CC number to respond to (0-127). Common CC numbers include 1 (mod wheel), 7 (volume), 10 (pan), 11 (expression), and 74 (filter cutoff). This attribute is required.
- **modAmount**: This value between 0 and 1 controls how much the modulation affects the things it is targeting. In conventional terms, this is like the modulation depth. Default value: 1.0.
- **channel**: Determines which MIDI channel's CC values to read. Options are:
 - **"voice"** (default): Uses the MIDI channel of the note that triggered the voice. This is essential for MPE and multi-channel setups where each voice may be on a different channel.
 - **1-16**: A specific channel number. The modulator will always read CC values from this channel, regardless of which channel triggered the voice. Useful for global control in non-MPE setups.

For voice-scope modulators, the default is "voice". For global-scope modulators, the default is "1".

Here's a practical example that uses the mod wheel (CC#1) to control filter cutoff with per-voice channel tracking:

```
<modulators>
  <midiCC number="1" modAmount="1.0" channel="voice" scope="voice">
    <binding type="effect" level="group" groupIndex="0" effectIndex="0" parameter=
    ↪ "FX_FILTER_FREQUENCY"
      modBehavior="add"
      translation="table"
      translationTable="0,33;0.3,150;0.4,450;0.5,1100;0.7,4100;0.9,11000;1.00001,
    ↪ 22000" />
  </midiCC>
</modulators>
```

Example of a global CC modulator that always reads from channel 1:

```

<modulators>
  <midiCC number="11" modAmount="1.0" channel="1" scope="global">
    <binding type="amp" level="instrument" parameter="AMP_VOLUME"
      modBehavior="set"
      translation="linear"
      translationOutputMin="0"
      translationOutputMax="1" />
  </midiCC>
</modulators>

```

Important distinction: The `<midiCC>` modulator creates *temporary* modulation that only lasts for the life of each note. This means the modulation values are computed per-note and do not permanently change the underlying parameter values. In contrast, the `<midi><cc>` element (documented in [the MIDI section](#)) allows MIDI continuous controllers to make *permanent* value changes to parameters, similar to how UI knobs work. Use `<midiCC>` modulators when you want per-note modulation behavior (like using the mod wheel to add vibrato to individual notes), and use `<midi><cc>` bindings when you want global parameter changes (like using a CC to control the overall cutoff frequency of a filter).

MPE and Multi-Channel Considerations: In MPE mode or multi-channel setups, CC messages are sent per-channel. The `channel` attribute determines how the modulator handles this:

- With `channel="voice"`, each note reads CC values from its own MIDI channel, allowing independent control per note (essential for MPE)
- With `channel="1"` (or any specific channel), all voices read from that channel's CC values, providing global control

1.7.4 The `<midiVelocity>` element

The `<midiVelocity>` element allows you to use note-on velocity as a modulation source. This makes it possible to control various parameters based on how hard a key is pressed, providing dynamic expression in your instrument.

```

<modulators>
  <midiVelocity modAmount="1.0" scope="voice">
    <!-- Bindings go here -->
  </midiVelocity>
</modulators>

```

This element has the following attributes:

- **modAmount:** This value between 0 and 1 controls how much the modulation affects the things it is targeting. In conventional terms, this is like the modulation depth. Default value: 1.0.
- **scope:** Whether or not this modulator exists for all notes or whether each keypress gets its own modulator. Possible values are `global` and `voice` (default for `midiVelocity`). If `voice` is chosen, each note retains its own velocity value for modulation.

Here's a practical example that uses velocity to control the brightness of a sound:

```

<modulators>
  <midiVelocity scope="voice" modAmount="1.0">
    <binding type="effect" level="group" groupIndex="0" effectIndex="0" parameter=
    ↪ "FX_FILTER_FREQUENCY"
      modBehavior="add"
      translation="linear"
      translationOutputMin="0"

```

(continues on next page)

(continued from previous page)

```

        translationOutputMax="5000" />
    </midiVelocity>
</modulators>

```

This example uses velocity to modulate the cutoff frequency of a low-pass filter, making softer notes sound darker and harder notes sound brighter. This is a common technique for adding expressiveness to sampled instruments.

Another example showing velocity controlling reverb amount:

```

<modulators>
  <midiVelocity scope="voice" modAmount="1.0">
    <binding type="effect" level="group" groupIndex="0" effectIndex="1" parameter=
    ↪ "FX_REVERB_WET_LEVEL"
        modBehavior="set"
        translation="linear"
        translationOutputMin="0.0"
        translationOutputMax="0.5" />
  </midiVelocity>
</modulators>

```

1.7.5 The <mpeTimbre> element

The <mpeTimbre> element allows users to control the timbre of an instrument in response to MPE messages. NOTE: In order for this to work, the plugin must be in MPE mode. This can be turned on by going into the **File > MIDI Input Settings..** dialog box. The <mpeTimbre> element has the following attributes:

- **scope:** Whether or not this MPE timbre exists for all notes or whether each keypress gets its own MPE timbre. Possible values are **global** and **voice** (default for MPE timbre). If **voice** is chosen, a new MPE timbre is started each time a new note is pressed.
- **risingSmoothingTime:** The time in milliseconds it takes for the MPE timbre to rise to its target value. Default value: 0 milliseconds.
- **fallingSmoothingTime:** The time in milliseconds it takes for the MPE timbre to fall to its target value. Default value: 0 milliseconds.

Here's a practical example of how to use the <mpeTimbre> element:

```

<modulators>
  <mpeTimbre scope="voice" fallingSmoothingTime="20" risingSmoothingTime="20">
    <!-- This binding modifies the frequency of a low-pass filter -->
    <binding type="effect" level="group" groupIndex="0" effectIndex="0" parameter=
    ↪ "FX_FILTER_FREQUENCY"
        modBehavior="add"
        translation="table"
        translationTable="0,33;0.3,150;0.4,450;0.5,1100;0.7,4100;0.9,11000;1.0001,
    ↪ 22000" />
  </mpeTimbre>
</modulators>

```

This example shows an MPE timbre modulator that modifies the frequency of a low-pass filter based on the MPE timbre value. This example uses a translation table to create a non-linear response curve, making the filter frequency changes more musical and intuitive.

1.7.6 The <mpePressure> element

The <mpePressure> element allows users to control an instrument in response to MPE Pressure messages. NOTE: In order for this to work, the plugin must be in MPE mode. This can be turned on by going into the **File > MIDI Input Settings..** dialog box. The <mpePressure> element has the following attributes:

- **scope**: Whether or not this MPE pressure exists for all notes or whether each keypress gets its own MPE pressure. Possible values are **global** and **voice** (default for MPE pressure). If **voice** is chosen, a new MPE pressure is started each time a new note is pressed.
- **risingSmoothingTime**: The time in milliseconds it takes for the MPE pressure to rise to its target value. Default value: 0 milliseconds.
- **fallingSmoothingTime**: The time in milliseconds it takes for the MPE pressure to fall to its target value. Default value: 0 milliseconds.

Here's a practical example of how to use the <mpePressure> element:

```
<modulators>
  <mpePressure scope="voice" fallingSmoothingTime="20" risingSmoothingTime="20">
    <binding type="amp" level="group" groupIndex="0" effectIndex="0" parameter="AMP_
    ↪ VOLUME"
      modBehavior="set"
      translation="linear"
      translationOutputMin="0.5"
      translationOutputMax="1" />
  </mpePressure>
</modulators>
```

This example shows an MPE pressure modulator that modifies the volume of a group based on the MPE pressure value. The translation is linear, meaning that the pressure value directly scales the volume between 0.5 and 1.

1.7.7 The <random> element

The <random> element allows you to generate random values for modulation. This can be useful for adding unpredictability and variation to your sounds. The <random> element always produces a value between -1 and 1. It has the following attributes:

- **mode**: The event that triggers the random value generation. This can be either **note_on** or **periodic**.
- **frequency**: How often the random value is generated. This is only relevant if the **mode** is set to **periodic**.
- **trigger**: Two possible values: **attack** means that the random value generator is reset on note-on events, **none** means that it is not. This is really only help for periodic random value generators.
- **seed**: An integer seed value for the random number generator. This can be useful for creating reproducible results.
- **scope**: Whether or not this modulator exists for all notes or whether each keypress gets its own modulator. Possible values are **global** and **voice**. If **voice** is chosen, a new random modulator is started each time a new note is pressed.

Here's an example of how to use the <random> element:

```
<modulators>
  <random mode="note_on" seed="12345" scope="voice">
    <binding type="amp" level="group" groupIndex="0" effectIndex="0" parameter="AMP_
    ↪ VOLUME"
```

(continues on next page)

(continued from previous page)

```

        modBehavior="set"
        translation="linear"
        translationOutputMin="0.5"
        translationOutputMax="1" />
    </random>
</modulators>

```

1.7.8 How to use <binding>s in conjunction with modulators

In order to actually have your LFOs and envelopes do anything, you need to have bindings under them. If you are not familiar with the concept of bindings, you may want to read [this section](#) then return here. Bindings tell the engine which parameters the LFO should be affecting and how. Here is an example:

```

<modulators>
  <lfo shape="sine" frequency="2" modAmount="1.0">
    <!-- This binding modifies the frequency of a low-pass filter -->
    <binding type="effect" level="instrument" effectIndex="0" parameter="FX_FILTER_
    →FREQUENCY" modBehavior="add" translation="linear" translationOutputMin="0"
    →translationOutputMax="2000.0" />
  </lfo>
</modulators>

```

Controlling Modulator Parameters with Bindings

You can bind to modulator parameters themselves to control them in real-time from UI controls, MIDI CC, or other sources. The binding uses `type="modulator"` and `level="instrument"`, with a `modulatorIndex` attribute pointing to the modulator (0-based) and a `parameter` attribute specifying which property to change.

Bindable parameters by modulator type

<lfo>

parameter value	Description
MOD_AMOUNT	Modulation depth (0.0–1.0)
FREQUENCY	LFO rate (Hz when <code>frequencyFormat="hz"</code> , subdivision index when <code>frequencyFormat="musical_time"</code>)
SHAPE	Waveform shape ("sine", "saw", "square", "triangle")
MOD_DELAY_TIM	Delay before the LFO begins (seconds)
TRIGGER	Reset behavior ("attack" or "none")

<envelope>

parameter value	Description
MOD_AMOUNT	Modulation depth (0.0–1.0)
ENV_ATTACK	Attack time (seconds)
ENV_ATTACK_CURVE	Attack curve shape
ENV_DECAY	Decay time (seconds)
ENV_DECAY_CURVE	Decay curve shape
ENV_SUSTAIN	Sustain level (0.0–1.0)
ENV_RELEASE	Release time (seconds)
ENV_RELEASE_CURVE	Release curve shape
MOD_DELAY_TIME	Delay before the envelope begins (seconds)

<random>

parameter value	Description
MOD_AMOUNT	Modulation depth (0.0–1.0)
FREQUENCY	Generation rate in Hz (only relevant when mode="periodic")
TRIGGER	Reset behavior ("attack" or "none")

<midiCC>

parameter value	Description
MOD_AMOUNT	Modulation depth (0.0–1.0)

<midiVelocity>

parameter value	Description
MOD_AMOUNT	Modulation depth (0.0–1.0)

<mpeTimbre>

parameter value	Description
MOD_AMOUNT	Modulation depth (0.0–1.0)

<mpePressure>

parameter value	Description
MOD_AMOUNT	Modulation depth (0.0–1.0)

Example: controlling LFO parameters from UI knobs

```

<DecentSampler>
  <ui>
    <tab>
      <!-- Controls LFO depth -->
      <labeled_knob x="10" y="30" width="90" textSize="16" textColor="AA000000"
        parameterName="Depth" minValue="0" maxValue="1" value="1">
        <binding type="modulator" level="instrument" parameter="MOD_AMOUNT"
          modulatorIndex="0" translation="linear" />
      </labeled_knob>
      <!-- Controls LFO rate -->
      <labeled_knob x="110" y="30" width="90" textSize="16" textColor="AA000000"
        parameterName="Rate" minValue="0.1" maxValue="20" value="2">
        <binding type="modulator" level="instrument" parameter="FREQUENCY"
          modulatorIndex="0" translation="linear" />
      </labeled_knob>
      <!-- Controls LFO delay time -->
      <labeled_knob x="210" y="30" width="90" textSize="16" textColor="AA000000"
        parameterName="Delay" minValue="0" maxValue="2" value="0.5">
        <binding type="modulator" level="instrument" parameter="MOD_DELAY_TIME"
          modulatorIndex="0" translation="linear" />
      </labeled_knob>
    </tab>
  </ui>
  <modulators>
    <lfo shape="sine" frequency="2" modAmount="1.0" delayTime="0.500">
      <binding type="effect" level="instrument" effectIndex="0" parameter="FX_FILTER_
→ FREQUENCY"
        modBehavior="add" translation="linear" translationOutputMin="0"
→ translationOutputMax="2000.0" />
    </lfo>
  </modulators>
</DecentSampler>

```

You can also control modulator parameters via MIDI CC:

```

<DecentSampler>
  <midi>
    <cc number="74">
      <binding type="modulator" level="instrument" parameter="MOD_AMOUNT"
        modulatorIndex="0" translation="linear" translationOutputMin="0"
→ translationOutputMax="1" />
    </cc>
  </midi>
  <modulators>
    <lfo shape="sine" frequency="2" modAmount="1.0" delayTime="0.500">
      <binding type="effect" level="instrument" effectIndex="0" parameter="FX_FILTER_
→ FREQUENCY"
        modBehavior="add" translation="linear" translationOutputMin="0"
→ translationOutputMax="2000.0" />
    </lfo>
  </modulators>

```

(continues on next page)

(continued from previous page)

</DecentSampler>

There are a few differences between bindings as they are used by knobs and the ones used by modulators. Specifically, when you move a UI control that has a binding attached, the engine actually goes out and changes the value of the parameter that is targeted by that binding. For example, if you have a knob that controls a lowpass filter's cutoff frequency, moving that knob will cause that actual frequency of that filter to change. In other words, the changes that the knob is making on the underlying sample library are *permanent*. The same is also true for bindings associated with MIDI continuous controllers.

Modulators, on the other hand, are temporary. If a modulator (such as an LFO) changes its value, the engine looks at the bindings associated with that LFO and then makes a list of *temporary* changes to the underlying data. When it comes time to render out the effect, it consults both the *permanent* value and the *temporary* modulation values. As a result of this difference in the way bindings are handled, only some parameters are “modulatable.” At time of press, the following parameters are modulatable:

- Almost all effect parameters
- Group Volume
- Global Volume
- Group Pan
- Global Pan
- Group Tuning
- Global Tuning

1.7.9 Modulator scope: global or voice-level

By default, all modulators will be created at the global level. This means that there will be exactly one modulator that is shared by all voices. In many situations, such as an LFO modulating a single low-pass filter which is shared by all of voices, this is often what we want.

But there are other situations where we don't want our modulator to be global. For example, what if we want to have a unique envelope for each key-press? Well, for that we use the `scope` attribute of the `<lfo>` or `<envelope>` element. This attribute can be set to `voice`, which means that each time a new note is pressed, a new modulator will be created for that note. This is particularly useful for envelopes, which are often used to control parameters that are unique to each note, such as the volume or filter cutoff of a note:

```
<envelope attack="2" decay="0" sustain="1" release="0.5" modAmount="1.0" scope="voice">
```

For certain parameters, such as **Group Tuning** or **Group Pan**, it may always make sense to have `scope="voice"` set. For others, such as **Global Volume** or **Global Pan**, it may make more sense to have `scope="global"` set.

For a full discussion of how to use modulators, check out [this article on the Decent Samples blog](#).

1.8 The <tags> element

The <tags> element lives right below your top-level <DecentSampler> element. It allows you to specify details about the tags you use throughout your instrument. It is however not actually necessary to include a <tags> element for every tag you use. You only need to create this if you want to specify additional details about your tags.

1.8.1 The <tag> element

Underneath the <tags> element, you can have any number of <tag> elements. These specify details for each individual tag that you use throughout your sample mapping. Attributes:

At-tribute	Description
enabled (optional)	Whether or not this tag is enabled. Possible values: true, false. Default: true
volume (optional)	A number for 0.0 to 1.0 that specifies the initial volume for a tag. Default: 1.0
polyphor (optional)	A whole number that specifies the number of voices allowed for this tag. Default: -1 (unlimited). Set to 1 for monophonic behavior.

1.8.2 Example

This example shows a tag configuration with polyphony control:

```
<tags>
  <tag name="voice1" volume="1" pan="0" polyphony="12"/>
</tags>
```

The polyphony attribute can be controlled dynamically using bindings with the TAG_POLYPHONY parameter. This is useful for creating mono/poly switches:

```
<button x="740" y="130" width="12" height="12" style="image" value="0">
  <state name="Poly" mainImage="Images/Checkbox On.png">
    <binding type="general" level="tag" identifier="voice1"
      parameter="TAG_POLYPHONY" translation="fixed_value" translationValue="12"/>
  </state>
  <state name="Mono" mainImage="Images/Checkbox Off.png">
    <binding type="general" level="tag" identifier="voice1"
      parameter="TAG_POLYPHONY" translation="fixed_value" translationValue="1"/>
  </state>
</button>
```

1.9 The <buses> element

As of version 1.12.0, DecentSampler has support for audio buses, which can be used to create a more complex mix of the samples in the sample library as well as route audio to various audio outputs. Sample library designers can specify up to 16 buses in the <buses> element using the <bus> tag. Each bus can have its own volume and audio output settings.

1.9.1 The <bus> element

Within the <buses> element, you can have up to 16 <bus> sub-elements. These specify parameters for each individual bus that you would like to have in your sample library. The <bus> element has the following attributes:

- **busVolume:** The volume of the bus. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.
- **output1Target:** The first audio output for this bus. This is a string that specifies the audio output that the bus should be routed to. The available options are MAIN_OUTPUT (the main audio output), AUX_STEREO_OUTPUT_1 (auxiliary output 1), AUX_STEREO_OUTPUT_2 (auxiliary output 2), AUX_STEREO_OUTPUT_3 (auxiliary output 3), AUX_STEREO_OUTPUT_4 (auxiliary output 4), AUX_STEREO_OUTPUT_5 (auxiliary output 5), AUX_STEREO_OUTPUT_6 (auxiliary output 6), AUX_STEREO_OUTPUT_7 (auxiliary output 7), AUX_STEREO_OUTPUT_8 (auxiliary output 8), AUX_STEREO_OUTPUT_9 (auxiliary output 9), AUX_STEREO_OUTPUT_10 (auxiliary output 10), AUX_STEREO_OUTPUT_11 (auxiliary output 11), AUX_STEREO_OUTPUT_12 (auxiliary output 12), AUX_STEREO_OUTPUT_13 (auxiliary output 13), AUX_STEREO_OUTPUT_14 (auxiliary output 14), AUX_STEREO_OUTPUT_15 (auxiliary output 15), and AUX_STEREO_OUTPUT_16 (auxiliary output 16).
- **output2Target:** The second audio output for this bus. This is a string that specifies the audio output that the bus should be routed to. The available options are the same as for output1Target.
- **output3Target:** The third audio output for this bus. This is a string that specifies the audio output that the bus should be routed to. The available options are the same as for output1Target.
- **output4Target:** The fourth audio output for this bus. This is a string that specifies the audio output that the bus should be routed to. The available options are the same as for output1Target.
- **output5Target:** The fifth audio output for this bus. This is a string that specifies the audio output that the bus should be routed to. The available options are the same as for output1Target.
- **output6Target:** The sixth audio output for this bus. This is a string that specifies the audio output that the bus should be routed to. The available options are the same as for output1Target.
- **output7Target:** The seventh audio output for this bus. This is a string that specifies the audio output that the bus should be routed to. The available options are the same as for output1Target.
- **output8Target:** The eighth audio output for this bus. This is a string that specifies the audio output that the bus should be routed to. The available options are the same as for output1Target.
- **output1Volume:** The volume of the audio being sent to the bus' first output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.
- **output2Volume:** The volume of the audio being sent to the bus' second output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.
- **output3Volume:** The volume of the audio being sent to the bus' third output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.
- **output4Volume:** The volume of the audio being sent to the bus' fourth output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.

- **output5Volume:** The volume of the audio being sent to the bus' fifth output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.
- **output6Volume:** The volume of the audio being sent to the bus' sixth output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.
- **output7Volume:** The volume of the audio being sent to the bus' seventh output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.
- **output8Volume:** The volume of the audio being sent to the bus' eighth output. This is a floating-point number between 0.0 and 1.0, where 0.0 is silent and 1.0 is full volume.

Here is an example of a `<buses>` element with two buses:

```
<buses>
  <bus busVolume="0.5" output1Target="MAIN_OUTPUT" output2Target="AUX_STEREO_OUTPUT_1"
  ↪output1Volume="0.8" output2Volume="0.5"/>
  <bus busVolume="0.8" output1Target="AUX_STEREO_OUTPUT_2" output2Target="AUX_STEREO_
  ↪OUTPUT_3" output1Volume="0.6" output2Volume="0.7"/>
</buses>
```

In this example, there are two buses defined. The first bus has a volume of 0.5 and is routed to the main output with a volume of 0.8 and auxiliary output 1 with a volume of 0.5. The second bus has a volume of 0.8 and is routed to auxiliary outputs 2 and 3 with volumes of 0.6 and 0.7, respectively.

1.9.2 Effects and buses

Buses can also have effects applied to them. To apply an effect to a bus, you can use the `<effects>` element within the `<bus>` element. The `<effects>` element can contain one or more `<effect>` sub-elements, each of which specifies an effect to apply to the bus. Details on the attributes of the `<effect>` element can be found in the [Effects](#) section.

Here is an example of a `<bus>` element with an effect applied to it:

```
<bus busVolume="0.5" output1Target="MAIN_OUTPUT" output2Target="AUX_STEREO_OUTPUT_1"
  ↪output1Volume="0.8" output2Volume="0.5">
  <effects>
    <effect type="Reverb" wetDryMix="0.5" roomSize="0.5" damping="0.5"/>
  </effects>
</bus>
```

In this example, a reverb effect is applied to the bus with a wet/dry mix of 0.5, room size of 0.5, and damping of 0.5.

1.9.3 Using buses in the sample library

Once you have defined buses in your sample library, you can use them in the `<sample>` elements to route the audio of the samples to specific buses. To do this, you can use the `outputXTarget` attributes in the `<sample>`, `<group>`, or `<groups>` elements. The `outputXTarget` attributes specifies the audio output that the sample should be routed to. The available options are the same as for the `output1Target` attribute above, but, with additional options for the buses defined in the `<buses>` element using the format: `BUS_1`, `BUS_2`, ..., `BUS_16`.

Here is an example of a `<group>` element with samples routed to a bus:

```
<group output1Target="MAIN_OUTPUT" output1Volume="1.0" output2Target="BUS_1"
  ↪output2Volume="0.2">
  <sample path="Samples/Volca Keys Poly-V127-29-F0.wav" loNote="17" hiNote="17"
  ↪
```

(continues on next page)

(continued from previous page)

```

↪rootNote="17"/>
    <!-- More samples here -->
</group>

```

In this example, the audio of the samples is routed to both the main output and to the first bus defined in the `<buses>` element. The volume of the audio sent to the bus is 0.2 (20% of full volume).

1.10 Appendix A: The Color Format

Colors are represented throughout the **dspreset** files using an 8-digit ARGB color format. These are identical to web color hex codes except with an additional 2-digit hex number in front of them. The first two digits are a hexadecimal representation of alpha level with 00 being fully transparent, 80 being 50% transparent, and FF being fully opaque.

Examples:

- Black (solid): FF000000
- Black (90% transparency): E6000000
- Red (solid): FFFF0000
- Red (50% transparency): 80FF0000
- Blue (solid): FF0000FF

1.11 Appendix B: The `<binding>` element

Adding a binding to a UI control, MIDI handler, or modulator tells the DecentSampler engine that it should take input from a source and use it to change values in another part of the engine. An example of this would be a knob which controls the volume of a group, a CC controller that changes an effect parameter, or an LFO that modulates an effect parameter.

In order to set up a binding for a specific source, create a `<binding>` element within the source element.

In this example, a labeled knob is controlling the volume of the first group of samples (group 0):

```

<DecentSampler>
  <ui>
    <tab>
      <labeled-knob x="420" y="100" label="RT" type="float" minValue="0" maxValue="1"
↪value="0.3" textSize="20">
        <binding type="amp" level="group" position="0" parameter="AMP_VOLUME"
↪translation="linear" translationOutputMin="0" translationOutputMax="1.0" />
      </labeled-knob>
    </tab>
  </ui>
</DecentSampler>

```

Here's a full list of parameters for the `<binding>` element:

Attribute	Description	
type	This tells the engine what type of parameter this is. Valid values are: <code>amp</code> , <code>general</code> , <code>effect</code> , <code>control</code> , <code>note</code> , <code>note_binding</code> , <code>velocity_binding</code> , <code>button_state_binding</code> , <code>keyboard_color</code> , <code>modulator</code> , <code>note_sequence</code> .	Required
level	Valid values are <code>ui</code> , <code>instrument</code> , <code>group</code> , <code>tag</code> , <code>midi</code>	Required
position	The specific 0-based index of the element to be modified by this binding. If you are targeting a group, for example, the first group would be 0, the second group would be 1, etc.	Required
control	When a binding is targeting a control, this is the same thing as the <code>position</code> attribute. It is a specific 0-based index of the control to be modified by this binding. If you are targeting an group-level effect, this would specified the group under which the effect lives.	Optional
groupIndex	When a binding is targeting a group, this is the same thing as the <code>position</code> attribute. It is a specific 0-based index of the group to be modified by this binding. If you are targeting an group-level effect, this would specified the group under which the effect lives.	Optional
effectIndex	When a binding is targeting an effect, this is the same thing as the <code>position</code> attribute. It is a specific 0-based index of the effect to be modified by this binding. As an alternative to using an index, you can use the <code>effectTags</code> attribute (see below) to target effects by their tags name.	Optional
modulatorIndex	When a binding is targeting a modulator, this is the same thing as the <code>position</code> attribute. It is a specific 0-based index of the modulator to be modified by this binding. As an alternative to using an index, you can use the <code>modulatorTags</code> attribute (see below) to target modulators by their tags name.	Optional
tags	A comma-separated list of tags. The meaning depends on context: at <code>level="group"</code> , this selects the target group(s) by tag. For backward compatibility, <code>tags</code> is also promoted into the appropriate typed tag attribute (<code>groupTags</code> , <code>effectTags</code> , <code>modulatorTags</code> , or <code>controlTags</code>) when the corresponding typed attribute is not explicitly set. Prefer the typed attributes for new presets.	Optional
groupTags	A comma-separated list of group tags. At <code>level="group"</code> , targets all <code><group></code> elements whose <code>tags</code> attribute contains any of these tags. This is the explicit, typed form of the group-selection behavior that <code>tags</code> has always provided.	Optional
effectTags	A comma-separated list of effect tags. When <code>type="effect"</code> , targets all <code><effect></code> elements (at instrument or group level) whose <code>tags</code> attribute contains any of these tags. Use this instead of <code>effectIndex</code> when you want to address effects by name rather than position.	Optional
modulatorTags	A comma-separated list of modulator tags. When <code>type="modulator"</code> at <code>level="instrument"</code> , targets all modulator elements (<code><lfo></code> , <code><envelope></code> , etc.) whose <code>tags</code> attribute contains any of these tags. Use this instead of <code>modulatorIndex</code> when you want to address modulators by name rather than position.	Optional
controlTags	A comma-separated list of UI control tags. When <code>type="control"</code> at <code>level="ui"</code> , targets all UI controls whose <code>tags</code> attribute contains any of these tags. Use this instead of <code>controlIndex</code> when you want to address controls by name rather than position.	Optional
enable	A value that turns the binding on and off. Valid values are: <code>true</code> , <code>false</code> .	Optional
identity	A string identifying the specific parameter that you wish to change. If you are modulating based on tags, you would put the tag you are targeting here. See Appendix D for example.	Required
parameter	A token describing the specific kind of parameter that you wish to change. A list of controller parameters are below.	Required
translation	Valid values are <code>fixed_value</code> , <code>linear</code> and <code>table</code> . Explanation of both translation modes is in a separate section below. Default: <code>linear</code>	Optional
translationMin	This is the min value this binding should send to the target parameter. This is only looked at if translation is set to <code>linear</code> .	Optional
translationMax	This is the max value this binding should send to the target parameter. This is only looked at if translation is set to <code>linear</code> .	Optional
translationFixed	Valid values are <code>true</code> and <code>false</code> . Default: <code>false</code> . This is only looked at if translation is set to <code>linear</code> .	Optional
translationTable	A list of input-output pairs that make up the translation table. The input and output are separated by commas. The groups of coordinates themselves are separated by semi-colons. Default: <code>0,0;1,1</code> . You must have at least two coordinates in your list. This is only looked at if translation is set to <code>table</code> .	Optional

1.11.1 Binding Parameters for Targeting Note Sequences

A special set of binding attributes exist for targeting note sequences:

Attribute	Description	Default	Required
seqIndex	A 0-based index of a sequence underneath the <code><noteSequences></code> section	None	Required
seqFollow	Whether or not the sequence should follow the global tempo. Valid values are <code>true</code> and <code>false</code> . If this is set to <code>false</code> , then playback will be hardcoded at 120BPM. This can be useful if you want to assure that sequences will always play back at the same rate regardless of the DAW clock.	<code>true</code>	Optional
seqTrigger	What the binding should do with the sequence in question. Valid values are <code>on</code> (start playing the sequence), <code>off</code> (stop playing the sequence), <code>midi_key</code> (special value that will cause the binding to follow a specific MIDI key note binding)	<code>midi_key</code>	Optional
seqPlaye	An identifier used for tracking the state of a sequence. This value can be any sequence of numbers or letters.	None	Required when <code>seqTriggerBehavior</code> is <code>on</code> or <code>off</code>
seqTrack	Whether or not the sequence should respect the velocity of the incoming MIDI note. This can only be used when the sequence is being triggered by a MIDI note binding. Value should be a floating point number from 0.0 to 1.		Optional
seqTrans	Transpose the notes in the sequence by an arbitrary number of half steps. Value should be a floating point number from -36 to 36.	Any sequence of numbers or letters	Optional
seqTrans	Transpose the notes in the sequence relative to the pitch of the incoming MIDI note. This can only be used when the sequence is being triggered by a MIDI note binding. Value should be a floating point number from 0 to 127.	Any sequence of numbers or letters	Optional
seqPlayt	The speed of playback. Value should be a floating point number from 0.001 to 10000.	1.0	Optional
seqLoop	Valid values are: <code>forward</code> , <code>reverse</code> , <code>random</code> , <code>random_no_repeat</code> , <code>no_loop</code>	<code>forward</code>	Optional

1.11.2 Controllable Parameters

This is a list of parameters that can be used in conjunction with the `<binding>` element above. NOTE: The table below scrolls to the right.

Description	type	level	parameter
Global Volume	<code>amp</code>	<code>instrument</code>	<code>AMP_VOLUME</code>
Global Tuning	<code>amp</code>	<code>instrument</code>	<code>GLOBAL_TUNING</code>
Global Pan	<code>amp</code>	<code>instrument</code>	<code>PAN</code>
Sample Start (see note 2 below)	<code>general</code>	<code>instrument or group</code>	<code>SAMPLE_START</code>
Sample End (see note 2 below)	<code>general</code>	<code>instrument or group</code>	<code>SAMPLE_END</code>
Loop Start (see note 2 below)	<code>general</code>	<code>instrument or group</code>	<code>LOOP_START</code>

Description	type	level	parameter
Loop End (see note 2 below)	general	instrument or group	LOOP_END
Low Note	general	instrument or group	LO_NOTE
High Note	general	instrument or group	HI_NOTE
Low Velocity	general	group	LO_VEL
High Velocity	general	group	HI_VEL
Root Note	general	instrument or group	ROOT_NOTE
Silencing Decay	general	instrument or group	SILENCING_DECAY
Silencing Mode	general	instrument or group	SILENCING_MODE
Pitch Key Track	general	instrument or group	PITCH_KEY_TRACK
Amplitude Velocity Tracking	amp	instrument	AMP_VEL_TRACK
Global Amp Envelope Attack	amp	instrument	ENV_ATTACK
Global Amp Envelope Attack Curve Shape	amp	instrument	ENV_ATTACK_CURV
Global Amp Envelope Decay	amp	instrument	ENV_DECAY
Global Amp Envelope Decay Curve Shape	amp	instrument	ENV_DECAY_CURVE
Global Amp Envelope Sustain	amp	instrument	ENV_SUSTAIN
Global Amp Envelope Release	amp	instrument	ENV_RELEASE
Global Amp Envelope Release Curve Shape	amp	instrument	ENV_RELEASE_CUR
Glide/Portamento Time	amp	instrument	GLIDE_TIME
Group Enabled / Disabled	amp	group	ENABLED
Group Amplitude Envelope Enabled / Disabled	amp	group	AMP_ENV_ENABLED
Group Volume	amp	group	AMP_VOLUME
Group Tuning	amp	group	GROUP_TUNING
Pan	amp	group	PAN
Amplitude Velocity Tracking	amp	group	AMP_VEL_TRACK
Group Amp Envelope Attack	amp	group	ENV_ATTACK
Group Amp Envelope Decay	amp	group	ENV_DECAY
Group Amp Envelope Sustain	amp	group	ENV_SUSTAIN
Group Amp Envelope Release	amp	group	ENV_RELEASE
Group Output 1 Volume	amp	group	OUTPUT_1_VOLUME
Group Output 2 Volume	amp	group	OUTPUT_2_VOLUME
Group Output 3 Volume	amp	group	OUTPUT_3_VOLUME
Group Output 4 Volume	amp	group	OUTPUT_4_VOLUME
Group Output 5 Volume	amp	group	OUTPUT_5_VOLUME
Group Output 6 Volume	amp	group	OUTPUT_6_VOLUME
Group Output 7 Volume	amp	group	OUTPUT_7_VOLUME
Group Output 8 Volume	amp	group	OUTPUT_8_VOLUME
Group Output 1 Target	amp	group	OUTPUT_1_TARGET
Group Output 2 Target	amp	group	OUTPUT_2_TARGET
Group Output 3 Target	amp	group	OUTPUT_3_TARGET
Group Output 4 Target	amp	group	OUTPUT_4_TARGET
Group Output 5 Target	amp	group	OUTPUT_5_TARGET
Group Output 6 Target	amp	group	OUTPUT_6_TARGET
Group Output 7 Target	amp	group	OUTPUT_7_TARGET
Group Output 8 Target	amp	group	OUTPUT_8_TARGET
Group Glide/Portamento Time	amp	group	GLIDE_TIME
Oscillator Waveform (group level)	general	group	OSCILLATOR_WAVE
Oscillator Damping (pluck1 only)	general	group	OSCILLATOR_DAMP
Oscillator Pluck Type (pluck1 only)	general	group	OSCILLATOR_PLUC
Oscillator Wavetable Position (wavetable only)	general	group	OSCILLATOR_WAVE
Oscillator Wavetable Frame Interpolation (wavetable only)	general	group	OSCILLATOR_WAVE

Description	type	level	parameter
Harmonic Oscillator Partial Count (harmonic only)	general	group	OSCILLATOR_HARM
Harmonic Oscillator Tilt (harmonic only)	general	group	OSCILLATOR_HARM
Harmonic Oscillator Odd/Even Balance (harmonic only)	general	group	OSCILLATOR_HARM
Harmonic Oscillator Normalization (harmonic only)	general	group	OSCILLATOR_HARM
Harmonic Oscillator Partial Levels (harmonic only)	general	group	OSCILLATOR_HARM
FM6 Algorithm (fm6op only)	general	group	OSCILLATOR_FM_A
FM6 Operator 1 Level (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 2 Level (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 3 Level (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 4 Level (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 5 Level (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 6 Level (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 1 Ratio (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 2 Ratio (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 3 Ratio (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 4 Ratio (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 5 Ratio (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 6 Ratio (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 1 Feedback (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 2 Feedback (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 3 Feedback (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 4 Feedback (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 5 Feedback (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 6 Feedback (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 1 Detune (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 2 Detune (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 3 Detune (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 4 Detune (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 5 Detune (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 6 Detune (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 1 Mode (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 2 Mode (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 3 Mode (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 4 Mode (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 5 Mode (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 6 Mode (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 1 Fixed Frequency (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 2 Fixed Frequency (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 3 Fixed Frequency (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 4 Fixed Frequency (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 5 Fixed Frequency (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 6 Fixed Frequency (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 1 Velocity Sensitivity (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 2 Velocity Sensitivity (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 3 Velocity Sensitivity (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 4 Velocity Sensitivity (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 5 Velocity Sensitivity (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 6 Velocity Sensitivity (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 1 Attack (fm6op only)	general	group	OSCILLATOR_FM_O
FM6 Operator 2 Attack (fm6op only)	general	group	OSCILLATOR_FM_O

Description	type	level	parameter
FM6 Operator 3 Attack (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 Attack (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 Attack (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 Attack (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 Decay (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 Decay (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 Decay (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 Decay (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 Decay (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 Decay (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 Sustain (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 Sustain (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 Sustain (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 Sustain (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 Sustain (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 Sustain (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 Release (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 Release (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 Release (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 Release (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 Release (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 Release (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Type (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Type (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Type (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Type (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Type (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 EG Type (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Rate 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Rate 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Rate 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Rate 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Rate 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Rate 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Rate 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Rate 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Rate 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Rate 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Rate 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Rate 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Rate 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Rate 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Rate 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Rate 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Rate 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Rate 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Rate 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Rate 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 EG Rate 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 EG Rate 2 (fm6op only)	general	group	OSCILLATOR_FM_0

Description	type	level	parameter
FM6 Operator 6 EG Rate 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 EG Rate 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Level 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Level 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Level 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 1 EG Level 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Level 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Level 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Level 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 2 EG Level 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Level 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Level 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Level 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 3 EG Level 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Level 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Level 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Level 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 4 EG Level 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Level 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Level 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Level 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 5 EG Level 4 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 EG Level 1 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 EG Level 2 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 EG Level 3 (fm6op only)	general	group	OSCILLATOR_FM_0
FM6 Operator 6 EG Level 4 (fm6op only)	general	group	OSCILLATOR_FM_0
Tag Enabled	amp	tag	TAG_ENABLED
Tag Volume	amp	tag	TAG_VOLUME
Tag Polyphony	general	tag	TAG_POLYPHONY
MIDI CC Binding Enabled	cc_binding	midi	ENABLED
MIDI Note Mapping Enabled	note	midi	ENABLED
MIDI Note Binding Enabled	note_binding	midi	ENABLED
MIDI Note Binding Change seqIndex	note_binding	midi	SEQ_INDEX
MIDI Note Binding Change seqLoopMode	note_binding	midi	SEQ_LOOP_MODE
MIDI Note Binding Change seqTransposeWithRootNote	note_binding	midi	SEQ_TRANSPOSE_W
MIDI Note Binding Change seqPlaybackRate	note_binding	midi	SEQ_PLAYBACK_RA
MIDI Note Binding Change seqTrackMidiInputVelocity	note_binding	midi	SEQ_TRACK_MIDI_
MIDI Note Binding Change seqTranspose	note_binding	midi	SEQ_TRANSPOSE
MIDI Velocity Binding Enabled	velocity_binding	midi	ENABLED
Modulator Amount (Depth)	modulator	instrument	MOD_AMOUNT
LFO Modulator Rate (or Frequency)	modulator	instrument	FREQUENCY
LFO Modulator Delay Time	modulator	instrument	MOD_DELAY_TIME
Envelope Modulator Attack	modulator	instrument	ENV_ATTACK
Envelope Modulator Attack Curve Shape	modulator	instrument	ENV_ATTACK_CURV
Envelope Modulator Decay	modulator	instrument	ENV_DECAY
Envelope Modulator Decay Curve Shape	modulator	instrument	ENV_DECAY_CURVE
Envelope Modulator Sustain	modulator	instrument	ENV_SUSTAIN
Envelope Modulator Release	modulator	instrument	ENV_RELEASE
Envelope Modulator Release Curve Shape	modulator	instrument	ENV_RELEASE_CUR
Sequence Rate	note_sequence	instrument	RATE

Description	type	level	parameter
All Notes Off	general	instrument	ALL_NOTES_OFF

UI Parameters

NOTE: The table below scrolls to the right.

De- scrip- tion	type	leve	para	Valid Range	Mod- u- lat- able	Additional required parameters
UI Back- ground Image Path	gener	ui	BG_I	Text		
UI Button State Bind- ing En- abled	butt	ui	ENAB	true, false		tags or controlIndex contains the 0-based index of the control in question (see note 1 below), stateIndex contains the 0-based index of the state in question, and bindingIndex that contains the index of the binding being referenced.
UI Control En- abled	contr	ui	ENAB	true, false		tags or controlIndex/position contains the 0-based index of the control in question (see note 1 below)
UI Control Visible	contr	ui	VISI	true, false		tags or controlIndex or position contains the 0-based index of the control in question (see note 1 below)
UI Control Value	contr	ui	VALU	Any number		tags or controlIndex or position contains the 0-based index of the control in question (see note 1 below)
UI Control Text	contr	ui	TEXT	Text		tags or controlIndex or position contains the 0-based index of the control in question (see note 1 below)
UI Control Mini- mum Value	contr	ui	MIN_	Any number		tags or controlIndex or position contains the 0-based index of the control in question (see note 1 below)
UI Control Max- imum Value	contr	ui	MAX_	Any number		tags or controlIndex or position contains the 0-based index of the control in question (see note 1 below)
UI Control Value Type	contr	ui	VALU	float, integer, musical_time		tags or controlIndex or position contains the 0-based index of the control in question (see note 1 below)
Image or Ani- mation File	contr	ui	PATH	Text		tags or controlIndex or position contains the 0-based index of the control in question (see note 1 below)
Path Image or Ani- mation Opac-	contr	ui	OPAC	0.0 - 1.0		tags or controlIndex or position contains the 0-based index of the control in question (see note 1 below)

1. NOTE: The indexes of the controls within the UI also include UI controls that are not editable, such as `<label>` elements, so you'll want to account for that when calculating your positions.

Here's a quick example:

If your UI's `<tab>` section has the following elements under it: `<label>`, `<control>`, `<label>`, `<control>`. The position indexes of the four elements will be 0, 1, 2, 3. Therefore, the indexes of the two `<control>` elements will be 1 and 3, respectively.

It's also possible to use tags to address several controls at once. For example, if you have several `<control>` elements with the tag `some-controls`, you can use the following binding to address all of those controls in a single binding:

```
<binding type="control" level="ui" tags="some-controls" parameter="VISIBLE"
↪translation="fixed_value" translationValue="true"></binding>
```

Effects Parameters

NOTE: The table below scrolls to the right.

Description	type	level	parameter	Valid Range	Modulatable	Additional required parameters	
Effect Enabled (all effects)	effe	instru	ENABLED	false, true	Yes	effectIndex, position, or tags identifies the target effect(s)	
Convolution Mix Level	effe	instru	FX_MIX	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Convolution IR File	effe	instru	FX_IR_FILE	Text	No	effectIndex, position, or tags identifies the target effect(s)	
Filter Frequency (for several filters)	effe	instru	FX_FILTER_F	0.0 - 22000.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Peak or Notch Filter Q	effe	instru	FX_FILTER_C	0.01 - 18.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Peak or Notch Filter Gain	effe	instru	FX_FILTER_G	0.0 - 10.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Low-pass or High-pass Filter Resonance	effe	instru	FX_FILTER_R	0.0 - 5.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Reverb Wet Level	effe	instru	FX_REVERB_W	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Reverb Room Size	effe	instru	FX_REVERB_R	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Reverb Damping	effe	instru	FX_REVERB_D	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Chorus/Phaser/Pitch-Shift/Convolution Mix Level	effe	instru	FX_MIX	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Chorus/Phaser Depth	Mod	effe	instru	FX_MOD_DEPTH	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)
Chorus/Phaser Rate	Mod	effe	instru	FX_MOD_RATE	0.0 - 10.0	Yes	effectIndex, position, or tags identifies the target effect(s)
Phaser Center Frequency	effe	instru	FX_CENTER_F	0.0 - 22000.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Phaser/Delay Feedback	effe	instru	FX_FEEDBACK	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Delay Time	effe	instru	FX_DELAY_TI	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Delay Time Format	effe	instru	FX_DELAY_TI	seconds, musical_time	Yes	effectIndex, position, or tags identifies the target effect(s)	
Delay Stereo Offset	effe	instru	FX_STEREO_C	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target effect(s)	
Delay Wet Level	effe	instru	FX_WET_LEVE	0.0 - 1.0	Yes	effectIndex, position, or tags identifies the target ef-	

74

Chapter 1. Contents

Bus-related Parameters

NOTE: The table below scrolls to the right.

De- scrip- tion	type	leve	parameter	Valid Range	Mod- u- lat- able	Additional required param- eters
Bus Vol- ume	amp	bus	BUS_VOLUME	0.0 - 16.0	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 1 Vol- ume	amp	bus	OUTPUT_1_VOLUME	0.0 - 1	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 2 Vol- ume	amp	bus	OUTPUT_2_VOLUME	0.0 - 1	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 3 Vol- ume	amp	bus	OUTPUT_3_VOLUME	0.0 - 1	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 4 Vol- ume	amp	bus	OUTPUT_4_VOLUME	0.0 - 1	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 5 Vol- ume	amp	bus	OUTPUT_5_VOLUME	0.0 - 1	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 6 Vol- ume	amp	bus	OUTPUT_6_VOLUME	0.0 - 1	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 7 Vol- ume	amp	bus	OUTPUT_7_VOLUME	0.0 - 1	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 8 Vol- ume	amp	bus	OUTPUT_8_VOLUME	0.0 - 1	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 1 Target	amp	bus	OUTPUT_1_TARGET	Any of the valid values for output1Target	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 2 Target	amp	bus	OUTPUT_2_TARGET	Any of the valid values for output2Target	No	busIndex or position con- tains the 0-based index of the bus
Bus Out- put 3 Target	amp	bus	OUTPUT_3_TARGET	Any of the valid values for output3Target	No	busIndex or position con- tains the 0-based index of the bus

2. NOTE: If your sample library manipulates `start`, `end`, `loopStart`, or `loopEnd` after a sample library's initial load, the sample playback engine must be in RAM/Memory mode (not disk streaming) or you will get very unpredictable results. In order to enforce this, sample creators should use the `playbackMode` attribute.

1.11.3 Translation Modes

There are currently three binding translation modes: `linear`, `table`, `fixed_value`

Mode #1: `linear`

linear mode allows values that come in to be scaled up or down before they get passed along to the binding's target. If you set your translation mode to `linear` you should also `translationOutputMin` and `translationOutputMax`.

Example usage:

```
<binding level="ui" type="control" position="0" parameter="value" translation="linear"
translationOutputMin="0" translationOutputMax="1"/>
```

Mode #2: `table`

table mode allows you to transform the binding's input in a more complex fashion before it gets passed along to the binding's target. If you set your translation mode to `table` you must define the `translationTable` parameter as well. This consists of a series of input-output pairs, separated by semi-colons.

Mode #3: `fixed_value`

fixed_value mode allows you to completely disregard the input of a binding and instead always use a supplied value. In order to use this translation mode, you must also specify a `translationValue`. This can be very useful when trying to have menu options enable and disable groups. An example usage:

```
<binding type="general" level="group" position="1" parameter="ENABLED" translation=
↪ "fixed_value" translationValue="true" />
```

1.12 Appendix C: Boilerplate .dspreset File

```
<?xml version="1.0" encoding="UTF-8"?>
<DecentSampler minVersion="1.0.0">
  <ui width="812" height="375" layoutMode="relative"
    bgMode="top_left">
    <tab name="main">
      <labeled-knob x="445" y="75" width="90" textSize="16" textColor="AA000000"
        trackForegroundColor="CC000000" trackBackgroundColor="66999999"
        label="Attack" type="float" minValue="0.0" maxValue="4.0" value="0.01
↪ " >
        <binding type="amp" level="instrument" position="0" parameter="ENV_ATTACK" />
      </labeled-knob>
      <labeled-knob x="515" y="75" width="90" textSize="16" textColor="AA000000"
        trackForegroundColor="CC000000" trackBackgroundColor="66999999"
```

(continues on next page)

(continued from previous page)

```

        label="Release" type="float" minValue="0.0" maxValue="20.0" value="1
→ " >
    <binding type="amp" level="instrument" position="0" parameter="ENV_RELEASE" />
</labeled-knob>
<labeled-knob x="585" y="75" width="90" textSize="16" textColor="AA000000"
    trackForegroundColor="CC000000" trackBackgroundColor="66999999"
    label="Chorus" type="float" minValue="0.0" maxValue="1" value="0" >
    <binding type="effect" level="instrument" position="1" parameter="FX_MIX" />
</labeled-knob>
<labeled-knob x="655" y="75" width="90" textSize="16" textColor="FF000000"
    trackForegroundColor="CC000000" trackBackgroundColor="66999999"
    label="Tone" type="float" minValue="0" maxValue="1" value="1">
    <binding type="effect" level="instrument" position="0" parameter="FX_FILTER_
→ FREQUENCY"
        translation="table"
        translationTable="0,33;0.3,150;0.4,450;0.5,1100;0.7,4100;0.9,11000;1.
→ 0001,22000"/>
</labeled-knob>
<labeled-knob x="725" y="75" width="90" textSize="16" textColor="AA000000"
    trackForegroundColor="CC000000" trackBackgroundColor="66999999"
    label="Reverb" type="percent" minValue="0" maxValue="100"
    textColor="FF000000" value="50">
    <binding type="effect" level="instrument" position="2"
        parameter="FX_REVERB_WET_LEVEL" translation="linear"
        translationOutputMin="0" translationOutputMax="1" />
</labeled-knob>
</tab>
</ui>
<groups attack="0.000" decay="25" sustain="1.0" release="0.430" volume="-3dB">
    <group>
        <sample loNote="21" hiNote="21" rootNote="21" path="DefaultPiano-21.aif"
            length="805888"/>
        <sample loNote="22" hiNote="33" rootNote="33" path="DefaultPiano-33.aif"
            length="807552"/>
        <sample loNote="34" hiNote="45" rootNote="45" path="DefaultPiano-45.aif"
            length="759168"/>
        <sample loNote="46" hiNote="57" rootNote="57" path="DefaultPiano-57.aif"
            length="756480"/>
        <sample loNote="58" hiNote="69" rootNote="69" path="DefaultPiano-69.aif"
            length="758656"/>
        <sample loNote="70" hiNote="77" rootNote="77" path="DefaultPiano-77.aif"
            length="595328"/>
        <sample loNote="78" hiNote="89" rootNote="89" path="DefaultPiano-89.aif"
            length="457600"/>
        <sample loNote="90" hiNote="96" rootNote="96" path="DefaultPiano-96.aif"
            length="469888"/>
        <sample loNote="94" hiNote="108" rootNote="108" path="DefaultPiano-108.aif"
            length="75264"/>
    </group>
</groups>
<effects>
    <effect type="lowpass" frequency="22000.0"/>

```

(continues on next page)

(continued from previous page)

```

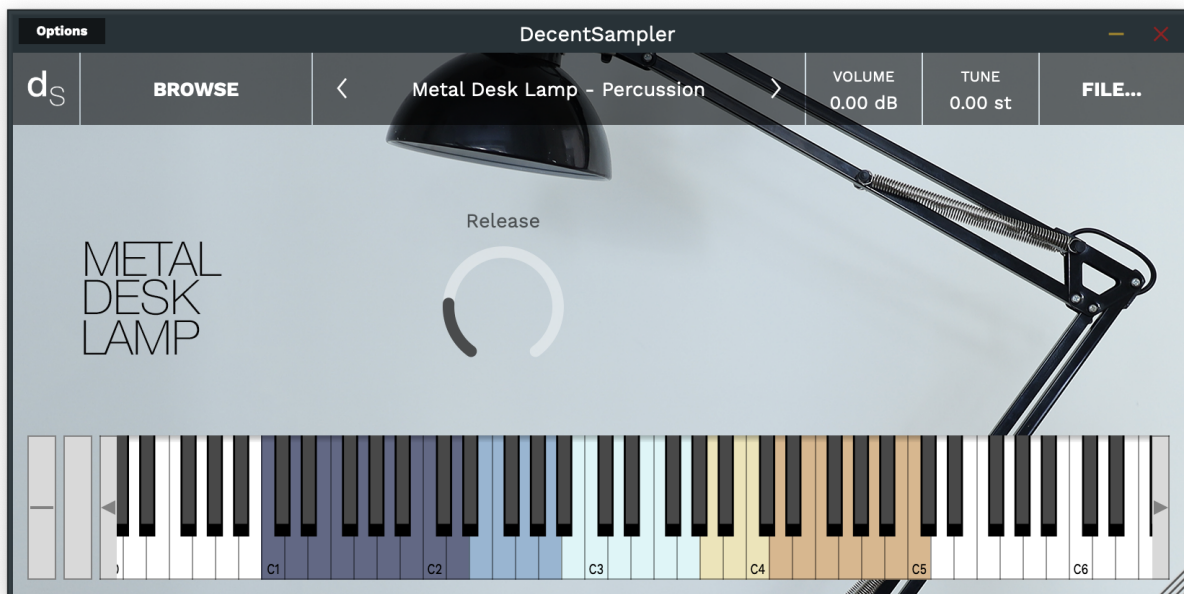
<effect type="chorus" mix="0.0" modDepth="0.2" modRate="0.2" />
<effect type="reverb" wetLevel="0.5"/>
</effects>
<midi>
  <!-- This causes MIDI CC 1 to control the 4th knob (cutoff) -->
  <cc number="1">
    <binding level="ui" type="control" parameter="VALUE" position="3"
      translation="linear" translationOutputMin="0"
      translationOutputMax="1" />
  </cc>
</midi>
</DecentSampler>

```

1.13 Useful Tutorials and Resources

1.13.1 UI Elements

How to Color the Keys of the On-Screen Keyboard



As of version 1.4.0, it is now possible to color the keys of the on-screen keyboard.

This can be useful for showing different ranges of notes that serve different purposes or highlighting notes used as keyswitches. In order to implement colored keys, make use of the new `<keyboard>` and `<color>` elements as follows:

```

<DecentSampler>
  <ui>
    <!-- Other stuffhere -->
    <keyboard>
      <color loNote="36" hiNote="50" color="FF2C365E" />

```

(continues on next page)

(continued from previous page)

```

        <color loNote="51" hiNote="57" color="FF6D9DC5" />
        <color loNote="58" hiNote="67" color="FFCCF3F5" />
        <color loNote="68" hiNote="73" color="FFE8DA9B" />
        <color loNote="74" hiNote="84" color="FFD19D61" />
    </keyboard>
</ui>
<!-- Other stuff here -->
</DecentSampler>

```

By default, Decent Sampler highlights all of the notes that are mapped for a given sample. If you use the color keys feature, this default highlighting will be turned off and it will be up to you to color whatever keys you want.

Full documentation for the color element is [here](#).

How to turn keyboard note coloring on and off using bindings

It's also possible to programmatically turn keyboard coloring on and off using bindings. Here's an example of how to do that:

```

<DecentSampler>
  <ui>
    <button name="Strum Enabled" x="464" y="82" width="14" height="14" style="image"
    ↪value="1" visible="false">
      <state name="Off" mainImage="Images/StrumsCheckboxUnchecked.png" hoverImage=
    ↪"Images/StrumsCheckboxUnchecked.png" clickImage="Images/StrumsCheckboxUnchecked.png">
        <binding level="ui" type="keyboard_color" colorIndex="0" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="false"/>
        <binding level="ui" type="keyboard_color" colorIndex="1" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="false"/>
        <binding level="ui" type="keyboard_color" colorIndex="2" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="false"/>
        <binding level="ui" type="keyboard_color" colorIndex="3" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="false"/>
        <binding level="ui" type="keyboard_color" colorIndex="4" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="false"/>
      </state>
      <state name="On" mainImage="Images/StrumsCheckboxChecked.png" hoverImage="Images/
    ↪StrumsCheckboxChecked.png" clickImage="Images/StrumsCheckboxChecked.png">
        <binding level="ui" type="keyboard_color" colorIndex="0" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="true"/>
        <binding level="ui" type="keyboard_color" colorIndex="1" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="true"/>
        <binding level="ui" type="keyboard_color" colorIndex="2" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="true"/>
        <binding level="ui" type="keyboard_color" colorIndex="3" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="true"/>
        <binding level="ui" type="keyboard_color" colorIndex="4" parameter="ENABLED"
    ↪translation="fixed_value" translationValue="true"/>
      </state>
    </button>
    <keyboard>
      <color loNote="36" hiNote="50" color="FF2C365E" />

```

(continues on next page)

(continued from previous page)

```

        <color loNote="51" hiNote="57" color="FF6D9DC5" />
        <color loNote="58" hiNote="67" color="FFCCF3F5" />
        <color loNote="68" hiNote="73" color="FFE8DA9B" />
        <color loNote="74" hiNote="84" color="FFD19D61" />
    </keyboard>
</ui>
<!-- Other stuff here -->
</DecentSampler>

```

How to add Dropdown Menus

Dropdown Menus

In order to implement dropdown menus, use the new `<menu>` and `<option>` elements. The `<menu>` element defines where the dropdown menu will show up in the ui, whereas the XML elements determine what menu options it has and what if anything those options actually do:

```

<menu x="10" y="40" width="120" height="30" value="2">
  <option name="Menu Option 1">
    <!-- Turn on this group -->
    <binding type="general" level="group" position="0" parameter="ENABLED"
      translation="fixed_value" translationValue="true" />
    <!-- Turn off this group -->
    <binding type="general" level="group" position="1" parameter="ENABLED"
      translation="fixed_value" translationValue="false" />
  </option>
  <option name="Menu Option 2">
    <!-- Turn off this group -->
    <binding type="general" level="group" position="0" parameter="ENABLED"
      translation="fixed_value" translationValue="false" />
    <!-- Turn on this group -->
    <binding type="general" level="group" position="1" parameter="ENABLED"
      translation="fixed_value" translationValue="true" />
  </option>
</menu>

```

In this example, a menu is being used to switch between two groups (the first menu option turns group 0 on and group 1 off; the second option turns group 0 off and group 1 on). Full documentation for the new `<menu>` and `<option>` elements is [here](#).

The new `fixed_value` translation type

You'll note, in the example above, there's something new in the bindings: the four bindings elements have a `translation` parameter of type `fixed_value`. This is a new translation type. Up until now, binding translation has strictly been about taking an input parameter (such as a knob value or continuous controller amount) and translating it so that it is useful for some other purpose (it's our way of being able to do a little bit of math without having a full-blown scripting language). This new `fixed_value` binding is different. It ignores the input value completely and instead provides whatever is specified in the `translationValue` parameter. In this way, each menu option can have hardcoded values that it provides its bindings when it is selected.

Customizing Menu Colors

You can customize the appearance of dropdown menus using the `textColor`, `backgroundColor`, `highlightedTextColor`, and `highlightedBackgroundColor` attributes:

```
<menu x="10" y="40" width="120" height="30" value="1"
      textColor="FFFFFF" backgroundColor="FF333333"
      highlightedTextColor="FF000000" highlightedBackgroundColor="FFCCCC">
  <option name="Option 1">
    <!-- binding code here -->
  </option>
  <option name="Option 2">
    <!-- binding code here -->
  </option>
</menu>
```

Attribute	Description
<code>textColor</code>	A hex ARGB color value for the menu text (e.g., “FFFFFF” for white text)
<code>backgroundColor</code>	A hex ARGB color value for the menu background (e.g., “FF333333” for dark gray)
<code>highlightedTextColor</code>	A hex ARGB color value for the highlighted menu text (e.g., “FF000000” for black)
<code>highlightedBackgroundCo</code>	A hex ARGB color value for the highlighted menu background (e.g., “FFCCCC” for light gray)

Text Alignment

You can control the text alignment within the menu using the `vAlign` and `hAlign` attributes, similar to labels:

```
<menu x="10" y="40" width="120" height="30" value="1"
      vAlign="center" hAlign="center">
  <option name="Centered Option">
    <!-- binding code here -->
  </option>
</menu>
```

Attribute	Valid Values	Description	Default
<code>vAlign</code>	“top”, “center”, “bottom”	Vertical alignment of the menu text	“center”
<code>hAlign</code>	“left”, “center”, “right”	Horizontal alignment of the menu text	“left”

Dynamic Control via Bindings

All color attributes can also be controlled dynamically using bindings. The color values should be specified as 8-character hex strings in ARGB format (Alpha, Red, Green, Blue).

```
<!-- Example: Dynamic color control -->
<control x="200" y="40" width="100" height="30">
  <binding type="control" level="ui" position="0" parameter="HIGHLIGHTED_TEXT_COLOR"
    translation="linear" translationOutputMin="FF000000"
```

(continues on next page)

(continued from previous page)

```
translationOutputMax="FFFFFFF" />
</control>
```

Available binding parameters for menu color control:

- **TEXT_COLOR**: Controls the menu text color
- **BACKGROUND_COLOR**: Controls the menu background color
- **HIGHLIGHTED_TEXT_COLOR**: Controls the highlighted menu text color
- **HIGHLIGHTED_BACKGROUND_COLOR**: Controls the highlighted menu background color

How to Use Animations

As of version 1.12.9, DecentSampler has support for simple animations. You can use animations to create more compelling user interfaces. You can also use animations to create visual feedback for your controls. For example, a knob might control an animated visual element that rotates as the knob is turned.

Animations are defined using the `<multiFrameImage>` tag. This tag allows you to specify a series of images that will be displayed in sequence. It uses the same sprite sheet format that is used elsewhere in DecentSampler for defining knob skins. The `<multiFrameImage>` tag has the following attributes:

- **x** (required): The x position of your image where (0,0) is the top-left corner
- **y** (required): The y position of your image where (0,0) is the top-left corner
- **width** (required): The width in pixels of the image component
- **height** (required): The height in pixels of the image component
- **path** (required): The relative path of the image file to show in this component
- **numFrames** (required): The number of frames in the animation
- **frameRate** (required): The frame rate of the animation in frames per second. The maximum supported frame rate is 24 frames per second.
- **sourceFormat** (required): The orientation of the frames within the image strip. Valid values: `horizontal_image_strip`, `vertical_image_strip`.
- **playbackMode** (optional): The direction in which the animation should play. Valid values: `forward_loop`, `forward_once`, `reverse_loop`, `reverse_once`, `ping_pong_loop` (forth and back), and `stopped`. Default value is `forward_loop`.
- **visible** (optional): This controls whether or not this image is visible. There are two valid values: `true` (default), `false`.
- **tooltip** (optional): A tool tip to display when the user hovers over this image.

Here is an example of a simple animation that loops continuously:

```
<DecentSampler pluginVersion="1">
  <ui bgColor="FFADD8E6">
    <tab>
      <multiFrameImage x="350" y="80" width="64" height="64" path="Images/
↳ AnimationDemo128.png" numFrames="31" sourceFormat="vertical_image_strip" frameRate="24"
↳ playbackMode="forward_loop"/>
    </tab>
```

(continues on next page)

(continued from previous page)

```

</ui>
</DecentSampler>

```

In this example, the image `Images/AnimationDemo128.png` is a vertical image strip with 31 frames. The animation will play at 24 frames per second in a forward loop. The image will be displayed at position (10,10) with a width of 64 pixels and a height of 64 pixels.

Controlling Animations

Current Frame

You can control the current frame of an animation using the `currentFrame` attribute. This attribute is a zero-based index that specifies the current frame of the animation. You can use this attribute to create animations that are controlled by other components. For example, you could use a knob to control the current frame of an animation.

Here is an example of an animation that is controlled by a knob:

```

<DecentSampler>
  <ui bgColor="FFADD8E6">
    <tab>
      <multiFrameImage x="450" y="80" width="64" height="64" path="Images/
      ↪ AnimationDemo128.png" numFrames="31" sourceFormat="vertical_image_strip" frameRate="24" ↪
      ↪ playbackMode="stopped"/>
      <labeled-knob x="280" y="50" label="Frame" type="integer" minValue="0" maxValue="31
      ↪ " value="0" textColor="FF000000" value="0">
        <binding type="control" level="ui" position="0" parameter="CURRENT_FRAME" ↪
        ↪ translation="linear" translationOutputMin="0" translationOutputMax="31" />
      </labeled-knob>
    </tab>
  </ui>
</DecentSampler>

```

In this example, the knob controls the current frame of the animation. The `binding` tag is used to bind the knob to the `CURRENT_FRAME` parameter of the animation. The `translation` attribute specifies how the knob value should be translated to the animation frame. In this case, the knob value is linearly translated to the range 0-31, which corresponds to the 31 frames of the animation.

You can download a working example of this code [here](#). This is Example 2 in the `example-011-how-to-use-animations` folder.

Frame Rate

You can control the frame rate of an animation using the `frameRate` attribute. This attribute specifies the number of frames per second that the animation should play at. You can use this attribute to create animations that play at different speeds. For example, you could use a slider to control the frame rate of an animation.

Here is an example of an animation that is controlled by a slider:

```

<DecentSampler>
  <ui bgColor="FFADD8E6">
    <tab>
      <multiFrameImage x="450" y="80" width="64" height="64" path="Images/

```

(continues on next page)

(continued from previous page)

```

↪ AnimationDemo128.png" numFrames="31" sourceFormat="vertical_image_strip" frameRate="24"
↪ playbackMode="forward_loop"/>
    <labeled-knob x="280" y="50" label="Frame Rate" type="integer" minValue="1"
↪ maxValue="60" value="24" textColor="FF000000" value="24">
        <binding type="control" level="ui" position="0" parameter="FRAME_RATE"
↪ translation="linear" translationOutputMin="1" translationOutputMax="24" />
    </labeled-knob>
</tab>
</ui>
</DecentSampler>

```

In this example, the slider controls the frame rate of the animation. The `binding` tag is used to bind the slider to the `FRAME_RATE` parameter of the animation. The `translation` attribute specifies how the slider value should be translated to the animation frame rate. In this case, the slider value is linearly translated to the range 1-24, which corresponds to the frame rates supported by DecentSampler.

You can download a working example of this code [here](#). This is Example 3 in the `example-011-how-to-use-animations` folder.

Playback Mode

You can control the playback mode of an animation using the `playbackMode` attribute. This attribute specifies how the animation should play. You can use this attribute to create animations that play in different ways. For example, you could use a drop-down menu to select the playback mode of an animation.

Here is an example of an animation that is controlled by a drop-down menu:

```

<DecentSampler>
  <ui bgColor="FFADD8E6">
    <tab>
      <multiFrameImage x="450" y="80" width="64" height="64" path="Images/
↪ AnimationDemo128.png" numFrames="31" sourceFormat="vertical_image_strip" frameRate="24"
↪ playbackMode="forward_loop"/>
      <label x="275" y="50" height="30" text="Playback Mode" textColor="FF000000"
↪ textSize="16" />
      <menu x="280" y="80" width="100" height="30">
        <option name="Forward Loop">
          <binding type="control" level="ui" position="0" parameter="PLAYBACK_MODE"
↪ translation="fixed_value" translationValue="forward_loop"/>
        </option>
        <option name="Forward Once">
          <binding type="control" level="ui" position="0" parameter="PLAYBACK_MODE"
↪ translation="fixed_value" translationValue="forward_once"/>
        </option>
        <option name="Reverse Loop">
          <binding type="control" level="ui" position="0" parameter="PLAYBACK_MODE"
↪ translation="fixed_value" translationValue="reverse_loop"/>
        </option>
        <option name="Reverse Once">
          <binding type="control" level="ui" position="0" parameter="PLAYBACK_MODE"
↪ translation="fixed_value" translationValue="reverse_once"/>
        </option>
      </menu>
    </tab>
  </ui>
</DecentSampler>

```

(continues on next page)

(continued from previous page)

```

    <option name="Ping Pong Loop">
      <binding type="control" level="ui" position="0" parameter="PLAYBACK_MODE"
      ↪translation="fixed_value" translationValue="ping_pong_loop"/>
    </option>
    <option name="Stopped">
      <binding type="control" level="ui" position="0" parameter="PLAYBACK_MODE"
      ↪translation="fixed_value" translationValue="stopped"/>
    </option>
  </menu>
</tab>
</ui>
</DecentSampler>

```

In this example, the drop-down menu controls the playback mode of the animation. The `binding` tag is used to bind the menu to the `PLAYBACK_MODE` parameter of the animation. The `translation` attribute specifies how the menu value should be translated to the animation playback mode. In this case, the menu value is translated to one of the valid playback modes: `forward_loop`, `forward_once`, `reverse_loop`, `reverse_once`, `ping_pong_loop`, and `stopped`.

You can download a working example of this code [here](#). This is Example 4 in the `example-011-how-to-use-animations` folder.

Conclusion

In this guide, we have covered how to use animations in DecentSampler. You can use animations to create more compelling user interfaces and provide visual feedback for your controls. You can control animations using the `currentFrame`, `frameRate`, and `playbackMode` attributes. You can also bind animations to other components to create interactive animations.

1.13.2 Sample Mapping and Effects

How to Use Buses and Auxiliary Outputs

As of version 1.12.0, Decent Sampler has support for both audio buses and auxiliary outputs. Buses are a powerful feature that allow you to route audio to different auxiliary outputs and apply effects to the audio. This can be useful for creating complex audio routing setups and adding effects to groups of samples. This tutorial will show you how to use buses and auxiliary outputs in Decent Sampler.

Auxiliary Outputs

By default, all audio in Decent Sampler is routed to the main audio output. However, you can also route audio to secondary, auxiliary outputs. Auxiliary outputs are additional audio outputs that can be used to route audio to external effects processors or other audio devices within your DAW. Decent Sampler supports up to 16 auxiliary outputs, which can be used to create complex audio routing setups.

There are two ways that you can route audio to auxiliary outputs in Decent Sampler:

1. You can use the `outputXTarget` attributes in the `<sample>`, `<group>`, or `<groups>` elements to directly specify the audio output that the sample should be routed to. The available options are `MAIN_OUTPUT` (the main audio output, which is the default) and `AUX_STEREO_OUTPUT_1` through `AUX_STEREO_OUTPUT_16` (the auxiliary outputs).

2. You can route audio from the samples to user-defined buses, and then route audio from the buses to the auxiliary outputs. This allows you to apply effects to the audio before sending it to the auxiliary outputs.

Buses

As mentioned above, buses can be used to create a more complex mix of the samples in the sample library as well as route audio to various audio outputs. Sample library designers can specify up to 16 buses in the `<buses>` element using the `<bus>` tag. Each bus can have its own volume and audio output settings. For a full list of attributes that can be used in the `<bus>` element, see the [Buses Element](#) documentation.

Examples

Here are a few examples of how you can use buses and auxiliary outputs in Decent Sampler:

Example 1: Routing audio to an auxiliary output

In this example, we will route audio from a sample to an auxiliary output. To do this, we will use the `outputXTarget` attributes in the `<sample>` element to specify the audio output that the sample should be routed to. Here is an example of how you can route audio to `AUX_STEREO_OUTPUT_1`:

```
<group output1Target="MAIN_OUTPUT" output1Volume="1.0" output2Target="AUX_STEREO_OUTPUT_1"
  ↳" output2Volume="0.5" />
  <!-- Samples go here -->
</group>
```

In this example, the audio from the samples in the `<group>` element will be routed to the main audio output with a volume of 1.0 and to auxiliary output 1 with a volume of 0.5.

Example 2: Applying effects to audio using buses

In this example, we will apply an effect to audio using a bus. To do this, we will define a bus with an effect applied to it, and then route audio from the samples to the bus. Here is an example of how you can apply a reverb effect to audio using a bus:

```
<buses>
  <bus busVolume="0.5" output1Target="MAIN_OUTPUT" output2Target="AUX_STEREO_OUTPUT_1"
  ↳output1Volume="0.8" output2Volume="0.5">
    <effects>
      <effect type="reverb" wetLevel="1"/>
    </effects>
  </bus>
</buses>
<group output1Target="MAIN_OUTPUT" output2Target="BUS_1" output2Volume="1.0">
  <!-- Samples go here -->
</group>
```

In this example, a bus is defined with a reverb effect applied to it. The audio from the samples in the `<group>` element will be routed to the main audio output with a volume of 1.0, as well as to the bus with the reverb effect applied to it.

Conclusion

Buses and auxiliary outputs are powerful features in Decent Sampler that allow you to create complex audio routing setups and apply effects to groups of samples. By using buses and auxiliary outputs, you can create more dynamic and interesting sample libraries that take full advantage of Decent Sampler's capabilities.

How to Use Note Sequences within your Sample Libraries

As of version 1.11.1, **DecentSampler** now supports embedding note sequences with your sample libraries. This means that you can create patterns or musical motifs that can be triggered either using MIDI or via the UI. This guide will show you how to create a sample library with note sequences and how to trigger them using MIDI.

Creating a sequence by hand

In this part of the guide, we will go over how to create a sequence by hand. This is useful if you want to create a sequence that is simple, and if you want the XML for your sequence to continue to be easily edited by hand. The first thing you need to do is add a `<noteSequences>` element to your `<DecentSampler>` file:

The `<noteSequences>` element

The `<noteSequences>` element is how you specify note sequences that can be used by this playback engine. There should be exactly one `<noteSequences>` element in each `<DecentSampler>` file.

The `<noteSequences>` element can contain one or more `<sequence>` elements:

The `<sequence>` element

The `<sequence>` element has the following attributes:

- **name** (optional): An optional descriptive name for the sequence. This is only used in the sample editor UI to help you identify the sequence.
- **length** (required): The length of the sequence in beats. This is a floating point number.
- **rate** (optional): The rate at which the sequence is played. This is a floating point number. The default is 1.0.

The `<sequence>` element can contain one or more `<note>` elements:

The `<note>` element

The `<note>` element has the following attributes:

- **position** (required): The position of the note in the sequence, in beats. This is a whole number.
- **velocity** (required): The velocity of the note. This is a floating point number between 0 and 1.
- **note** (required): The MIDI note number of the note.
- **length** (required): The length of the note in beats. This is a whole number.

An example sequence might look like this:

```

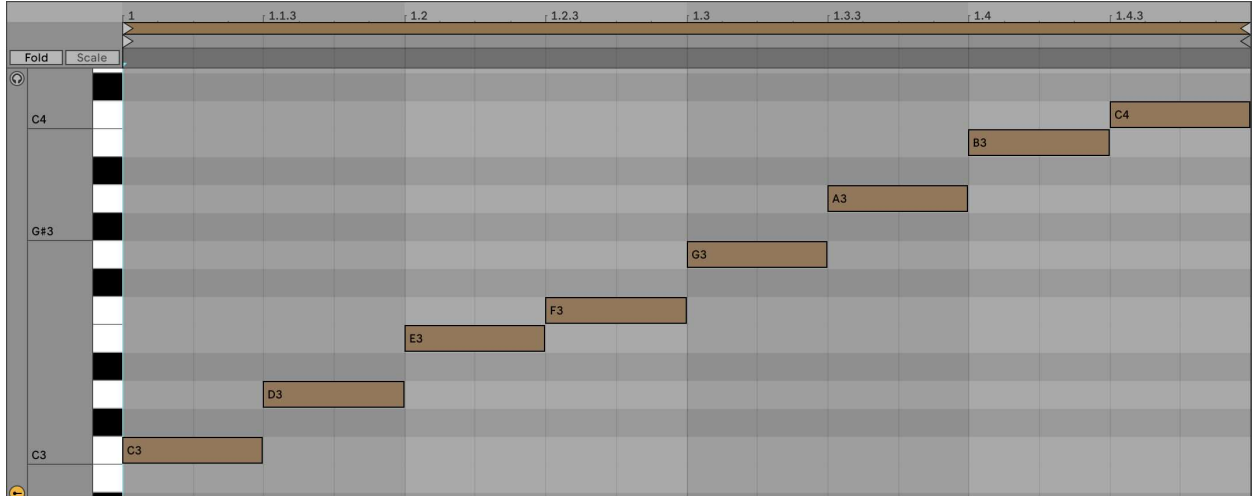
<noteSequences>
  <sequence name="Maj1Slow" length="16" rate="1">
    <note position="0" velocity="1" note="48" length="16"/>
    <note position="1" velocity="1" note="52" length="15"/>
    <note position="2" velocity="1" note="55" length="14"/>
    <note position="3" velocity="1" note="60" length="13"/>
    <note position="4" velocity="1" note="64" length="12"/>
    <note position="5" velocity="1" note="67" length="11"/>
    <note position="6" velocity="1" note="72" length="10"/>
    <note position="7" velocity="1" note="76" length="9"/>
    <note position="8" velocity="1" note="79" length="8"/>
    <note position="9" velocity="1" note="84" length="7"/>
    <note position="10" velocity="1" note="88" length="6"/>
    <note position="11" velocity="1" note="91" length="5"/>
  </sequence>
</noteSequences>

```

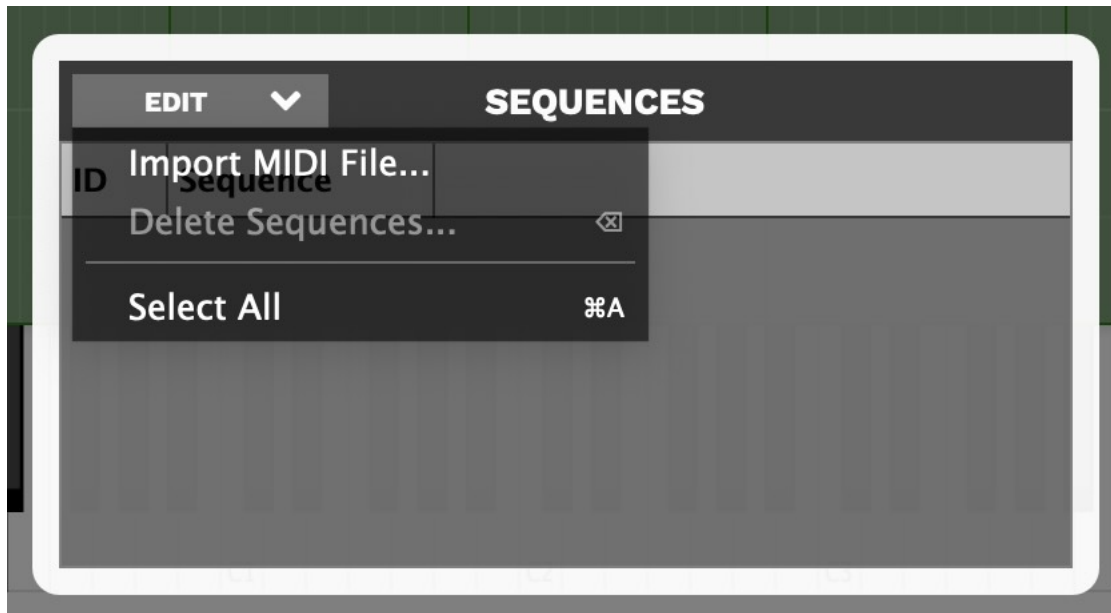
Creating a sequence using the sample editor

Another way to create sequences is via MIDI. While the Decent Sampler preset editor does not have a sequence editor built in, it does allow you to import MIDI files to create sequences. This means that you can use your favorite DAW to create sequences and then import them into Decent Sampler. This list of steps assumes that already have a preset created and that you are just looking to add sequences to it.

1. Create a MIDI sequence in your favorite DAW. Here is a sequence I created in Ableton Live:



2. Export the sequence as a MIDI file. The command for doing this will be different in every DAW. In Ableton Live, you can do this by right-clicking on the MIDI clip and selecting **Export MIDI Clip...**
3. Load your preset in Decent Sampler
4. Open the preset editor by going to **File > Developer Tools > Preset Editor...** You can also get there by hitting F12.
5. Once you're in the preset editor, choose the **File > Sequence Manager...** menu item. A little box will pop up. This is the sequence manager. It will be empty at first.
6. Click the **Edit > Import MIDI File** menu option, and select the MIDI file you created in step 2.



You will be presented with a dialog box that will ask you whether you want to quantize your sequence. Once you click OK, Decent Sampler will automatically convert the MIDI file into a sequence and add it to the sequence manager. You can then use the sequence in your preset.

7. Close the sequence manager window and save your preset.

Triggering sequences using MIDI

In order to set up your preset so that specific notes trigger specific sequences, you'll need to make use of the `<midi>` element in your `<DecentSampler>` file. Underneath the `<midi>` element, you can define any number of `<note>` elements. Each `<note>` element has the following attributes:

- **note** (required): The MIDI note number that will trigger the sequence. This can also be a range of notes, such as 24-35.
- **enabled** (optional): A boolean value that determines whether the note handler is currently functional. The default is `true`.
- **swallowNotes** (optional): A boolean value that determines whether the note handler should swallow the incoming MIDI note or pass it on to the playback engine. The default is `false`, but most people will want to set this to `true`.

Within your `<note>` element, you can define any number of bindings. The bindings we will be using will look something like this:

```
<DecentSampler pluginVersion="1" minVersion="1.11.1">
  <midi>
    <!-- We define a listener for the notes 24 through 35... -->
    <note note="24-35" enabled="true" swallowNotes="true">
      <binding enabled="true" level="instrument" type="note_sequence" seqIndex="0"
    ↪ seqLoopMode="forward" seqTriggerBehavior="midi_key" seqTransposeWithRootNote="12"
    ↪ seqTrackMidiInputVelocity="1.0" seqPlaybackRate="1"/>
    </note>
  </midi>
  <noteSequences>
```

(continues on next page)

(continued from previous page)

```

<sequence name="Maj1Slow" length="4" rate="1">
  <note position="0" velocity="1" note="48" length="1"/>
  <note position="1" velocity="1" note="52" length="1"/>
  <note position="2" velocity="1" note="55" length="1"/>
  <note position="3" velocity="1" note="60" length="1"/>
</sequence>
</noteSequences>
</DecentSampler>

```

Here is a description of all of the useful attributes for the `<binding>` element:

- **enabled** (optional): A boolean value that determines whether the binding is currently functional. The default is `true`.
- **level** (required): When triggering note sequences, this should always be `instrument`, as these all live at the instrument level.
- **type** (required): The type of binding. When triggering note sequences, this should always be `note_sequence`.
- **seqIndex** (required): The index of the sequence in the sequence manager that you want to trigger. This is a zero-based index.
- **seqFollowGlobalTempo** (optional): Whether or not the sequence should follow the global tempo. Valid values are `true` and `false`. If this is set to `false`, then playback will be hardcoded at 120BPM. This can be useful if you want to assure that sequences will always play back at the same rate regardless of the DAW clock.
- **seqLoopMode** (optional): The loop mode of the sequence. This can be `forward`, `reverse`, `random`, or `no_loop`.
- **seqTriggerBehavior** (optional): The trigger behavior of the sequence. this can be `midi_key`, `on`, or `off`. When you are triggering a sequence using a MIDI key, this should always be `midi_key`.
- **seqTransposeWithRootNote** (optional): The amount by which the sequence should be transposed when it is triggered, relative to the incoming note. For example, if you set this to 24, and the MIDI note that triggered it was 28, the sequence would be transposed up by 4 semitones, since $28 - 24 = 4$.
- **seqTrackMidiInputVelocity** (optional): A floating point value between 0 and 1 that determines whether the sequence should track the velocity of the incoming MIDI note. The default is `1.0`.
- **seqPlaybackRate** (optional): A floating point value that determines the rate at which the sequence should be played. The default is `1.0`.

You can download the example file [here](#) and try it out for yourself. You will want to look in the `example-006-working-with-note-sequences/Example 6A - Triggering Sequences Using MIDI.dspreset` file.

Triggering note sequences using the UI

In order to trigger note sequences using the UI, you will need to make use of the `<ui>`, `<tab>`, and various UI controls elements in your `<DecentSampler>` file. This tutorial assumes you already have something like this set up. If you don't, you can refer to the [<ui> element section](#) for more information.

```

<DecentSampler pluginVersion="1" minVersion="1.11.1">
  <ui>
    <tab>
      <!-- This button has two states -->
      <button x="200" y="17" width="100" height="30" style="text" parameterName=

```

(continues on next page)

(continued from previous page)

```

↪ "Sequencer Enabled" value="0" defaultValue="0">
  <!-- In the Off state, we silence any sequence that might be playing that has
↪ the identifier "sequence_button_1" -->
  <state name="Off">
    <binding level="instrument" type="note_sequence" seqIndex="0"
↪ seqTriggerBehavior="off" seqPlayerIdentifier="sequence_button_1"/>
  </state>
  <!-- In the `On` state, we tell the engine to play the sequence "Maj1Slow"
↪ (identifier as sequence with index 0).
  We will specify that it should be tracked internally using the identifier
↪ "sequence_button_1". That fact that we are specifying this identifier
  now will allow us to turn the sequence off later. -->
  <state name="On">
    <binding level="instrument" type="note_sequence" seqIndex="0" seqLoopMode=
↪ "forward" seqTriggerBehavior="on" seqTrackMidiInputVelocity="1.0" seqPlaybackRate="1"
↪ seqTranspose="12" seqPlayerIdentifier="sequence_button_1"/>
  </state>
</button>
</tab>
</ui>
<noteSequences>
  <sequence name="Maj1Slow" length="4" rate="1">
    <note position="0" velocity="1" note="48" length="1"/>
    <note position="1" velocity="1" note="52" length="1"/>
    <note position="2" velocity="1" note="55" length="1"/>
    <note position="3" velocity="1" note="60" length="1"/>
  </sequence>
</noteSequences>
</DecentSampler>

```

In this example, we have a button that has two states: Off and On. When the button is in the Off state, we silence any sequence that might be playing that has the identifier `sequence_button_1`. When the button is in the On state, we tell the engine to play the sequence `Maj1Slow`. We will specify that it should be tracked internally using the identifier `sequence_button_1`. That fact that we are specifying this identifier now will allow us to turn the sequence off later.

You can download the example file [here](#) and try it out for yourself. You will want to look in the `example-006-working-with-note-sequences/Example 6B - Triggering Sequences Using UI.dspreset` file.

Conclusion

In this guide, we went over how to create note sequences by hand and how to import them from MIDI files. We also went over how to trigger sequences using MIDI and the UI. We hope this guide has been helpful to you.

How to control parameters using tags (Example: Mic-level Knobs)

As of version 1.0.2, the best way to implement mic-level knobs is using the new sample tagging feature. It is possible to assign tags to specific samples. In this way, you can specify which type of sound they are:

```
<sample volume="0.0dB" tags="note,mic1" />
<sample volume="0.0dB" tags="rt,mic1" />
<sample volume="0.0dB" tags="note,mic2" />
<sample volume="0.0dB" tags="rt,mic2" />
```

You can also assign tags at the group level. You can also mix and match, and the tags specified at the group level will be added to the list of tags already specified at the sample level:

```
<group tags="note">
  <sample volume="0.0dB" tags="mic1" />
  <sample volume="0.0dB" tags="mic2" />
</group>
<group tags="rt">
  <sample volume="0.0dB" tags="mic1" />
  <sample volume="0.0dB" tags="mic2" />
</group>
```

Then you can make controls with bindings that reference those tags:

```
<control x="246" y="115" parameterName="MIC 1" style="linear_bar_vertical" type="float"
  ↪minValue="0" maxValue="100" value="60" width="20" height="70" trackForegroundColor=
  ↪"FFFFFF" trackBackgroundColor="FF888888">
  <binding type="amp" level="tag" identifier="mic1" parameter="AMP_VOLUME" />
</control>

<control x="346" y="115" parameterName="MIC 2" style="linear_bar_vertical" type="float"
  ↪minValue="0" maxValue="100" value="60" width="20" height="70" trackForegroundColor=
  ↪"FFFFFF" trackBackgroundColor="FF888888">
  <binding type="amp" level="tag" identifier="mic2" parameter="AMP_VOLUME" />
</control>
```

How to do voice-muting for drums

Voice muting makes use of the tags functionality, these are text labels that you can use to identify samples or groups of samples. You start by adding tags to all of your groups (you can also add them to individual samples if you'd like). Next, you add a `silencedByTags` attribute to groups or sample elements that you want to be silenced by other samples. When a sample with a tag matching one of the tags in the `silencedByTags` is played, it will silence the current sample (or group).

Here's an example:

```
<DecentSampler>
  <groups>
    <group tags="hihat" silencedByTags="hihat" silencingMode="fast">
      <!-- Your samples go here. -->
    </group>
  </groups>
</DecentSampler>
```

Note the use of the `silencingMode` attribute (a value of “fast” means we immediately silence, whereas “normal” means we trigger the ADSR release phase). Alternatively, you can use the `silencingDecay` attribute to specify an exact fade-out time in seconds. For example, `silencingDecay="0.02"` creates a 20ms fade-out when hi-hats are muted, which can sound more natural than an immediate cutoff.

How to implement true legato

Let’s walk through how to build a basic true legato instrument. Legato instruments generally consist of either two or three groups:

1. First, there’s the initial looping sustain sample that will get played when the note is first pressed down.
2. Then there is the legato transition sample that will get played when a second note gets pressed
3. (optional) Depending on the implementation, there may be a third looping sustain group that gets played after the legato transition sample plays. In such cases, this third group usually contains the same samples as were used in group 1.

Step 1: Voice muting

Before we get into legato, let’s talk about voice muting. This is the behavior wherein one set of samples causes another set of samples to stop playing. This can be desirable in situations such as legato instruments where two samples should not be sounding at the same time.

In Decent Sampler, voice-muting makes use of tags. These are text labels that you can use to identify samples or groups of samples. You can add a `silencedByTags` attribute to groups or sample elements. This consists of a comma-separated list of tags that specify which samples should silence the current samples. When a sample with a tag matching one of the tags in the `silencedByTags` is played, it will silence the current sample (or group). Here’s an example:

```
<DecentSampler>
  <groups>
    <group tags="sustain" silencedByTags="legato" silencingMode="normal" release="1.0"
    <!-- Your sustain samples go here. -->
  </group>
    <group tags="legato" silencedByTags="legato" silencingMode="normal" release="1.0"
    <!-- Your legato samples go here. -->
  </group>
</groups>
</DecentSampler>
```

In the above scenario, if a legato sample is matched, any sample that might be playing from the “sustain” group will be stopped.

It’s also worth mentioning the `silencingMode` attribute (a value of `fast` means we immediately silence that sample, whereas `normal` means we trigger the ADSR release phase). Alternatively, you can use the `silencingDecay` attribute to specify an exact fade-out time in seconds, which overrides the `silencingMode` behavior. For example, `silencingDecay="0.05"` creates a 50ms fade-out when voices are silenced.

Step 2: Legato

In order to specify which samples should be triggered first, we use the `trigger` attribute. This can that can be added to the `<group>` or individual `<sample>` tags. The default value is `attack`, but there are two useful new values:

- **first**: This value means that this sample will only trigger if no other notes are playing
- **legato**: This value means that this sample will only trigger if other notes are already playing

Here is an example of how this might look in conjunction with our example from above:

```
<DecentSampler>
  <groups>
    <group trigger="first" tags="sustain" silencedByTags="legato" silencingMode=
↳ "normal" release="1.0">
      <!-- Your sustain samples go here. -->
    </group>
    <group trigger="legato" tags="legato" silencedByTags="legato" silencingMode=
↳ "normal" release="1.0">
      <!-- Your legato samples go here. -->
    </group>
  </groups>
</DecentSampler>
```

Step 3: Specifying previous notes

When creating a legato instrument, it is often essential to limit which legato transition gets played based on which note we are transition from. This can be achieved using either the `previousNote` or the `legatoInterval` attributes. The `previousNote` attribute causes the engine to only play this sample if the previously triggered note matches the specified note. Example usage:

```
<sample path="Samples/LV_Legato_F2_G2.wav" rootNote="G2" loNote="G2" hiNote="G2"
↳ previousNote="F2" start="43000" />
<sample path="Samples/LV_Legato_G2_A2.wav" rootNote="A2" loNote="A2" hiNote="A2"
↳ previousNote="G2" start="43000" />
```

The `legatoInterval` attribute causes the engine to only play the sample if distance between the current note and the previously triggered note is exactly this semitone distance. For example, if the note for which this sample is being triggered is a C3 and the `legatoInterval` is set to -2, then the sample will only play if the previous note was a D3 because D3 minus two semitones equals C3.

Step 4: Polyphony

In legato instruments, it is sometimes useful to limit polyphony for a specific sample or set of samples. This is achieved using tags. At the top-level of your file, you can specify a element as follows:

```
<DecentSampler>
  <groups>
    <group tags="some-tag" >
      <!-- ... -->
    </group>
  </groups>
  <tags>
    <tag name="some-tag" polyphony="1" />
  </tags>
</DecentSampler>
```

Putting it all together

Here is a full example of a legato instrument:

```
<?xml version="1.0" encoding="UTF-8"?>

<DecentSampler pluginVersion="1">
  <groups>
```

(continues on next page)

(continued from previous page)

```

<group tags="sustain" silencedByTags="legato" trigger="first"
    silencingMode="normal" release="0.3">
  <sample path="Samples/LV_Sustain_D3.wav" rootNote="D3" loNote="C3" hiNote="D3"/>
  <sample path="Samples/LV_Sustain_E3.wav" rootNote="E3" loNote="D#3" hiNote="E3"/>
  <!-- More samples go here -->
</group>
<group tags="legato" silencedByTags="legato" trigger="legato"
    silencingMode="normal" attack="0.1" decay="0.2" sustain="0" release="1">
  previousNote="C#3"/>
  <sample path="Samples/LV_Legato_D3_E3.wav" rootNote="E3" loNote="E3"
    hiNote="E3" start="43000" previousNote="D3"/>
  <sample path="Samples/LV_Legato_E3_F3.wav" rootNote="F3" loNote="F3"
    hiNote="F3" start="43000" previousNote="E3"/>
  <!-- More samples go here -->
</group>
<group tags="legato-tails" silencedByTags="legato-tails" trigger="legato"
    attack="0.3" silencingMode="normal" release="0.3" start="5000">
  <sample path="Samples/LV_Sustain_D3.wav" rootNote="D3" loNote="C3" hiNote="D3"/>
  <sample path="Samples/LV_Sustain_E3.wav" rootNote="E3" loNote="D#3" hiNote="E3"/>
  <!-- More samples go here -->
</group>
</groups>
</DecentSampler>

```

How to add Keyswitches

As of DecentSampler 1.4.0, it is now possible to implement keyswitches. For a long time, it's been possible to trigger events when a MIDI continuous controller event is received: for example, using MIDI CCs we can change knob values or group volumes. Well, it is also possible to trigger events using MIDI notes as well. Here's what the setup for a MIDI note-based event mapping would look like:

```

<midi>
  <note note="11">
    <binding type="general" level="group" position="0" parameter="ENABLED"
      translation="fixed_value" translationValue="true" />
    <binding type="general" level="group" position="1" parameter="ENABLED"
      translation="fixed_value" translationValue="false" />
  </note>
  <note note="12">
    <binding type="general" level="group" position="0" parameter="ENABLED"
      translation="fixed_value" translationValue="false" />
    <binding type="general" level="group" position="1" parameter="ENABLED"
      translation="fixed_value" translationValue="true" />
  </note>
</midi>

```

In this example, MIDI note 11 turns on group 0 and turns off group 1, whereas MIDI note 12 does the opposite. Note the use of the `fixed_value` translation type.

More documentation on this can be found [here](#).

How to Use FM Synthesis (fm6op)

DecentSampler includes a built-in 6-operator FM synthesizer that implements the 32 classic algorithm topologies from the Yamaha DX7. You can use it alongside traditional samples, as a pure synthesizer, or anywhere else an `<oscillator>` element is valid.

Quick Start

Set `waveform="fm6op"` on an `<oscillator>` element and configure the FM parameters on the parent `<group>`:

```
<group fmAlgorithm="5"
  fmOp1Ratio="1.0" fmOp1Level="0.9"
  fmOp2Ratio="14.0" fmOp2Level="0.5"
  fmOp2Attack="0.001" fmOp2Decay="0.4" fmOp2Sustain="0.0" fmOp2Release="0.1"
  fmOp6Feedback="0.0"
  attack="0.001" decay="0.5" sustain="0.0" release="1.5">
  <oscillator waveform="fm6op"
    loNote="0" hiNote="127" rootNote="60"
    loVel="0" hiVel="127" volume="1.0"/>
</group>
```

Concepts

Operators

The fm6op oscillator has six *operators* numbered 1–6. Each operator is a sine-wave oscillator with:

- A **frequency ratio** (`fmOpNRatio`) — multiplies the played note’s frequency. 1.0 = fundamental, 2.0 = one octave up, 0.5 = one octave down.
- An **output level** (`fmOpNLevel`) — how much the operator contributes (0.0–1.0).
- A **feedback amount** (`fmOpNFeedback`) — how much the operator feeds its own output back into its phase input (0.0–1.0). In the DX7 architecture only one operator per algorithm physically carries the feedback path; see the note on feedback below.
- An independent **per-operator ADSR envelope** (`fmOpNAttack`, `fmOpNDecay`, `fmOpNSustain`, `fmOpNRelease`).
- An optional **DX7-compatible 4-stage rate/level envelope** (`fmOpNEgType="dx7"` plus `fmOpNEgRate1–fmOpNEgLevel4`).

Carriers and Modulators

Each algorithm divides operators into two roles:

- **Carrier** — its output is added to the final audio signal you hear.
- **Modulator** — its output is fed into the phase input of another operator, adding harmonics and timbral complexity. Modulators are not heard directly.

Algorithms

The `fmAlgorithm` attribute (1–32) selects the routing topology. A higher algorithm number generally means more carriers (brighter, additive sound). A lower number generally means deeper modulation chains (more harmonically rich FM timbres).

Some useful algorithm categories:

Algorithms	Topology	Character
1–8	Deep chains (1 or 2 carriers)	Rich, complex FM timbres — basses, pads, strings
9–19	Mixed chains + parallel	Balanced, versatile — pianos, plucks, bells
20–28	3+ carriers	Bright, additive-leaning — choir, organs, leads
29–32	4–6 carriers	Additive/organ-like — drawbar organs, rich pads

Algorithm 32 is a special case: all six operators are carriers, each producing a sine wave at its own ratio. This is pure additive synthesis.

Feedback

Each algorithm designates exactly one operator as the feedback source. In the DX7 this is always Op 6 (the `fmOp6Feedback` attribute), so `fmOp6Feedback` is the feedback parameter that works in all 32 algorithms. Setting feedback on other operators has no audible effect unless the active algorithm routes them as the feedback source.

Small feedback values (0.0–0.15) add subtle harmonics and warmth. Larger values (0.3–0.6) create bright sawtooth-like waves. Very high values (0.7–1.0) produce noise and distortion.

Detune

Each operator can be pitch-detuned using DX7-compatible detune values (`fmOpNDetune`, range -7 to +7). Detune creates subtle pitch offsets that add richness and movement to the sound:

- **0** = no detune (default)
- **Positive values** (1–7) = sharpen the pitch slightly
- **Negative values** (-7 to -1) = flatten the pitch slightly

The DX7 detune algorithm produces larger pitch offsets at lower notes and smaller offsets at higher notes, matching the classic hardware behavior. Common use cases:

- **Chorus-like thickness:** Set operators at the same ratio but slightly different detune values (e.g., Op1 detune=0, Op2 detune=2).
- **Detuned stacks:** Create evolving textures by detuning multiple carriers in algorithm 32.
- **Bell-like inharmonicity:** Slight detunes on modulators add metallic character.

Fixed Frequency Mode

By default, each operator's frequency is a **ratio** of the played note (`fmOpNMode="ratio"`). You can instead set an operator to **fixed frequency mode** (`fmOpNMode="fixed"`) and specify an absolute frequency in Hz using `fmOpNFixedFreq`:

```
<group fmAlgorithm="1"
  fmOp1Ratio="1.0" fmOp1Level="0.8"
  fmOp2Mode="fixed" fmOp2FixedFreq="987.5" fmOp2Level="0.4">
```

In this example, Op2 always oscillates at 987.5 Hz regardless of which note is played. This creates:

- **Metallic/bell timbres:** Fixed-frequency modulators add inharmonic partials.
- **Detuned octaves:** A fixed carrier at a specific frequency creates tension against the played note.
- **Special effects:** Ring modulation and clangorous textures.

Fixed frequency is often combined with ratio mode operators in the same patch for hybrid harmonic/inharmonic sounds.

Velocity Sensitivity

Each operator can respond to MIDI velocity using `fmOpNVelocitySensitivity` (range 0–7, DX7 convention):

- **0** = no velocity response (default) — operator always plays at full level.
- **1–7** = increasing velocity scaling — level varies from near-zero at vel=1 to full at vel=127.

Velocity sensitivity is most useful on:

- **Carriers:** Make the overall volume respond to playing dynamics.
- **Modulators:** Make the brightness/harmonic complexity respond to velocity — lighter touches produce simpler tones, harder strikes add more harmonics.

Example: Velocity-sensitive electric piano (algorithm 5):

```
<group fmAlgorithm="5"
  fmOp1Ratio="1.0" fmOp1Level="0.85" fmOp1VelocitySensitivity="3"
  fmOp2Ratio="14.0" fmOp2Level="0.45" fmOp2VelocitySensitivity="6"
  fmOp2Attack="0.001" fmOp2Decay="0.6" fmOp2Sustain="0.0" fmOp2Release="0.1">
  <oscillator waveform="fm6op" loNote="0" hiNote="127" rootNote="60"/>
</group>
```

Here, Op1 (carrier) has moderate velocity response (3), while Op2 (modulator) has high velocity response (6) — soft notes are mellow, loud notes are bright.

Per-Operator Envelopes

Each operator has its own ADSR envelope that shapes how that operator's contribution evolves over time:

- **Carrier envelopes** shape the timbre loudness over time (like a VCA).
- **Modulator envelopes** shape the modulation depth: high modulation at attack = bright attack transient; low modulation after decay = simpler sustained tone.

Sentinel value (–1.0 for release): If you omit a per-operator release, or set it to –1.0, the operator has no independent release — it is fully controlled by the outer `<group>` ADSR's release stage. This is the default, so you only need to specify per-operator release when you want the operator to decay independently while the note is held.

DX7-Compatible 4-Stage Operator Envelope

In addition to the standard ADSR envelope, each operator can use a 4-stage rate/level envelope that matches the parameter conventions of classic 6-operator DX7 synthesizers. This mode is activated per-operator by setting `fmOpNEgType="dx7"`.

The DX7 envelope has four segments, each defined by a **rate** (how fast the level moves) and a **target level** (where it stops):

Stage	Attributes	Meaning
Attack	<code>fmOpNEgRate1</code> , <code>fmOpNEgLevel1</code>	Rise from silence to L1 at speed R1
Decay	<code>fmOpNEgRate2</code> , <code>fmOpNEgLevel2</code>	Move from L1 to L2 at speed R2
Sustain	<code>fmOpNEgRate3</code> , <code>fmOpNEgLevel3</code>	Move toward L3 at speed R3; held while key is down
Release	<code>fmOpNEgRate4</code> , <code>fmOpNEgLevel4</code>	Move from current level to L4 at speed R4 after key-off

All rates and levels use the range **0–99**, matching standard DX7 patch banks:

- **Rate 99** = near-instant transition. **Rate 0** = extremely slow.
- **Level 99** = full amplitude. **Level 0** = silence.

Default flat envelope (instant attack, infinite sustain, instant release):

```
R1=99  L1=99  ← instant rise to full
R2=99  L2=99  ← instant move (stays at full)
R3=0    L3=99  ← rate 0: holds at L3 indefinitely while key is down
R4=99  L4=0    ← instant fall to silence after key-off
```

When `fmOpNEgType="dx7"` is set, the `fmOpNAttack`/`fmOpNDecay`/`fmOpNSustain`/`fmOpNRelease` ADSR attributes are ignored for that operator.

Example: Brass patch with DX7 envelope

Classic brass sounds have a fast attack to L1, a small dip to a slightly lower sustain level, and a moderate release. This example uses algorithm 1, with Op1 as the carrier and Op2 as a modulator with its own shape:

```
<group fmAlgorithm="1"
  attack="0.001" decay="0.0" sustain="1.0" release="0.3"
  fmOp1Ratio="1.0" fmOp1Level="0.9"
  fmOp1EgType="dx7"
  fmOp1EgRate1="99" fmOp1EgLevel1="99"
  fmOp1EgRate2="86" fmOp1EgLevel2="94"
  fmOp1EgRate3="0"  fmOp1EgLevel3="94"
  fmOp1EgRate4="60" fmOp1EgLevel4="0"
  fmOp2Ratio="1.0" fmOp2Level="0.55"
  fmOp2EgType="dx7"
  fmOp2EgRate1="99" fmOp2EgLevel1="99"
  fmOp2EgRate2="70" fmOp2EgLevel2="60"
  fmOp2EgRate3="0"  fmOp2EgLevel3="60"
  fmOp2EgRate4="55" fmOp2EgLevel4="0"
  fmOp6Feedback="0.08">
  <oscillator waveform="fm6op" loNote="0" hiNote="127" rootNote="60"
```

(continues on next page)

(continued from previous page)

```

    loVel="0" hiVel="127" volume="1.0"/>
</group>

```

DX7 rate/level values from standard patch banks can be used directly without conversion — the numbers are the same.

Per-Operator ADSR Example: DX7 E. Piano Transient

The famous DX7 Electric Piano transient comes from the modulator decaying quickly on attack while the carrier sustains:

```

<group fmAlgorithm="5"
  attack="0.001" decay="0.5" sustain="0.0" release="1.5"
  fmOp1Ratio="1.0" fmOp1Level="0.90"
  fmOp2Ratio="14.0" fmOp2Level="0.55"
  fmOp2Attack="0.001" fmOp2Decay="0.40" fmOp2Sustain="0.0" fmOp2Release="0.15"
  fmOp3Ratio="1.0" fmOp3Level="0.70"
  fmOp4Ratio="14.0" fmOp4Level="0.40"
  fmOp4Attack="0.001" fmOp4Decay="0.30" fmOp4Sustain="0.0" fmOp4Release="0.12"
  fmOp5Ratio="1.0" fmOp5Level="0.55"
  fmOp6Ratio="14.0" fmOp6Level="0.30" fmOp6Feedback="0.0"
  fmOp6Attack="0.001" fmOp6Decay="0.25" fmOp6Sustain="0.0" fmOp6Release="0.10">
  <oscillator waveform="fm6op"
    loNote="0" hiNote="127" rootNote="60"
    loVel="0" hiVel="127" volume="1.0"/>
</group>

```

Here Op6/Op5/Op4/Op3/Op2 are modulators. The modulator envelopes are short (decay=0.25–0.40, sustain=0) so they decay to zero quickly — creating the bright “tine” transient at the note start. The outer <group> ADSR shapes the overall volume fade.

Real-Time Modulation with Bindings

All FM operator parameters support real-time modulation. This lets you connect LFOs, envelopes, MIDI CCs, or UI knobs to FM parameters for expressive control.

Example: Knob controls modulation depth (morph timbre)

```

<ui width="812" height="375">
  <tab>
    <labeled-knob x="350" y="30" width="100" height="100"
      label="FM Depth" parameterName="FMDepth"
      minValue="0" maxValue="1" value="0.5">
      <binding type="general" level="group" position="0"
        parameter="OSCILLATOR_FM_OP2_LEVEL"
        translation="linear"
        translationOutputMin="0.0" translationOutputMax="1.0"/>
    </labeled-knob>
  </tab>
</ui>

```

Supported binding parameters

Parameter	Range	What it controls
OSCILLATOR_FM_ALGORITHM	1–32	Routing topology (usually set once, not modulated)
OSCILLATOR_FM_OP1_LEVEL OSCILLATOR_FM_OP6_LEVEL	... 0.0–1.0	Per-operator modulation depth / carrier level
OSCILLATOR_FM_OP1_RATIO OSCILLATOR_FM_OP6_RATIO	... Any positive value	Per-operator frequency ratio (detune, octave shifts)
OSCILLATOR_FM_OP1_FEEDBACK OSCILLATOR_FM_OP6_FEEDBACK	... 0.0–1.0	Per-operator self-feedback

See *Appendix B* for a full listing.

Classic Sound Recipes

Drawbar Organ (Algorithm 32 — all carriers)

Algorithm 32 turns the FM engine into a pure additive synthesizer. Map each operator to a harmonic of the fundamental:

```
<group fmAlgorithm="32"
  attack="0.001" decay="0" sustain="1.0" release="0.05"
  fmOp1Ratio="1.0" fmOp1Level="0.85" fmOp1Release="0.05"
  fmOp2Ratio="2.0" fmOp2Level="0.65" fmOp2Release="0.05"
  fmOp3Ratio="3.0" fmOp3Level="0.45" fmOp3Release="0.05"
  fmOp4Ratio="4.0" fmOp4Level="0.30" fmOp4Release="0.05"
  fmOp5Ratio="6.0" fmOp5Level="0.18" fmOp5Release="0.05"
  fmOp6Ratio="8.0" fmOp6Level="0.10" fmOp6Feedback="0.08" fmOp6Release="0.05">
  <oscillator waveform="fm6op" loNote="0" hiNote="127" rootNote="60"
    loVel="0" hiVel="127" volume="1.0"/>
</group>
```

Short per-operator releases (0.05 s) produce the characteristic fast key-off click of a tonewheel organ.

Bell / FM Marimba (Algorithm 5 — three 2-op pairs)

Inharmonic ratios + all-decay envelopes = percussive bell tones:

```
<group fmAlgorithm="5"
  attack="0.001" decay="0" sustain="1.0" release="20"
  fmOp1Ratio="1.0" fmOp1Level="1.0"
  fmOp1Attack="0.001" fmOp1Decay="4.0" fmOp1Sustain="0.0" fmOp1Release="2.0"
  fmOp2Ratio="3.5" fmOp2Level="0.35"
  fmOp2Attack="0.001" fmOp2Decay="0.15" fmOp2Sustain="0.0" fmOp2Release="0.1"
  fmOp3Ratio="1.0" fmOp3Level="0.70"
  fmOp3Attack="0.001" fmOp3Decay="3.0" fmOp3Sustain="0.0" fmOp3Release="1.5"
  fmOp4Ratio="3.5" fmOp4Level="0.28"
  fmOp4Attack="0.001" fmOp4Decay="0.12" fmOp4Sustain="0.0" fmOp4Release="0.09"
  fmOp5Ratio="1.0" fmOp5Level="0.50"
```

(continues on next page)

(continued from previous page)

```

    fmOp5Attack="0.001" fmOp5Decay="2.0" fmOp5Sustain="0.0" fmOp5Release="1.0"
    fmOp6Ratio="3.5" fmOp6Level="0.22" fmOp6Feedback="0.0"
    fmOp6Attack="0.001" fmOp6Decay="0.10" fmOp6Sustain="0.0" fmOp6Release="0.08">
<oscillator waveform="fm6op" loNote="0" hiNote="127" rootNote="60"
    loVel="0" hiVel="127" volume="1.0"/>
</group>

```

Note the long outer release (`release="20"`) — this keeps the voice alive while the per-operator envelopes decay naturally to silence on their own.

Choir Pad (Algorithm 19 — shared modulator with detuned carriers)

Algorithm 19 routes a single Op6 modulator into both Op5 and Op4. Setting their ratios slightly apart creates a beating, chorus-like quality:

```

<group fmAlgorithm="19"
    attack="1.0" decay="0" sustain="1.0" release="20"
    fmOp4Ratio="1.003" fmOp4Level="0.75"
    fmOp4Attack="0.90" fmOp4Decay="0.0" fmOp4Sustain="1.0" fmOp4Release="1.5"
    fmOp5Ratio="1.0" fmOp5Level="0.85"
    fmOp5Attack="0.90" fmOp5Decay="0.0" fmOp5Sustain="1.0" fmOp5Release="1.5"
    fmOp6Ratio="1.0" fmOp6Level="0.30" fmOp6Feedback="0.08"
    fmOp6Attack="0.50" fmOp6Decay="0.30" fmOp6Sustain="0.20" fmOp6Release="0.5"
    fmOp1Ratio="1.0" fmOp1Level="0.60"
    fmOp1Attack="0.80" fmOp1Decay="0.0" fmOp1Sustain="1.0" fmOp1Release="1.5"
    fmOp2Ratio="1.0" fmOp2Level="0.50"
    fmOp2Attack="0.50" fmOp2Decay="1.0" fmOp2Sustain="0.50" fmOp2Release="1.0"
    fmOp3Ratio="3.0" fmOp3Level="0.35"
    fmOp3Attack="0.30" fmOp3Decay="0.50" fmOp3Sustain="0.30" fmOp3Release="0.5">
<oscillator waveform="fm6op" loNote="0" hiNote="127" rootNote="60"
    loVel="0" hiVel="127" volume="1.0"/>
</group>

```

Tips

- **Start simple.** Pick algorithm 5 and set only Op1 (carrier) + Op2 (modulator) to non-default values. Gradually raise `fmOp2Level` from 0 to 1 to hear FM modulation build from a sine wave to a complex timbre.
- **Keep loudness consistent.** When many operators have high levels the output can clip. Lower `volume` on the `<group>` or reduce carrier levels accordingly.
- **Per-operator release tuning.** Set per-operator releases independently to make different parts of the spectrum fade at different rates — carriers fade slowly (long `release`), modulators fade faster (short `release`) for a natural brightness decay.
- **Inharmonic ratios for metallic textures.** Try ratios like 3.5, 7.07, 1.41 ($\sqrt{2}$), or other non-integers on modulators for bells, gongs, and metallic sounds.
- **Operator 6 first.** `fmOp6Feedback` is nearly always the feedback operator. Start by adjusting this first when you want to add harmonic richness.
- **Combine with samples.** A group containing `<sample>` elements can share a track with an `fm6op` group. Use the groups' `output*` routing to send them to different buses if needed.

1.13.3 Developer Tools

How to Validate a Preset File

DecentSampler includes a built-in **Validate Preset** tool in the Developer Tools menu. It reads the currently loaded .dspreset file directly from disk and reports any structural or file-reference problems—without modifying the file.

Accessing the Tool

1. Open a .dspreset file in DecentSampler.
2. Click **FILE...** to open the file menu.
3. Navigate to **Developer Tools** → **Validate Preset**.

Note: **Validate Preset** is only available for non-copy-protected sample libraries. It will not appear in the menu (or will be disabled) when a commercially encrypted preset is loaded.

What Gets Checked

1. Preset File Availability

The tool first confirms that the loaded .dspreset URL points to a real, readable file on disk. If no preset is loaded, or if the file has been moved or deleted since it was last opened, this check will fail with an error.

2. XML Validity

The entire file is parsed as XML. Any syntax error—such as an unclosed tag, illegal character, or malformed declaration—is reported with the parser’s own error message. All subsequent checks are skipped if the XML is not well-formed.

3. File Path References

The following element/attribute pairs are scanned for file path references:

Element	Attribute	Typical use
<sample>	path	Audio sample files
<effect>	irFile	Convolution reverb IR files
<image>	path	Custom image files
<ui>	bgImage	Background image
(any)	file	Any other file attribute

For each reference the validator:

- Resolves the path relative to the directory that contains the .dspreset file.
- Performs a **case-insensitive** directory and file name search, traversing each path component individually.
- Reports an **error** if the file cannot be found at all.

- Reports an **error** if the file exists on disk but with **different capitalization** than the path written in the preset. This is important because paths that work on case-insensitive file systems (macOS, Windows) will silently break on case-sensitive ones (most Linux distributions and some package-based deployments).

Reading the Report

The report dialog shows:

- The full path to the preset file on disk.
- Whether the XML parsed successfully.
- A count of errors and warnings.
- A numbered list of every issue, prefixed with [ERROR] or [WARNING], and the exact element, attribute name, and offending path string.

A clean preset produces the message:

```
Validation passed. No issues were found.
```

Common Errors and How to Fix Them

Missing file

```
ERROR: Missing file reference: <sample> @path="Samples/Cello_A3.wav"
```

The file Samples/Cello_A3.wav could not be found relative to the preset directory. Check that the file exists and that the path in the <sample> tag is correct.

Incorrect capitalization

```
ERROR: Incorrect path capitalization: <sample> @path="samples/cello_A3.wav" (disk path:
↪ "Samples/Cello_A3.wav")
```

The file exists, but the folder name is written as `samples` in the preset while the real folder on disk is `Samples`. Update the `path` attribute to match the exact capitalization on disk, otherwise the preset will fail to load on case-sensitive platforms.